

Indice per tipologia

- **Tipologia 1: Notazione asintotica** (O , Ω , Θ)
 - Esercizio 1 — 02/2026 (confronto funzioni con gli operatori asintotici)
- **Tipologia 2: Codifiche** (rappresentare oggetti come stringhe binarie)
 - Esercizio 2 — 02/2026 (codifica di $\mathbb{Q}$, unione disgiunta, grafi diretti)
- **Tipologia 3: Costruzione di macchine di Turing** (progettare una TM per un problema specifico, con δ e analisi di complessità)
 - Esercizio 2 — 05/03/2026 (*inc*: incrementare un numero binario)
 - Esercizio 1 — 09/03/2026 (invertire una stringa, 2 nastri e 1 nastro)
 - Esercizio 2 — 09/03/2026 (flip di tutti i bit)
 - Esercizio 3 — 09/03/2026 (stesso numero di 0 e 1, 2 nastri)
 - Esercizio 1 — 12/03/2026 (parità: dispari 0's, pari 1's)
 - Esercizio 2 — 12/03/2026 (pattern 0110: ogni 01 seguito da 10)
 - Esercizio 1 — 28/05/2026 (ripasso: $|w|_0 \bmod 4$, decisione e calcolo)
- **Tipologia 4: Decidibilità/Indecidibilità** (dimostrare se un problema è calcolabile o no: auto-riferimento, riduzione, o Teorema di Rice)
 - Esercizio 1 — 05/03/2026 (Halting Problem, auto-riferimento con M_{uc})
 - Esercizio 3 — 05/03/2026 (*haltstar*, riduzione da L_{halt})
 - Esercizio 4 — 05/03/2026 (classificazione L_1, L_2, L_3)
 - Esercizio 4 — 09/03/2026 (Halting Problem, riduzione dal problema UC)
 - Esercizio 3 — 12/03/2026 (riduzioni $L_h^\varepsilon \leq L_h^{00}$ e viceversa)
 - Esercizio 4 — 12/03/2026 (Teorema di Rice, classificazione L_1-L_8)
 - Esercizio 3.1 — 28/05/2026 (ripasso: “ M decide 2-SAT?”), Teorema di Rice)
- **Tipologia 5: Complessità** (dimostrare che un problema è in **P/FP/EXP**)
 - Esercizio 1 — 16/04/2026 (*minmax* $\in$ FP)
 - Esercizio 2 — 16/04/2026 (raggiungibilità in grafi $\in$ P, con dimostrazione di correttezza)
 - Esercizio (Wedding Plan Problem) — 16/04/2026 (Massimo Insieme Indipendente $\in$ FEXP)
 - Teorema $P \subsetneq EXP$ — 16/04/2026
 - Esercizio 1 — 16/04/2026 (moltiplicazione tra matrici $\in$ FP)
 - Esercizio 3.2 — 28/05/2026 (ripasso: 2-IND.SET $\in$ P)
- **Tipologia 6: NP-completezza** (certificato+verificatore, riduzioni, teoremi generali)
 - Esercizio 2 — 16/04/2026 (BIPARTITE: P, NP, battuta sul milione di dollari)
 - Esercizio 3 — 16/04/2026 (5 problemi NP: HAM.PATH, EULER.PATH, TSP, IND.SET, HALT)
 - Teorema generale NP-completo (3 parti) + Knapsack, Bin-packing, Cliques, Vertex Cover

- Esercizi — 27/04/2026 4SAT / 3CLIQUES / VARINDSET / Teorema INDSET NP-hard
- Teorema: 2SAT $\in$ P (grafo delle implicazioni, SCC)
- Esercizio 2 — 28/05/2026 (ripasso: due camion, riduzione da HAM.CYCLE)
- **Tipologia 7: PAC-Learning**
 - Teorema PAC-learning dei rettangoli (con A_{BFT})
 - Teorema: congiunzioni di letterali PAC-learnabili
 - Teorema: 3-term DNF non PAC-learnabile a meno che $RP = NP$ (con 3-colorabilità dei grafi)
- **Tipologia 8: VC-dimension** — 18/05/2026
 - Esercizio 1 (unione di 2 intervalli in $\mathbb{R}$, $VC = 4$)
 - Esercizio 2 ($\mathcal{H} \subseteq \mathcal{H}' \Rightarrow VC(\mathcal{H}) \leq VC(\mathcal{H}')$)
 - Esercizio 3 ($VC(\mathcal{H} \cup \mathcal{H}') \leq VC(\mathcal{H}) + VC(\mathcal{H}') + 1$, Lemma di Sauer-Shelah)
 - Esercizio 4 (unioni finite di intervalli, $VC = \infty$)
 - Esercizio 5 (triangoli nel piano, $VC = 7$)
 - Esercizio 6 (funzioni di parità su stringhe, $VC = d$)
 - Esercizio 4 — 28/05/2026 (ripasso: dischi su un cerchio, $VC = 3$)
 - Esercizio 5 — 28/05/2026 (ripasso: polinomi, $VC = \infty$, Teorema di interpolazione)
- **Recap teoremi** — pagina di riepilogo di tutti i 14 teoremi/lemmi del corso

02/2026

Esercizio 1

Relate le seguenti coppie di funzioni per mezzo degli operatori asintotici.

$$\begin{aligned}f_1(n) &= n \log_2 n & g_1(n) &= 10 n \log_2(\log_2 n) \\f_2(n) &= 1000 n & g_2(n) &= \frac{1}{100} n \log_2 n\end{aligned}$$

L'esercizio chiede di stabilire, per ciascuna coppia di funzioni, quale delle due cresce più velocemente al tendere di $n \rightarrow \infty$, e di esprimere questa relazione usando O , Ω e Θ . Lo strumento per farlo è il calcolo del limite del rapporto tra le due funzioni.

Soluzione

Consideriamo la prima coppia di funzioni, $f_1(n)$ e $g_1(n)$, e calcoliamo il limite del loro rapporto per $n \rightarrow \infty$:

$$\lim_{n \rightarrow \infty} \frac{f_1(n)}{g_1(n)} = \lim_{n \rightarrow \infty} \frac{n \log_2 n}{10 n \log_2(\log_2 n)} = \lim_{n \rightarrow \infty} \frac{\log_2 n}{10 \log_2(\log_2 n)} = +\infty$$

Nel passaggio centrale i due fattori n (uno al numeratore, uno al denominatore) si semplificano, lasciando solo il rapporto tra i logaritmi. Poiché $\log_2 n$ cresce più velocemente di $\log_2(\log_2 n)$ (il logaritmo di un logaritmo cresce molto più lentamente), il rapporto tende a $+\infty$.

Dato un limite $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$:

- se il risultato è 0, allora $f(n)$ cresce più lentamente di $g(n)$, cioè $f(n) \in o(g(n))$;
- se il risultato è una costante finita $c \neq 0$, allora $f(n)$ e $g(n)$ crescono con la stessa velocità, cioè $f(n) \in \Theta(g(n))$;
- se il risultato è $+\infty$, allora $f(n)$ cresce più velocemente di $g(n)$, cioè $f(n) \in \omega(g(n))$.

Nel nostro caso il limite è $+\infty$, quindi $f_1(n)$ cresce strettamente più velocemente di $g_1(n)$. Da questo fatto derivano tutte le relazioni seguenti.

Relazioni tra g_1 e f_1 (la funzione più lenta rispetto alla più veloce)

$$g_1(n) \in O(f_1(n))$$

Vero: O significa "cresce al massimo come", e g_1 non è più veloce di f_1 .

$$g_1(n) \notin \Omega(f_1(n))$$

Vero: Ω significa "cresce almeno come", ma g_1 è più lenta di f_1 , quindi non può crescere almeno quanto f_1 .

$$g_1(n) \notin \Theta(f_1(n))$$

Vero: Θ richiederebbe la stessa velocità di crescita, condizione che qui non vale.

Relazioni tra f_1 e g_1 (la funzione più veloce rispetto alla più lenta)

$$f_1(n) \in \Omega(g_1(n))$$

Vero: f_1 cresce almeno quanto g_1 (anzi, di più).

$$f_1(n) \notin O(g_1(n))$$

Vero: f_1 non cresce “al massimo come” g_1 , perché è più veloce.

$$f_1(n) \notin \Theta(g_1(n))$$

Vero: per lo stesso motivo, non condividono la stessa velocità di crescita.

Trucco mnemonico. Si può pensare a O come “ $\leq$ ”, a Ω come “ $\geq$ ” e a Θ come “ $=$ ”, riferiti alla velocità di crescita. Se f_1 “vince” su g_1 (cresce più velocemente):

- $g_1 \leq f_1$ è vera $\Rightarrow g_1 \in O(f_1)$
- $g_1 \geq f_1$ è falsa $\Rightarrow g_1 \notin \Omega(f_1)$
- $f_1 \geq g_1$ è vera $\Rightarrow f_1 \in \Omega(g_1)$
- $f_1 \leq g_1$ è falsa $\Rightarrow f_1 \notin O(g_1)$
- nessuna uguaglianza $\Rightarrow \Theta$ falsa in entrambe le direzioni

Esercizio 2

Vogliamo definire codifiche (o rappresentazioni su stringhe binarie) dei seguenti insiemi di oggetti:

- (A) i numeri razionali $\mathbb{Q}$;
- (B) l'unione disgiunta $\mathbb{N} + \{0, 1\}^*$ tra i numeri naturali e le stringhe binarie;
- (C) i grafi diretti, cioè coppie (V, E) dove V è un insieme finito di nodi ed $E \subseteq V \times V$ è un insieme di archi. Esempi:

$$G_1 = (\{a, b\}, \{(a, a), (a, b), (b, b)\})$$
$$G_2 = (\{1, 2, 3, 4\}, \{(1, 2), (2, 3), (3, 4), (4, 2)\})$$

Per ciascun insieme dobbiamo trovare un modo di rappresentare ogni suo elemento come una stringa binaria, in modo che la codifica sia *univoca* (elementi diversi devono avere codifiche diverse) e, quando possibile, *decodificabile* (dalla stringa si deve poter risalire all'elemento originale).

Soluzione

(A) Numeri razionali

Osserviamo che

$$\mathbb{Q} = \left\{ \frac{z}{n} \mid z \in \mathbb{Z}, n \in \mathbb{N}, n > 0 \right\}$$

Poiché gli elementi di $\mathbb{Q}$ sono frazioni, prima le riduciamo ai minimi termini, in modo che z ed n non abbiano fattori in comune (a parte 1). Poi codifichiamo la coppia di numeri risultante nel modo consueto, usando un separatore.

La codifica di una frazione ridotta z/n è:

$$\lfloor z/n \rfloor = \lfloor s_z m_z \# n \rfloor$$

dove s_z è il segno di z e m_z è il valore assoluto di z in binario.

Esempio. Codifichiamo $-2/3$ (già ridotta ai minimi termini):

$$\lfloor -2/3 \rfloor = \lfloor 1 \ 10 \# 11 \rfloor$$

Qui 1 è il segno (negativo), 10 è il valore assoluto di 2 in binario, # è il separatore, e 11 è 3 in binario. Il risultato è però una stringa sull'alfabeto $\{0, 1, \#\}$, che ha tre simboli: non è ancora una stringa binaria vera e propria.

Da alfabeto ternario a stringa binaria. Per trasformare una stringa sull'alfabeto $\{0, 1, \#\}$ in una stringa binaria, possiamo codificare ogni simbolo con 2 bit fissi:

$$0 \mapsto 00 \qquad 1 \mapsto 01 \qquad \# \mapsto 10$$

In questo modo la decodifica resta univoca: basta leggere i bit a due a due.

Applicando questa mappa alla sequenza di simboli 1, 1, 0, #, 1, 1 otteniamo:

$$\underbrace{01}_1 \underbrace{01}_1 \underbrace{00}_0 \underbrace{10}_{\#} \underbrace{01}_1 \underbrace{01}_1$$

cioè la stringa binaria finale 01 01 00 10 01 01.

(B) Unione disgiunta $\mathbb{N} + \{0, 1\}^*$

Per definizione, l'unione disgiunta di due insiemi X e Y è

$$X + Y = \{(x, l) \mid x \in X\} \cup \{(y, r) \mid y \in Y\}$$

Stella di Kleene (Σ^*). Dato un alfabeto (insieme di simboli) Σ , la scrittura Σ^* indica l'insieme di *tutte* le stringhe finite costruibili concatenando simboli di Σ , di qualsiasi lunghezza, inclusa la stringa vuota ε .

Esempio: $\{0, 1\}^* = \{\varepsilon, 0, 1, 00, 01, 10, 11, 000, \dots\}$, cioè l'insieme di tutte le possibili stringhe binarie.

Non è un singolo elemento: è l'intero insieme (infinito) di stringhe, usato come “tipo di oggetto” quando si costruiscono insiemi più grandi (es. $\mathbb{N} + \{0, 1\}^*$).

Dalla formula generale al caso specifico. Partiamo dalla definizione generale di unione disgiunta:

$$X + Y = \{(x, l) \mid x \in X\} \cup \{(y, r) \mid y \in Y\}$$

Nel nostro esercizio $X = \mathbb{N}$ e $Y = \{0, 1\}^*$. Sostituendo (e rinominando le variabili generiche $x \rightarrow n$ e $y \rightarrow s$ per chiarezza):

$$\mathbb{N} + \{0, 1\}^* = \{(n, l) \mid n \in \mathbb{N}\} \cup \{(s, r) \mid s \in \{0, 1\}^*\}$$

Da qui si definisce la codifica per ciascuna delle due parti:

$$\lfloor (n, l) \rfloor = 0 \lfloor n \rfloor \qquad \lfloor (s, r) \rfloor = 1 s$$

Ogni elemento dell'unione disgiunta porta con sé un'etichetta (l oppure r) che indica da quale dei due insiemi proviene: questo serve a distinguere, ad esempio, il numero naturale 5 (che potrebbe corrispondere alla stringa binaria 101) dalla stringa binaria 101 stessa, che altrimenti sarebbero indistinguibili.

Perché serve l'etichetta. Senza etichette, il numero naturale 5 (codificato in binario come 101) e la stringa binaria 101 sarebbero indistinguibili una volta codificati: la stringa finale 101 non direbbe se rappresenta “il numero cinque” o “la sequenza di simboli 1,0,1”. Per questo ogni elemento dell'unione disgiunta porta con sé un marcatore che indica da quale dei due insiemi proviene.

Perché l ed r (left/right). Nella definizione

$$X + Y = \{(x, l) \mid x \in X\} \cup \{(y, r) \mid y \in Y\}$$

X compare a sinistra del $+$ e Y a destra: da qui i nomi l (left) ed r (right). È solo una convenzione (ricorrente in informatica teorica, es. tipi “sum”/Either), non ha altro significato. Nel nostro caso, $\mathbb{N} + \{0, 1\}^*$: $\mathbb{N}$ (sinistra) è marcato con l e codificato col bit 0; $\{0, 1\}^*$ (destra) è marcato con r e codificato col bit 1.

Possiamo usare le etichette l ed r come *marcatori*, codificandole come un bit iniziale. La codifica dell'unione disgiunta $\mathbb{N} + \{0, 1\}^*$ è allora:

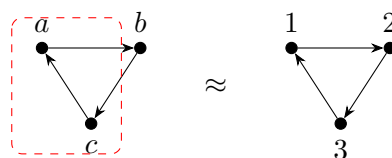
$$\lfloor (n, l) \rfloor = 0 \lfloor n \rfloor \qquad \lfloor (s, r) \rfloor = 1 s$$

Se l'elemento viene dal primo insieme ($\mathbb{N}$, marcato con l), si antepone un 0 seguito dalla codifica binaria standard del numero n . Se viene dal secondo insieme ($\{0, 1\}^*$, marcato con r), si antepone un 1 seguito direttamente dalla stringa binaria s stessa (che è già binaria, non necessita ulteriore codifica).

(C) Grafi diretti: codifica come stringhe

Un grafo diretto è una coppia $G = (V, E)$ dove V è un insieme finito di nodi ed $E \subseteq V \times V$ è un insieme di archi.

Come si può rappresentare un grafo come stringa? Il punto chiave è che due grafi *isomorfi* (cioè identici a meno di rinominare i nodi) si considerano lo stesso oggetto: **non ci interessa l'identità specifica dei nodi, solo la struttura delle connessioni.**



Per questo motivo si assume, senza perdita di generalità, che per ogni grafo $G = (V, E)$ esista $n \in \mathbb{N}$ tale che $V = \{1, 2, \dots, n\}$. In questo modo non serve codificare i “nomi” dei nodi: basta il numero n e la struttura degli archi.

Metodo 1: matrice di adiacenza.

Si rappresenta E con una matrice $n \times n$ dove

$$M_{ij} = \begin{cases} 1 & \text{se } (i, j) \in E \\ 0 & \text{se } (i, j) \notin E \end{cases}$$

La codifica del grafo è n , seguito da un separatore $\#$, seguito da tutte le righe della matrice concatenate in fila (riga per riga):

$$\lfloor G \rfloor = \lfloor n \rfloor \# M_{11} M_{12} \dots M_{1n} \\ M_{21} \dots M_{2n} \dots M_{nn}$$

Applicazione a $G_1 = (\{a, b\}, \{(a, a), (a, b), (b, b)\})$.



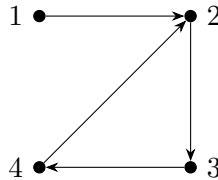
Rinominiamo $a \rightarrow 1, b \rightarrow 2$, quindi $n = 2$. Gli archi diventano $(1, 1), (1, 2), (2, 2)$. La matrice è:

$$M = \begin{pmatrix} M_{11} & M_{12} \\ M_{21} & M_{22} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$$

Quindi:

$$\lfloor G_1 \rfloor = \lfloor 2 \rfloor \# 1101 = 10 \# 1101$$

Applicazione a $G_2 = (\{1, 2, 3, 4\}, \{(1, 2), (2, 3), (3, 4), (4, 2)\})$.



Qui $n = 4$. La matrice è:

$$M = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix}$$

Concatenando le righe: 0100 0010 0001 0100. Quindi:

$$\lfloor G_2 \rfloor = \lfloor 4 \rfloor \# 0100001000010100 = 100 \# 0100001000010100$$

Metodo 2: liste di adiacenza.

Quando il numero di archi è asintoticamente minore di $|V|^2$ ("Quanti archi ci sono rispetto al massimo possibile?"), conviene rappresentare, per ogni nodo, solo la lista dei nodi raggiungibili in un passo, invece di scrivere anche tutti gli zeri della matrice (sprecati). La struttura è: n , un separatore $\#$, e poi per ciascun nodo $1, \dots, n$ la lista dei suoi vicini (separati tra loro da un simbolo diverso, es. $@$); le liste dei vari nodi sono a loro volta separate da $\#$.

Applicazione a G_1 . Il nodo 1 (ex a) raggiunge $\{1, 2\}$ (archi $(1, 1)$ e $(1, 2)$); il nodo 2 (ex b) raggiunge $\{2\}$ (arco $(2, 2)$). Quindi:

$$\lfloor (V, E) \rfloor = \lfloor 2 \rfloor \# \lfloor 1 \rfloor @ \lfloor 2 \rfloor \# \lfloor 2 \rfloor = 10 \# 1 @ 10 \# 10$$

Applicazione a G_2 . Il nodo 1 raggiunge $\{2\}$; il nodo 2 raggiunge $\{3\}$; il nodo 3 raggiunge $\{4\}$; il nodo 4 raggiunge $\{2\}$. Ogni lista ha un solo elemento, quindi non serve il separatore @ all'interno:

$$\lfloor (V, E) \rfloor = \lfloor 4 \rfloor \# \lfloor 2 \rfloor \# \lfloor 3 \rfloor \# \lfloor 4 \rfloor \# \lfloor 2 \rfloor = 100 \# 10 \# 11 \# 100 \# 10$$

Come si fa: Metodo 2 passo per passo, con G_1

Ricordiamo $G_1 = (\{a, b\}, \{(a, a), (a, b), (b, b)\})$, rinominato $a \rightarrow 1, b \rightarrow 2$, quindi $n = 2$.

Passo 1: per ogni nodo, trova chi raggiunge. Gli archi (rinominati) sono $(1, 1)$, $(1, 2)$, $(2, 2)$.

- Quali nodi raggiungo dal nodo 1? Trovo $(1, 1)$ e $(1, 2) \Rightarrow$ il nodo 1 raggiunge $\{1, 2\}$.
- Quali nodi raggiungo dal nodo 2? Trovo solo $(2, 2) \Rightarrow$ il nodo 2 raggiunge $\{2\}$.

Passo 2: codifica la lista di ciascun nodo. Il nodo 1 ha **due** vicini, quindi qui serve il separatore @; il nodo 2 ne ha uno solo, quindi non serve.

- nodo 1: vicini $\{1, 2\} \rightarrow \lfloor 1 \rfloor @ \lfloor 2 \rfloor = 1 @ 10$
- nodo 2: vicino $\{2\} \rightarrow \lfloor 2 \rfloor = 10$

Passo 3: metti insieme le liste dei vari nodi.

$$\text{lista}(1) \# \text{lista}(2) = 1 @ 10 \# 10$$

Passo 4: aggiungi n all'inizio.

$$\lfloor (V, E) \rfloor = \underbrace{10}_{n=2} \# \underbrace{1 @ 10}_{\text{lista nodo 1}} \# \underbrace{10}_{\text{lista nodo 2}}$$

Risultato finale.

$$\lfloor (V, E) \rfloor = 10 \# 1 @ 10 \# 10$$

05/03/2026

Esercizio 1

Dimostrare l'indcidibilità dell'Halting Problem, cioè il fatto che la funzione

$$\text{halt}(\alpha, x) = \begin{cases} 1 & \text{se } M_\alpha(x) \text{ termina} \\ 0 & \text{altrimenti} \end{cases}$$

non è calcolabile.

L'Halting Problem chiede: *esiste un algoritmo che, dato un programma qualsiasi e un suo input, riesca sempre a dirci se quel programma si fermerà (terminerà) oppure andrà avanti per sempre (loop infinito)?*

Il problema chiede se esiste una funzione calcolabile $halt(\alpha, x)$ che, dati α (la descrizione di una macchina) e x (un input per quella macchina), restituisca sempre correttamente:

$$halt(\alpha, x) = \begin{cases} 1 & \text{se } M_\alpha(x) \text{ termina} \\ 0 & \text{se } M_\alpha(x) \text{ non termina} \end{cases}$$

Il risultato (dimostrato da Turing nel 1936) è che una tale funzione **non esiste**: non è un problema difficile o non ancora risolto, è stato dimostrato matematicamente che è impossibile che esista, per qualsiasi programma si possa scrivere.

Soluzione

La dimostrazione procede per assurdo: supponiamo che $halt$ sia calcolabile da una macchina M_{halt} , e costruiamo con essa una macchina M_{uc} che, applicata a se stessa, porta a una contraddizione logica in entrambi i casi possibili (termina / non termina). Questo mostra che M_{halt} non può esistere.

Costruzione di M_{uc} .

Assumiamo per assurdo che esista M_{halt} che calcola $halt$. Costruiamo, usando M_{halt} come sottoprogramma, la macchina M_{uc} che, su input α , procede così:

```
def M_uc(alpha):
    x = M_halt(alpha, alpha)    # M_alpha termina su input alpha?
    if x == 0:
        return 1
    else:
        b = M(alpha, alpha)      # simula davvero M_alpha su input alpha
        return not(b)
```

In parole: M_{uc} chiede prima a M_{halt} se M_α terminerebbe eseguita su se stessa (input α). Se la risposta è “no”, M_{uc} restituisce 1. Se la risposta è “sì”, M_{uc} esegue davvero $M_\alpha(\alpha)$, ottiene un risultato b , e restituisce la negazione di b .

Perché serve la negazione. Se il codice restituisse semplicemente b (senza negarlo), applicando M_{uc} a se stessa (input u , la descrizione di M_{uc}): se $M_{uc}(u)$ termina, allora $b = M(u, u)$ è esattamente il valore di $M_{uc}(u)$ (perché $M(u, u)$ simula proprio M_{uc} su u). Il codice restituirebbe b , quindi:

$$M_{uc}(u) = b = M_{uc}(u)$$

Una tautologia, vera banalmente: nessuna contraddizione, la macchina sarebbe perfettamente coerente con se stessa e la dimostrazione non funzionerebbe.

Con la negazione, invece, si ottiene $M_{uc}(u) = \text{not}(b) = \text{not}(M_{uc}(u))$: il valore restituito è per definizione diverso da se stesso, il che è impossibile per un valore booleano. È questa la contraddizione che serve alla dimostrazione.

Il passo cruciale: applichiamo M_{uc} a se stessa.

Sia u la descrizione della macchina M_{uc} (ogni macchina ha una propria descrizione, quindi anche M_{uc} ne ha una). Chiediamoci cosa succede eseguendo $M_{uc}(u)$: ci sono solo due casi possibili.

Caso 1: $M_{uc}(u)$ non termina.

Allora $M_{halt}(u, u)$ risponderebbe $x = 0$. Ma per il codice di M_{uc} , se $x = 0$ allora M_{uc} restituisce 1, cioè **termina**. Questo contraddice l'ipotesi che non terminasse.

Caso 2: $M_{uc}(u)$ termina.

Allora $M_{halt}(u, u)$ risponde $x = 1$. A quel punto M_{uc} esegue davvero $b = M(u, u)$, cioè simula se stessa su se stessa, ottenendo esattamente il valore che $M_{uc}(u)$ restituisce, cioè b . Ma poi M_{uc} restituisce $\text{not}(b)$. Quindi il risultato finale è, allo stesso tempo, b e $\text{not}(b)$: cioè $b = \text{not}(b)$, impossibile per qualunque valore booleano.

In **entrambi** i casi si arriva a una contraddizione.

Conclusione.

M_{uc} , così come costruita, non può esistere in modo coerente. Ma è stata costruita usando M_{halt} come unico ingrediente esterno: se M_{halt} esistesse, M_{uc} esisterebbe. Poiché M_{uc} non può esistere, M_{halt} non può esistere. Quindi *halt* non è calcolabile.

Esercizio 2

Mostrare che la funzione $inc : \mathbb{N} \rightarrow \mathbb{N}$ tale che $inc(x) = x + 1$ può essere calcolata da una macchina di Turing, e che tale macchina lavora in tempo lineare.

Cos'è una macchina di Turing. Un modello astratto di calcolo composto da:

- un **nastro** infinito, diviso in caselle, su cui si possono scrivere simboli (es. 0, 1, casella vuota $\square$);
- una **testina** che legge/scrive il simbolo nella casella corrente e si muove di una casella a sinistra (L), a destra (R), oppure resta ferma (S);
- un insieme finito di **stati** (una specie di “stato mentale” interno, es. $q_0, q_1, \dots$), che rappresenta in che fase del calcolo ci si trova.

Ad ogni passo, la macchina guarda lo stato corrente e il simbolo letto, e una **funzione di transizione** δ dice cosa fare: che simbolo scrivere, in che nuovo stato passare, e da che parte muovere la testina. Il processo si ripete finché non si raggiunge uno stato di arresto (q_{halt}): a quel punto il contenuto del nastro è il risultato del calcolo.

È un modello così semplice da permettere di ragionare rigorosamente su cosa un algoritmo può o non può fare, ma si crede (Tesi di Church-Turing) che sia potente quanto qualsiasi computer reale o linguaggio di programmazione esistente.

Soluzione

La codifica usata scrive il numero sul nastro con il bit **meno significativo** subito dopo il marcatore $\triangleright$ (cioè “al contrario” rispetto alla notazione normale). Questo evita di dover prima cercare la fine del numero: il bit da cui partire per l'incremento è già lì, appena dopo il marcatore.

Legenda dei simboli.

$\triangleright$	il “muro” a sinistra: segna dove inizia il nastro
$\square$	casella vuota (non c'è ancora nulla scritto lì)
0, 1	le cifre binarie vere e proprie
q_{init}	stato iniziale: “sto appena partendo”
q_0	stato “sto scorrendo verso destra, propagando il riporto”
q_1	stato “ho finito, sto tornando indietro verso l'inizio”
q_{halt}	stato finale: “ho finito tutto, mi fermo”
R	muovi la testina di una casella a destra
L	muovi la testina di una casella a sinistra
S	resta ferma (<i>Stay</i>)
δ	la “tabella delle regole”: dice cosa fare in base a stato+simbolo letto

Come si legge una regola:

$$(q, \text{simbolo}) \mapsto (q', \text{simbolo}', M)$$

si legge: “se sono nello stato q e leggo simbolo, scrivo simbolo', passo allo stato q' , e mi muovo in direzione M ”.

Definizione della macchina.

$$Q = \{q_{init}, q_0, q_1, q_{halt}\} \quad \Gamma = \{\triangleright, \square, 0, 1\}$$

La funzione di transizione δ :

$$\begin{aligned} (q_{init}, \triangleright) &\mapsto (q_0, \triangleright, R) \\ (q_0, 0) &\mapsto (q_1, 1, L) \\ (q_0, 1) &\mapsto (q_0, 0, R) \\ (q_0, \square) &\mapsto (q_1, 1, L) \\ (q_1, 0) &\mapsto (q_1, 0, L) \\ (q_1, 1) &\mapsto (q_1, 1, L) \\ (q_1, \triangleright) &\mapsto (q_{halt}, \triangleright, S) \\ (q_{halt}, \triangleright) &\mapsto (q_{halt}, \triangleright, S) \end{aligned}$$

Come funziona.

Fase 1 (stato q_0): propaga il riporto scorrendo verso destra. Partendo dal bit meno significativo (subito dopo $\triangleright$):

- se si legge 1: si scrive 0 e si avanza a destra (il riporto si propaga al bit successivo, come nella somma in colonna);
- se si legge 0: si è trovato il primo bit che assorbe il riporto — si scrive 1 e si passa in q_1 , tornando indietro;
- se si legge $\square$ (il numero era tutto di 1, es. 1111): si scrive un nuovo 1 (il numero si allunga di una cifra) e si passa in q_1 .

Fase 2 (stato q_1): torna indietro fino al marcatore. Si scorre a sinistra senza modificare nulla, finché non si ritrova $\triangleright$, poi ci si ferma in q_{halt} .

Simulazione passo per passo

Il compito. Vogliamo una macchina che calcoli $x+1$. Prendiamo un esempio concreto: $x = 12$.

Passo 0: prepariamo il nastro.

12 in binario normale è 1100. Lo scriviamo al contrario (convenzione di questo esercizio) e mettiamo il muro $\triangleright$ prima:

Nastro iniziale: $\triangleright 0 0 1 1 \square \square \dots$

(le caselle dopo l'ultima cifra sono vuote, $\square$, semplicemente perché non c'è ancora nulla scritto lì)

La simulazione, passo per passo

Immaginiamo la testina come una piccola freccia $\downarrow$ che indica su quale casella ci troviamo.

PASSO 1.

Stato attuale: q_{init} . La testina è sulla prima casella:

$\triangleright 0 0 1 1$
 $\downarrow$

Leggiamo il simbolo sotto la testina: è $\triangleright$. Cerchiamo nella tabella δ la regola che inizia con $(q_{init}, \triangleright)$. La troviamo:

$$(q_{init}, \triangleright) \mapsto (q_0, \triangleright, R)$$

Questo ci dice tre cose: (1) scrivi $\triangleright$ al posto di quello che c'era (non cambia nulla, si riscrive lo stesso simbolo); (2) passa allo stato q_0 ; (3) muovi la testina a destra (R).

PASSO 2.

Ora siamo nello stato q_0 , e la testina si è spostata di una casella a destra:

$$\begin{array}{cccc} \triangleright & 0 & 0 & 1 & 1 \\ & \downarrow & & & \end{array}$$

Leggiamo il simbolo sotto la testina: è 0 (la prima cifra del numero). Cerchiamo la regola $(q_0, 0)$:

$$(q_0, 0) \mapsto (q_1, 1, L)$$

Cioè: scrivi 1 al posto dello 0 che c'era, passa allo stato q_1 , muovi a sinistra (L). Il nastro ora diventa:

$$\triangleright 1 0 1 1$$

(la prima cifra, che era 0, è diventata 1 — questo è esattamente il “flip” che serve per fare $+1$: se l'ultimo bit era 0, basta farlo diventare 1 e abbiamo finito con l'incremento vero e proprio)

PASSO 3.

Siamo nello stato q_1 , e ci siamo mossi a sinistra, tornando sulla casella del marcatore:

$$\begin{array}{cccc} \triangleright & 1 & 0 & 1 & 1 \\ & \downarrow & & & \end{array}$$

Leggiamo: è $\triangleright$. Cerchiamo la regola $(q_1, \triangleright)$:

$$(q_1, \triangleright) \mapsto (q_{halt}, \triangleright, S)$$

Cioè: scrivi $\triangleright$ (invariato), passa allo stato q_{halt} , resta ferma (S).

FINE.

Siamo in q_{halt} : la macchina si ferma. Il nastro finale è:

$$\triangleright 1 0 1 1$$

Se lo riconvertiamo al contrario per leggerlo normalmente: 1101, cioè 13. Esattamente $12 + 1$.
✓

Un esempio con più “lavoro” da fare: $x = 15$

Qui il numero è 1111 (tutti 1), quindi al contrario è ancora 1111 (per simmetria). Cosa succede?

- **Passo 1:** $(q_{init}, \triangleright) \rightarrow$ vai in q_0 , muovi a destra.
- **Passo 2:** leggo 1 $\rightarrow$ regola $(q_0, 1) \mapsto (q_0, 0, R)$: scrivo 0, **resto in** q_0 (non passo a q_1 !), muovo a destra. Il nastro diventa $\triangleright 0 1 1 1$.
- **Passo 3:** leggo di nuovo 1 $\rightarrow$ stessa regola: scrivo 0, resto in q_0 , muovo a destra. Nastro: $\triangleright 0 0 1 1$.
- **Passo 4:** leggo 1 $\rightarrow$ stessa regola ancora: nastro diventa $\triangleright 0 0 0 1$.

- **Passo 5:** leggo l'ultimo 1 $\rightarrow$ stessa regola: nastro diventa $\triangleright 0000$, mi muovo a destra.
- **Passo 6:** ora sono oltre l'ultima cifra, la casella è vuota, $\square$! Regola $(q_0, \square) \mapsto (q_1, 1, L)$: scrivo un nuovo 1 proprio lì (il numero si allunga), passo in q_1 , torno a sinistra. Nastro: $\triangleright 00001$.
- **Passi 7–10:** sono in q_1 , scorro a sinistra leggendo 0, 0, 0, 0 — regola $(q_1, 0) \mapsto (q_1, 0, L)$: non cambio nulla, continuo a muovermi a sinistra.
- **Passo 11:** trovo $\triangleright \rightarrow$ mi fermo in q_{halt} .

Risultato: $\triangleright 00001$, cioè (al contrario) $10000 = 16$. Esattamente $15 + 1$. ✓

Perché è tempo lineare.

Cosa vuol dire “tempo lineare”.

Tempo: quante volte si muove la testina prima che la macchina si fermi. Ogni lettura+scrittura+spostamento è un passo, e contiamo quanti passi servono in totale.

n : la lunghezza dell'input, cioè quante cifre ha il numero da incrementare.

Tempo lineare: il numero di passi cresce nella **stessa proporzione** di n . Se n raddoppia, raddoppiano (circa) anche i passi; se n diventa dieci volte più grande, i passi diventano dieci volte più numerosi. Non c'è nessuna crescita “esplosiva”: il legame tra lunghezza dell'input e numero di passi è diretto e proporzionale.

La nostra macchina per *inc* funziona proprio così: scorre le n cifre del numero una volta (andata, propagando il riporto) e le riscalda una seconda volta (ritorno, fino al marcatore). Il numero di passi è quindi proporzionale a n : per questo si dice che la macchina lavora in **tempo lineare**, cioè $O(n)$.

Sia n la lunghezza del numero in ingresso. Nel caso peggiore:

- la **Fase 1** scorre tutte le n cifre (se sono tutte 1) prima di fermarsi: al massimo $n + 1$ passi;
- la **Fase 2** ripercorre lo stesso tragitto all'indietro: al massimo $n + 1$ passi.

In totale, al massimo $2n + O(1)$ passi, cioè $O(n)$: **tempo lineare** nella lunghezza dell'input.

Esercizio 3

Consideriamo la funzione *haltstar* definita come segue:

$$\text{haltstar}(s) = \begin{cases} 1 & \text{se } s = [\alpha, x_1, \dots, x_n] \text{ e } M_\alpha(x_1) \text{ termina} \wedge \dots \wedge M_\alpha(x_n) \text{ termina} \\ 0 & \text{se } s = [\alpha, x_1, \dots, x_n] \text{ e esiste } i \in \{1, \dots, n\} \text{ tale che } M_\alpha(x_i) \text{ non termina} \end{cases}$$

haltstar è calcolabile o incalcolabile? Dimostrare la propria affermazione.

Soluzione

Dimostriamo che *haltstar* è incalcolabile mostrando che $L_{haltstar}$ è indecidibile. A differenza dell'Esercizio 1 (dove abbiamo usato l'auto-riferimento), qui usiamo una tecnica diversa: la **riduzione** da un problema già noto come indecidibile, cioè L_{halt} .

L'idea generale della riduzione.

L'idea, in una frase sola: se riuscissimo a “trasformare” ogni domanda su *halt* in una domanda equivalente su *haltstar*, allora “risolvere” *haltstar* ci permetterebbe di risolvere anche *halt*, ma sappiamo che *halt* non si può risolvere, quindi nemmeno *haltstar* si può risolvere.

Per dimostrare che un problema L_B è indecidibile, sapendo già che L_A è indecidibile, basta costruire una funzione φ **calcolabile** tale che:

$$s \in L_A \iff \varphi(s) \in L_B$$

Perché funziona: se L_B fosse decidibile, si potrebbe decidere anche L_A (si calcola $\varphi(s)$, che è facile, e si chiede a un ipotetico decisore di L_B se $\varphi(s) \in L_B$). Ma L_A non è decidibile: contraddizione. Quindi L_B non può essere decidibile.

Applicazione al nostro caso: $L_A = L_{halt}$, $L_B = L_{haltstar}$.

Dobbiamo costruire $\varphi : \{0, 1\}^* \rightarrow \{0, 1\}^*$ tale che:

- φ è calcolabile
- $s \in L_{halt} \iff \varphi(s) \in L_{haltstar}$

Definiamo φ come segue:

$$[(\alpha, x)] \xrightarrow{\varphi} [\alpha, x]$$

dove $[(\alpha, x)]$ è la codifica di una coppia (il formato di L_{halt} : un programma più un input), e $[\alpha, x]$ è la codifica di una tupla con $n = 1$ (il formato di $L_{haltstar}$: un programma più una lista di input, qui di lunghezza 1).

φ non “calcola” nulla di interessante: si limita a cambiare formato di codifica, da “coppia” a “tupla di lunghezza 1”. Per questo è ovviamente calcolabile: è pura manipolazione di stringhe (decodificare la coppia e ricodificarla come tupla).

Perché vale l'equivalenza.

$$\begin{aligned} s \in L_{halt} &\iff s = [\alpha, x], M_\alpha(x) \text{ termina} \\ &\iff \varphi(s) = [\alpha, x], M_\alpha(x) \text{ termina} \\ &\iff \varphi(s) = [\alpha, x_1, \dots, x_n] \wedge n = 1 \wedge M_\alpha(x_i) \text{ termina } \forall i \in \{1, \dots, n\} \\ &\iff \varphi(s) \in L_{haltstar} \end{aligned}$$

Il passaggio chiave è il terzo: con $n = 1$, “termina su tutti gli elementi della lista” equivale banalmente a “termina su quell’unico elemento”. Sono esattamente la stessa condizione.

Conclusione.

Poiché φ è calcolabile e vale l’equivalenza sopra, abbiamo mostrato $L_{halt} \leq_m L_{haltstar}$ (riduzione many-one). Dato che L_{halt} è indecidibile (Esercizio 1), anche $L_{haltstar}$ deve esserlo. Quindi $haltstar$ non è calcolabile.

Confronto con il metodo dell’Esercizio 1.

- **Esercizio 1 (auto-riferimento/diagonalizzazione):** si costruisce una macchina che, applicata a se stessa, porta a una contraddizione logica diretta (in entrambi i casi possibili: termina / non termina). Non si parte da nessun risultato precedente: la contraddizione nasce dalla struttura stessa della macchina costruita.
- **Esercizio 3 (riduzione):** si parte da un problema **già** noto come indecidibile (L_{halt}) e si costruisce una trasformazione φ calcolabile che porta istanze del problema noto in istanze del nuovo problema, preservando l’appartenenza al linguaggio. La contraddizione emerge indirettamente: se il nuovo problema fosse decidibile, lo sarebbe anche quello vecchio, che sappiamo non esserlo.

La riduzione è in genere **più semplice da scrivere** una volta che si ha già a disposizione un problema indecidibile noto: non serve reinventare un argomento di auto-riferimento ogni volta, basta “collegare” il nuovo problema a uno già dimostrato. Per questo, dopo il primo problema indecidibile (dimostrato per auto-riferimento), tutti i successivi si dimostrano tipicamente per riduzione.

Esercizio 4

Classificare i seguenti linguaggi per quanto riguarda la loro decidibilità o indecidibilità:

$$L_1 = \{\alpha \mid M_\alpha(001) \text{ termina con output } 110\}$$

$$L_2 = \{\alpha \mid M_\alpha \text{ è una macchina di Turing con al più 5 stati}\}$$

$$L_3 = \{(\alpha, x) \mid M_\alpha(x) \text{ non termina prima del terzo passo di computazione}\}$$

Soluzione

Per ciascun linguaggio ci chiediamo: esiste un algoritmo che, per ogni input, decide sempre correttamente se l’input appartiene al linguaggio? Il punto chiave è distinguere tra proprietà che si possono verificare **leggendo semplicemente la descrizione** della macchina (decidibili), proprietà di comportamento verificabili con un numero **limitato** di passi di simulazione (decidibili), e proprietà di comportamento **non limitate** nel tempo, come “termina” senza vincoli (tipicamente indecidibili).

$L_1 = \{\alpha \mid M_\alpha(001) \text{ termina con output } 110\}$: **indecidibile**

Passo 1 — che tipo di proprietà stiamo controllando?

Dobbiamo verificare due cose insieme: che M_α termini sull'input fisso 001, e che l'output sia esattamente 110. Il problema è che “termina” non ha un limite di tempo prefissato — potrebbe volerci 3 passi o 3 miliardi, non lo sappiamo mai in anticipo se non aspettando.

Passo 2 — la strategia: riduzione da L_{halt} .

Non dimostriamo da zero: ci appoggiamo al fatto che L_{halt} è già indecidibile.

Passo 3 — costruiamo la trasformazione.

Data un'istanza (α, x) di L_{halt} , costruiamo una nuova macchina α' che, qualunque input riceva (lo ignora completamente):

1. simula M_α sull'input x (incorporato nella descrizione di α' , non letto dall'input reale);
2. se la simulazione termina, scrive 110 e si ferma;
3. se la simulazione non termina, α' resta bloccata a simulare per sempre (non termina mai).

Passo 4 — perché questo dimostra l'equivalenza.

$M_{\alpha'}(001)$ termina con output 110 **esattamente quando** $M_\alpha(x)$ termina (indipendentemente da cosa fosse x , visto che α' lo ignora e usa sempre lo stesso x “cucito dentro”). Quindi:

$$(\alpha, x) \in L_{halt} \iff \varphi(\alpha, x) = \alpha' \in L_1$$

Passo 5 — conclusione.

Costruire α' a partire da (α, x) è pura manipolazione sintattica (calcolabile). Quindi $L_{halt} \leq_m L_1$: se L_1 fosse decidibile, lo sarebbe anche L_{halt} — contraddizione. L_1 è **indecidibile**.

$L_2 = \{\alpha \mid M_\alpha \text{ ha al più 5 stati}\}$: **decidibile**

Passo 1 — che tipo di proprietà stiamo controllando?

Qui la domanda è: “ α , come testo scritto, elenca al più 5 stati?” — non chiediamo nulla sul comportamento della macchina quando viene eseguita.

Passo 2 — l'algoritmo.

Basta leggere la stringa α (la descrizione formale della macchina, che include esplicitamente l'insieme degli stati Q) e contare quanti elementi ci sono in Q .

Passo 3 — perché questo termina sempre.

α è una stringa finita: leggerla e contare un insieme scritto al suo interno è un'operazione che finisce sempre, in un tempo proporzionale alla lunghezza di α . Nessuna simulazione, nessun rischio di loop.

Passo 4 — conclusione.

Esiste un algoritmo che decide sempre correttamente: L_2 è **decidibile**.

$L_3 = \{(\alpha, x) \mid M_\alpha(x) \text{ non termina prima del terzo passo}\}$: **decidibile**

Passo 1 — che tipo di proprietà stiamo controllando?

Sembra parlare di “terminazione” (che ricorda subito L_{halt}), ma con un vincolo speciale: il limite è fissato a soli 3 passi.

Passo 2 — l'algoritmo.

Simuliamo M_α su x per esattamente 3 passi (non di più).

Passo 3 — come rispondiamo dopo la simulazione.

- se dopo 3 passi la macchina non si è ancora fermata $\Rightarrow (\alpha, x) \in L_3$;
- se si è già fermata entro i primi 3 passi $\Rightarrow (\alpha, x) \notin L_3$.

Passo 4 — perché questo termina sempre.

Simulare un numero fisso di passi (qui: 3) richiede sempre un tempo finito e limitato — non c'è nessun “aspetta all'infinito” come nell'halting problem generale, perché ci si ferma comunque dopo il terzo passo indipendentemente da cosa succede.

Passo 5 — conclusione.

Esiste un algoritmo che decide sempre correttamente: L_3 è decidibile.

Il punto pedagogico

I tre casi mostrano la differenza tra:

- proprietà **sintattiche** della descrizione (decidibili, come L_2);
- proprietà di comportamento **limitate** nel tempo (decidibili, come L_3);
- proprietà di comportamento **non limitate** (“termina”, “produce un certo output” senza limite di passi), tipicamente indecidibili (come L_1).

09/03/2026

Esercizio 1

Consideriamo il problema definito da:

- **Input:** una stringa $x_1 \dots x_n \in \{0, 1\}^*$
- **Output:** la stringa invertita $x_n \dots x_1$

1. Descrivere (informalmente) una macchina di Turing che risolve il problema. Qual è la sua complessità in tempo?
2. Rifare lo stesso, ma con una macchina che usa **un solo nastro**.

Soluzione

Parte 1: macchina a due nastri

L'idea è leggere l'input **a ritroso** (dall'ultimo carattere al primo) e scriverlo in quest'ordine sul nastro di output: il risultato è automaticamente la stringa invertita.

Descrizione informale.

La macchina ha due nastri (uno per l'input, uno per l'output) e procede così:

- sul nastro di **input**, muovi la testina a destra finché non leggi $\square$ (il simbolo di casella vuota), senza toccare il nastro di output;
- poi muovi a sinistra sul nastro di input (per rileggere x_n , l'ultimo carattere) e a destra sul nastro di output (perché quel nastro inizia col simbolo $\triangleright$, quindi va scavalcato);
- finché non rileggi $\triangleright$ sul nastro di input: leggi il carattere corrente sul nastro di input, scrivilo sul nastro di output, muovi a sinistra sul nastro di input e a destra su quello di output.

Poiché la lettura procede dall'ultimo carattere dell'input verso il primo, e la scrittura sul nastro di output avviene sempre muovendosi in avanti, il risultato finale sul nastro di output è esattamente la stringa invertita.

Definizione formale.

Alfabeto: $\Gamma = \{0, 1, \square, \triangleright\}$. Stati: $Q = \{q_{init}, q_1, q_{accept}\}$.

La funzione di transizione δ (le coppie (x, y) indicano simbolo letto su nastro input/output; le direzioni sono indicate come (input, output)):

$$\begin{aligned}(q_{init}, x, y) &\mapsto (q_{init}, x, y, \rightarrow, S) && \text{per } x \neq \square \quad (\text{scansione verso destra, output invariato}) \\(q_{init}, \square, y) &\mapsto (q_1, \square, y, \leftarrow, \rightarrow) && (\text{fine input raggiunta: inverti direzione, avanza output}) \\(q_1, x, y) &\mapsto (q_1, x, x, \leftarrow, \rightarrow) && \text{per } x \neq \triangleright \quad (\text{copia il carattere sull'output}) \\(q_1, \triangleright, y) &\mapsto (q_{accept}, \triangleright, y, S, S) && (\text{fine raggiunta: accetta})\end{aligned}$$

Complessità in tempo.

- scansione iniziale verso destra: $n + 1$ passi
- ritorno verso sinistra, copiando: n passi

Totale: $2n + 1 = O(n)$: **tempo lineare**.

Parte 2: macchina a un solo nastro

Senza un secondo nastro non possiamo più tenere separati “input da leggere” e “output da scrivere”. Il trucco è simulare due zone sullo stesso nastro, separate da un secondo marcatore $\triangleright$: una zona contiene ciò che resta dell'input, l'altra accumula progressivamente l'output.

Descrizione informale.

1. vai a destra finché non vedi $\square$ (fine dell'input), e sovrascrivilo con $\triangleright$ (crea un secondo marcatore, dividendo il nastro in due zone);
2. torna a sinistra finché non vedi il primo $\triangleright$;
3. (**fase ***) vai a destra finché non vedi un carattere diverso da $\triangleright$:

- se è un $x_i \in \{0, 1\}$: cancellalo (scrivi $\square$), vai a destra, oltrepassa il secondo $\triangleright$, vai a destra fino al primo $\square$ e sovrascrivilo con x_i (sposti quel carattere nella seconda zona); poi torna al primo $\triangleright$ e ripeti la fase (*);
 - se è $\triangleright$ (tutto l'input è stato spostato), passa alla fase successiva;
4. (**fase ****) vai a destra finché non vedi $\square$, poi vai a sinistra: se leggi un $x_i \in \{0, 1\}$, cancellalo, vai a sinistra, oltrepassa il secondo $\triangleright$, vai a sinistra e scrivi x_i sull'ultimo $\square$ della prima zona (lo riporti indietro, ma stavolta nell'ordine opposto); se leggi $\triangleright$, cancellalo e la macchina si ferma.

La fase (*) sposta i caratteri, uno alla volta, dalla prima zona alla seconda, mantenendo il loro ordine originale (è semplicemente una copia). La fase (**) li rilegge però **dal fondo verso l'inizio** della seconda zona, riportandoli nella prima zona: è questo secondo passaggio, letto in ordine inverso, a produrre finalmente la stringa invertita.

Complessità in tempo.

Ogni singolo carattere, per essere spostato da una zona all'altra, richiede di attraversare tutto il nastro (fino a $O(n)$ passi). Poiché questo va ripetuto per ciascuno degli n caratteri (sia nella fase * che nella fase **), il tempo totale è $O(n) \times O(n) = O(n^2)$.

Punto chiave: eliminare un nastro non rende il problema irrisolvibile, ma introduce un costo aggiuntivo **polinomiale** (da lineare a quadratico) — il prezzo da pagare per simulare più nastri con uno solo.

Esercizio 2

Progettare una macchina di Turing con un solo nastro che calcola la funzione che ribalta (flip) tutti i bit della stringa in input (ogni 0 diventa 1, ogni 1 diventa 0).

Soluzione

Basta scorrere la stringa una sola volta, da sinistra a destra, scrivendo ad ogni casella il bit opposto a quello letto: non serve tornare indietro, ogni bit si trasforma sul posto, indipendentemente dagli altri.

Ingredienti.

Alfabeto: $\Gamma = \{0, 1, \square, \triangleright\}$. Stati: $Q = \{q_{init}, q_{flipping}, q_{accept}\}$.

La logica, passo per passo.

1. **Da q_{init} :** leggi $\triangleright$ (il marcatore iniziale), non lo tocchi, ti sposti a destra, passi in $q_{flipping}$.
2. **In $q_{flipping}$:** finché leggi 0 o 1, li scambi (scrivi 1 al posto di 0, e viceversa) e ti muovi a destra, restando in questo stesso stato.
3. **Quando leggi $\square$:** ti fermi, passi in q_{accept} .

Definizione formale di δ .

$(q_{init}, \triangleright) \mapsto (q_{flipping}, \triangleright, \rightarrow)$ entra nel ciclo di flip
 $(q_{flipping}, 0) \mapsto (q_{flipping}, 1, \rightarrow)$ flip di $0 \rightarrow 1$
 $(q_{flipping}, 1) \mapsto (q_{flipping}, 0, \rightarrow)$ flip di $1 \rightarrow 0$
 $(q_{flipping}, \square) \mapsto (q_{accept}, \square, S)$ fine stringa: accetta

Esempio: simuliamo su input 1101.

Nastro iniziale: $\triangleright 1101$

Passo	Stato	Simbolo letto	Azione
1	q_{init}	$\triangleright$	scrivi $\triangleright$, vai a $q_{flipping}$, muovi $\rightarrow$
2	$q_{flipping}$	1	scrivi 0, resta in $q_{flipping}$, muovi $\rightarrow$
3	$q_{flipping}$	1	scrivi 0, resta in $q_{flipping}$, muovi $\rightarrow$
4	$q_{flipping}$	0	scrivi 1, resta in $q_{flipping}$, muovi $\rightarrow$
5	$q_{flipping}$	1	scrivi 0, resta in $q_{flipping}$, muovi $\rightarrow$
6	$q_{flipping}$	$\square$	vai a q_{accept} , fermati

Nastro finale: $\triangleright 0010$ — esattamente 1101 con ogni bit ribaltato. ✓

Complessità in tempo.

La macchina scorre tutta la stringa **una sola volta**: $n + 1$ passi (le n cifre più il passo iniziale sul marcatore). Tempo $O(n)$: **lineare**.

Confronto con l'Esercizio 1: qui basta una singola passata perché ogni bit si trasforma indipendentemente dagli altri, senza bisogno di ricordare o riordinare nulla — a differenza di “invertire la stringa”, dove serve invece leggere tutto e poi tornare indietro (o, con un solo nastro, spostare i caratteri da una zona all'altra).

Esercizio 3

Definire una macchina di Turing a 2 nastri che accetta il linguaggio:

$$\{w \in \{0, 1\}^* \mid w \text{ ha lo stesso numero di 0's e 1's}\}$$

Soluzione

Invece di contare separatamente quanti 0 e quanti 1 ci sono e poi confrontare i due numeri, la macchina “consuma” una coppia 0, 1 alla volta: accumula prima tutti gli 0 su un secondo nastro, poi consuma in parallelo i 1 dell'input e i caratteri del secondo nastro. Se finiscono esattamente insieme, sono in numero uguale.

Perché basta un solo nastro di supporto (non due).

Un'idea naive userebbe 3 nastri: copiare gli 0 sul nastro 2 e i 1 sul nastro 3, poi confrontare le due copie. In realtà basta un secondo nastro: si copiano solo gli 0, e poi si confrontano direttamente con i 1 letti sull'input — non serve “raddoppiare” l'informazione su due nastri extra.

Fase 1 (stato q_{init}): accumula gli 0.

Si scorre l'input da sinistra a destra:

- se si legge 0: lo si scrive anche sul nastro di supporto, ci si muove a destra su **entrambi** i nastri, si resta in q_{init} ;
- se si legge 1: non si tocca il supporto, ci si muove a destra solo sull'input, si resta in q_{init} .

Quando l'input arriva a $\square$: si torna all'inizio di entrambi i nastri (rileggendo $\triangleright$), e si passa in q_1 .

Fase 2 (stato q_1): consuma in parallelo.

- se si legge 1 sull'input e un carattere sul supporto: si consumano entrambi, muovendosi a destra su entrambi i nastri, restando in q_1 ;
- se si legge 0 sull'input (si ignora): ci si muove a destra solo sull'input, restando in q_1 ;
- se l'input arriva a $\square$ e **anche** il supporto è a $\square$ nello stesso momento: si va in q_{accept} ;
- se uno dei due arriva a $\square$ mentre l'altro no: si va in q_{reject} .

I tre esiti possibili della Fase 2:

- input finisce con un 1 ancora da leggere, ma supporto già a $\square \Rightarrow$ **rifiuta** (troppi 1);
- input a $\square$, ma supporto non ancora a $\square \Rightarrow$ **rifiuta** (troppi 0);
- entrambi a $\square$ nello stesso passo $\Rightarrow$ **accetta**.

Definizione formale.

Alfabeto: $\Gamma = \{0, 1, \square, \triangleright\}$. Stati: $Q = \{q_{init}, q_1, q_{accept}, q_{reject}\}$. Le coppie (x, y) indicano simbolo letto su nastro input/supporto; le direzioni sono indicate come (input, supporto):

$(q_{init}, 0, y) \mapsto (q_{init}, 0, 0, \rightarrow, \rightarrow)$	copia 0 sul supporto, avanza entrambi
$(q_{init}, 1, y) \mapsto (q_{init}, 1, y, \rightarrow, S)$	ignora, avanza solo input
$(q_{init}, \square, y) \mapsto (q_1, \square, y, \text{riavvolgi}, \text{riavvolgi})$	fine fase 1: torna a $\triangleright$ su entrambi i nastri
$(q_1, 1, x) \mapsto (q_1, 1, x, \rightarrow, \rightarrow)$	per $x \neq \square$: consuma una coppia
$(q_1, 0, x) \mapsto (q_1, 0, x, \rightarrow, S)$	ignora 0, avanza solo input
$(q_1, \square, \square) \mapsto (q_{accept}, \square, \square, S, S)$	entrambi finiti insieme: accetta
$(q_1, 1, \square) \mapsto (q_{reject}, 1, \square, S, S)$	troppi 1: rifiuta
$(q_1, \square, x) \mapsto (q_{reject}, \square, x, S, S)$	per $x \neq \square$: troppi 0, rifiuta

Complessità in tempo.

La Fase 1 scorre l'input una volta: $O(n)$ passi. La Fase 2 lo scorre di nuovo: altri $O(n)$ passi. Totale: $O(n)$, **tempo lineare**.

Esercizio 4

Dimostrare l'indecidibilità dell'Halting Problem:

- **Input:** una stringa $s \in \{0,1\}^*$
- **Output:** True se $s = \langle \langle M \rangle, x \rangle$ tale che M termina sull'input x ; False altrimenti

Soluzione: riduzione dal problema UC

Supponiamo per assurdo che esista una macchina M_H che risolve l'Halting Problem. La usiamo per costruire una macchina M_{uc} che risolverebbe il problema UC. Ma sappiamo già che UC è indecidibile: quindi M_{uc} non può esistere, e di conseguenza nemmeno M_H .

Cos'è il problema UC.

- **Input:** stringa $s \in \{0,1\}^*$
- **Output:** False se $s = \langle M \rangle$ e $M(\langle M \rangle) = \text{True}$; True altrimenti

UC prende la descrizione di una macchina M , la esegue **su se stessa** (cioè le dà in input la sua stessa descrizione), e restituisce il **contrario** di quello che M risponde.

Passo 1: cosa fa M_{uc} , per prima cosa.

M_{uc} prende una stringa s in input e fa:

```
input s
run M_H on <s,s>
```

Cioè: la prima cosa che fa M_{uc} è chiedere a M_H se la macchina descritta da s , eseguita su se stessa, termina.

Perché $\langle s, s \rangle$, con s ripetuto due volte. L'Halting Problem si aspetta in input una coppia $\langle \langle M \rangle, x \rangle$: la descrizione di una macchina, più un input x su cui eseguirla. Qui vogliamo chiedere specificamente “ M termina se eseguita **su se stessa**?”, cioè vogliamo che l'input x sia proprio $\langle M \rangle$, la descrizione di M stessa. Poiché s è già (per ipotesi) la descrizione di una macchina, cioè $s = \langle M \rangle$, scrivere $\langle s, s \rangle$ significa esattamente $\langle \langle M \rangle, \langle M \rangle \rangle$: la macchina M come primo elemento, e la sua stessa descrizione come input x . Non sono due cose diverse per caso uguali: è una scelta deliberata, per costruire l'auto-applicazione $M(\langle M \rangle)$ richiesta dalla definizione di UC.

Passo 2: i due casi possibili.

- if M_H rejects then accept
 - if M_H accepts then
 - it means that $s = \langle M \rangle$ such that M halts on $\langle M \rangle$
 - run M on $\langle M \rangle$,
 - if it accepts then reject
 - if it rejects then accept
- **Caso 1** — M_H risponde “**rifiuta**” (cioè: “no, la macchina descritta da s non termina su se stessa”): allora M_{uc} **accetta**.
 - **Caso 2** — M_H risponde “**accetta**” (cioè: “sì, la macchina descritta da s termina su se stessa”): allora M_{uc} esegue davvero quella macchina su se stessa ($M(\langle M \rangle)$), guarda il risultato, e fa l’**opposto**: se avrebbe accettato, M_{uc} rifiuta; se avrebbe rifiutato, M_{uc} accetta.

Passo 3: perché questo dimostra che M_H non può esistere.

Ricordiamo la definizione di UC: **False** se $s = \langle M \rangle$ e $M(\langle M \rangle) = \text{True}$; **True** altrimenti. Guardando i due casi sopra, M_{uc} fa esattamente questo:

- se $M(\langle M \rangle)$ termina e accetta (**True**) $\Rightarrow M_{uc}$ rifiuta (**False**)
- in ogni altro caso (non termina, oppure termina e rifiuta) $\Rightarrow M_{uc}$ accetta (**True**)

Cioè: M_{uc} **calcola proprio UC**.

Conclusione.

Sappiamo già che UC è indecidibile: nessuna macchina può calcolarlo. Ma abbiamo appena costruito M_{uc} usando **solo** M_H come ingrediente. Se M_H esistesse, M_{uc} esisterebbe — ma M_{uc} (cioè UC) non può esistere. Contraddizione: quindi M_H non esiste, e l’Halting Problem non è calcolabile.

1 **12/03/2026**

Esercizio 1 (Es. 2.5 dal libro di esercizi)

Definire una macchina di Turing che decide il linguaggio:

$$L_1 := \{w \in \{0,1\}^* \mid w \text{ contiene un numero } \mathbf{dispari} \text{ di } 0\text{'s e un numero } \mathbf{pari} \text{ di } 1\text{'s}\}$$

Soluzione

L’idea è tenere traccia, mentre si scorre la stringa, di **quattro possibili combinazioni di parità** (parità degli 0 incrociata con la parità degli 1), usando quattro stati distinti. Solo una di queste quattro combinazioni corrisponde a ciò che chiede L_1 .

I quattro stati.

Il nome di ogni stato indica la parità vista finora (non il valore assoluto, solo pari/dispari):

- q_{EE} = **E**ven 0's, **E**ven 1's — stato iniziale (“zero letti” è pari per entrambi)
- q_{OE} = **O**dd 0's, **E**ven 1's — **l'unico stato “buono”**, quello richiesto da L_1
- q_{EO} = Even 0's, **O**dd 1's
- q_{OO} = **O**dd 0's, **O**dd 1's

Regola generale delle transizioni. Ogni volta che si legge un carattere, si inverte **solo** la parità corrispondente a quel carattere, lasciando l'altra invariata:

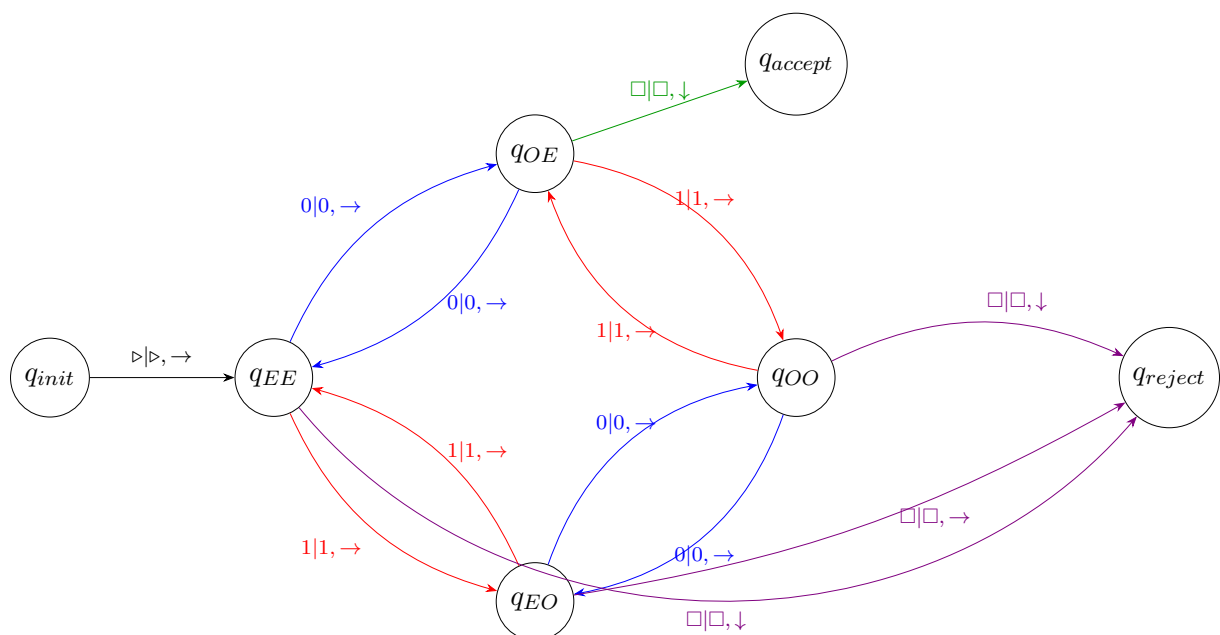
- se si legge 0: la parità dei 0 si inverte (pari→dispari o viceversa); quella degli 1 resta invariata;
- se si legge 1: viceversa.

Esempio: le transizioni da q_{EE} e da q_{OE} .

- $q_{EE} \xrightarrow{0} q_{OE}$: si legge 0, la parità dei 0 passa da pari a dispari ($E \rightarrow O$), gli 1 restano pari (E resta E).
- $q_{EE} \xrightarrow{1} q_{EO}$: si legge 1, la parità degli 1 passa da pari a dispari ($E \rightarrow O$), gli 0 restano pari (E resta E).
- $q_{OE} \xrightarrow{0} q_{EE}$: si legge 0 di nuovo, la parità dei 0 torna pari ($O \rightarrow E$).
- $q_{OE} \xrightarrow{1} q_{OO}$: si legge 1, la parità degli 1 diventa dispari ($E \rightarrow O$), gli 0 restano dispari (O resta O).

Lo stesso schema si ripete simmetricamente per q_{EO} e q_{OO} .

L'automa completo.



■ transizione iniziale ■ leggo 0 ■ leggo 1 ■ leggo □, verso accept ■ leggo □, verso reject

Perché solo q_{OE} accetta. L_1 chiede un numero **dispari** di 0's (**O**) e un numero **pari** di 1's (**E**): esattamente la combinazione codificata dal nome q_{OE} . Da qualunque altro stato, se la stringa finisce ($\square$), si va in q_{reject} .

Definizione formale (parziale).

$$\Gamma = \{0, 1, \triangleright, \square\} \quad Q = \{q_{init}, q_{EE}, q_{EO}, q_{OE}, q_{OO}, q_{accept}, q_{reject}\}$$

$$\begin{aligned} (q_{init}, \triangleright) &\mapsto (q_{EE}, \triangleright, \rightarrow) && \text{tutte le (possibili) transizioni da } q_{init} \\ (q_{EE}, 0) &\mapsto (q_{OE}, 0, \rightarrow) \\ (q_{EE}, 1) &\mapsto (q_{EO}, 1, \rightarrow) && \text{tutte le transizioni da } q_{EE} \\ (q_{EE}, \square) &\mapsto (q_{reject}, \square, \downarrow) \\ &&& \text{e così via } \dots \end{aligned}$$

Nota. Invece di avere stati q_{accept} e q_{reject} separati per accettare o rifiutare un input, si potrebbe usare un nastro di output su cui scrivere 1 (accetta) oppure 0 (rifiuta). Questo corrisponde all'equivalenza:

$$\text{linguaggio } L \subseteq \{0, 1\}^* \quad \longleftrightarrow \quad \text{funzione } \mathbb{K}_L : \{0, 1\}^* \rightarrow \{0, 1\}, \quad w \mapsto \begin{cases} 1 & \text{se } w \in L \\ 0 & \text{se } w \notin L \end{cases}$$

A margine: la funzione di transizione con più nastri. Con k nastri, la funzione di transizione ha la forma

$$\begin{aligned} \delta : Q \times \Gamma^k &\rightarrow Q \times \Gamma^k \times \{\leftarrow, \rightarrow\}^k \\ (q, a_1, \dots, a_k) &\mapsto (q', b_1, \dots, b_k, m_1, \dots, m_k) \end{aligned}$$

cioè: legge un simbolo per ciascun nastro, e per ciascun nastro decide separatamente cosa scrivere e in che direzione muoversi. Nella rappresentazione ad automa, un arco tra due stati porta l'etichetta $(a_1, \dots, a_k) \mid (b_1, \dots, b_k), (m_1, \dots, m_k)$.

Esercizio 2

Definire una macchina di Turing che decide il linguaggio:

$$L_2 := \{w \in \{0, 1\}^* \mid \text{ogni occorrenza di } 01 \text{ in } w \text{ è seguita da } 10\}$$

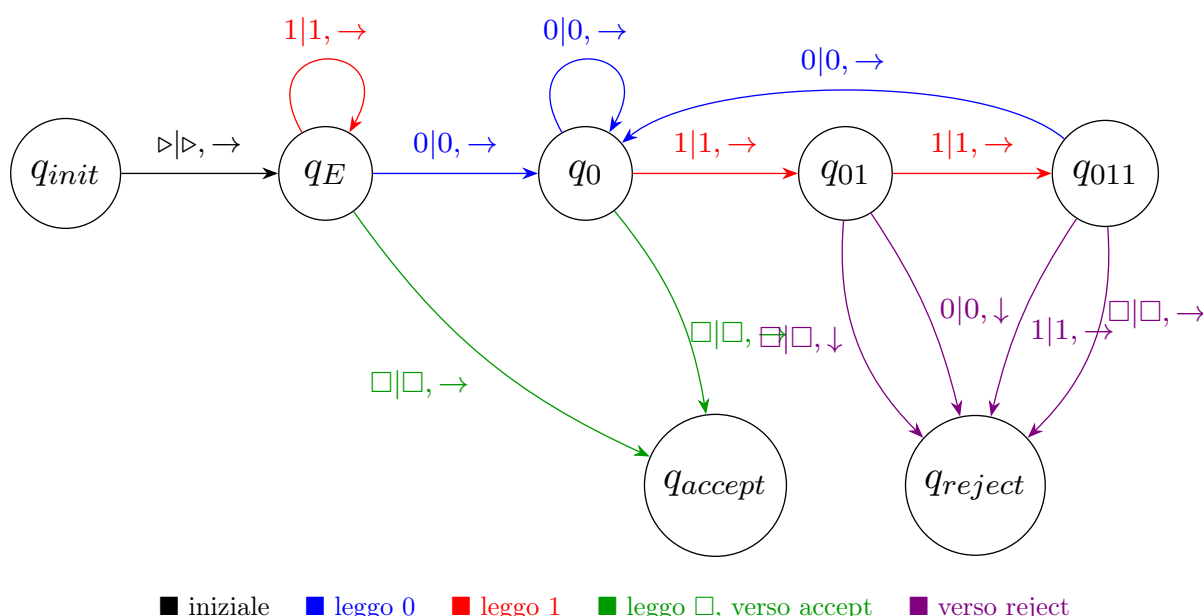
In pratica: ogni volta che compare il blocco “01”, subito dopo deve venire “10” — cioè il blocco completo dev'essere sempre “0110”. Se un “01” non è seguito immediatamente da “10”, la stringa va rifiutata.

Soluzione

Gli stati e il loro significato.

- q_E — stato “sicuro”: nessun 01 in sospeso che aspetta un completamento (stato di partenza)
- q_0 — ho appena letto un 0 isolato (potenziale inizio di “01”)
- q_{01} — ho letto “01”: ora **serve** che arrivi 1 (per continuare verso “10”)
- q_{011} — ho letto “011”: ora **serve** che arrivi 0 per completare “0110”

L'automa.



Perché il lungo arco torna a q_0 (non resta in q_{011}). Dopo aver letto “0110” per intero, l’ultimo carattere letto è di nuovo uno 0 isolato — esattamente la stessa situazione di q_0 (potenziale inizio di un *nuovo* “01”). Per questo la macchina torna in q_0 , non in uno stato “fresco” separato: la logica si ripete identica per un’eventuale prossima occorrenza di “01”.

Perché q_E e q_0 sono gli unici stati che accettano. Sono gli stati in cui **non c’è nessun “01” in sospeso** che aspetta di essere completato da “10”. Se la stringa finisce mentre siamo in q_{01} o q_{011} , vuol dire che c’è un “01” iniziato ma non completato correttamente: si rifiuta.

Perché q_{01} rifiuta se legge 0 (invece di 1).

Dopo “01”, il linguaggio richiede che il carattere successivo sia 1 (per proseguire verso “10”). Se invece arriva 0, quell’occorrenza di “01” **non** è seguita da “10”: la stringa va rifiutata immediatamente, indipendentemente da cosa segue dopo.

Perché q_{011} rifiuta se legge 1 (invece di 0).

Simmetricamente: dopo “011”, serve 0 per completare “0110”. Se arriva 1 invece, il pattern non si completa correttamente: rifiuto.

Complessità in tempo.

La macchina scorre l'input una sola volta, senza mai tornare indietro: $O(n)$, **tempo lineare**.

Esercizio 3

Consideriamo:

$$L_h^\varepsilon := \{ \lfloor M \rfloor \mid M \text{ termina sull'input } \varepsilon \} \qquad L_h^{00} := \{ \lfloor M \rfloor \mid M \text{ termina sull'input } 00 \}$$

Scrivere una riduzione $L_h^\varepsilon \leq L_h^{00}$. (E poi: una riduzione $L_h^{00} \leq L_h^\varepsilon$?)

Promemoria: cos'è una riduzione. Una riduzione è una funzione calcolabile $R : \{0,1\}^* \rightarrow \{0,1\}^*$ tale che $w \in L_h^\varepsilon \iff R(w) \in L_h^{00}$.

- Per controllare se $w \in L_h^\varepsilon$ basta trasformarlo in $R(w)$ e controllare se $R(w) \in L_h^{00}$.
- Una MT che decide L_h^{00} può quindi essere usata per costruire una MT che decide L_h^ε .
- In altre parole: L_h^ε è “più facile da decidere” di L_h^{00} .

Direzione 1: $L_h^\varepsilon \leq L_h^{00}$

Costruiamo M' che, qualunque input riceva, **cancella prima i primi due caratteri**, poi simula M su ciò che resta. Se l'input di M' è proprio 00 (due caratteri), dopo la cancellazione resta ε : esattamente la condizione che vogliamo tradurre.

Definizione della funzione R .

$$R : \{0,1\}^* \rightarrow \{0,1\}^*, \quad R(w) = \begin{cases} w & \text{se } w \text{ non è la codifica } \lfloor M \rfloor \text{ di una MT } M \\ \lfloor M' \rfloor & \text{se } w = \lfloor M \rfloor \end{cases}$$

dove M' è definita come: *cancella i primi due caratteri dell'input, poi esegui M .*

R è calcolabile perché si limita ad aggiungere una porzione finita di codice all'inizio del codice di M (le istruzioni per cancellare due caratteri) — non serve nessun calcolo complesso, solo manipolazione sintattica del codice.

Perché vale l'equivalenza.

$$\begin{aligned} w \in L_h^\varepsilon &\iff w = \lfloor M \rfloor \text{ e } M \text{ termina sull'input } \varepsilon \\ &\iff R(w) = \lfloor M' \rfloor \text{ e } M' \text{ termina sull'input } 00 \\ &\iff R(w) \in L_h^{00} \end{aligned}$$

Direzione 2: $L_h^{00} \leq L_h^\varepsilon$

Idea speculare: costruiamo M' che **scrive prima** 00 nelle prime due celle del nastro, poi passa il controllo a M . Se l'input di M' è ε (vuoto), dopo la scrittura il nastro contiene 00: di nuovo, esattamente la condizione che vogliamo tradurre.

Definizione della funzione R .

$$R(w) = \begin{cases} w & \text{se } w \text{ non è la codifica } \lfloor M \rfloor \text{ di una MT } M \\ \lfloor M' \rfloor & \text{se } w = \lfloor M \rfloor \end{cases}$$

dove M' è definita come: *scrivi 00 sulle prime due celle del nastro, poi esegui M .*

Anche qui R è calcolabile per lo stesso motivo: aggiunge solo una porzione finita di codice (le istruzioni per scrivere due 0) all'inizio del codice di M .

Perché vale l'equivalenza.

$$\begin{aligned} w \in L_h^{00} &\iff w = \lfloor M \rfloor \text{ e } M \text{ termina sull'input } 00 \\ &\iff R(w) = \lfloor M' \rfloor \text{ e } M' \text{ termina sull'input } \varepsilon \\ &\iff R(w) \in L_h^\varepsilon \end{aligned}$$

Costruzione formale di M' (con un bug scoperto durante la lezione!).

Il tutor definisce $M' = (\Gamma, Q', \delta')$ dove:

$$\begin{aligned} Q' &= Q \cup \{q'_{init}, q_0, q_{00}\} \\ \delta' &= \delta \cup \left\{ \begin{array}{l} (q'_{init}, \triangleright) \mapsto (q_0, \triangleright, \rightarrow), \\ (q_0, x) \mapsto (q_{00}, 0, \rightarrow), \\ (q_{00}, x) \mapsto (q_{init}, 0, \rightarrow) \mid x \in \Gamma \end{array} \right\} \end{aligned}$$

Il bug (notato dal tutor stesso a margine). Dopo aver scritto i due 0 (passando da q'_{init} a q_0 a q_{00}), la testina si è spostata **due caselle a destra** del marcatore $\triangleright$. Ma l'ultima transizione passa il controllo direttamente a q_{init} (lo stato iniziale della M originale) **senza prima tornare all'inizio del nastro**. Questo è un errore: M si aspetta di iniziare a leggere dalla prima cella dopo $\triangleright$, ma qui la testina si trova già spostata in avanti di due posizioni.

Come si sistemerebbe: basterebbe aggiungere, dopo aver scritto i due 0, una fase che riporta la testina fino al marcatore $\triangleright$ (esattamente come abbiamo già fatto in altri esercizi, es. “torna indietro fino al marcatore” in **inc** o nell'inversione di stringa), **prima** di passare il controllo a q_{init} .

Esercizio 4

Classificare gli 8 linguaggi seguenti rispetto a tre proprietà: **Triviale?** (vera per tutte le macchine, o per nessuna?), **Semantico?** (dipende solo dalla funzione calcolata, non da come è scritto il programma?), **Decidibile?**

- $L_1 := \{ \lfloor M \rfloor \mid M \text{ rifiuta tutti gli input} \}$
- $L_2 := \{ \lfloor M \rfloor \mid M \text{ accetta ogni input in 42 passi} \}$
- $L_3 := \{ \lfloor M \rfloor \mid M \text{ accetta più di 42 input} \}$
- $L_4 := \{ \lfloor M \rfloor \mid M \text{ rifiuta } 0^{42} \}$
- $L_5 := \{ \lfloor M, x \rfloor \mid \text{su input } x, M \text{ scrive sulla 42-esima cella del nastro} \}$
- $L_6 := \{ \lfloor M \rfloor \mid M \text{ ha più di 42 stati} \}$
- $L_7 := \{ \lfloor M \rfloor \mid L(M) \text{ è finito} \}$
- $L_8 := \{ w \in \{0,1\}^* \mid \text{se } w = \lfloor M \rfloor \text{ allora } M \text{ accetta } \varepsilon \text{ o } M \text{ rifiuta } \varepsilon \}$

Il concetto chiave: il Teorema di Rice

Teorema di Rice. Ogni proprietà **non-triviale** e **semantica** di un linguaggio riconosciuto da una macchina di Turing è **indecidibile**.

Cosa vuol dire “semantica”. Una proprietà è semantica se dipende **solo** dalla funzione calcolata dalla macchina (cioè dal comportamento input→output), e **non** da come il programma è scritto internamente. Due macchine diverse che calcolano esattamente la stessa funzione devono quindi condividere lo stesso valore di verità per una proprietà semantica.

Cosa vuol dire “triviale”. Una proprietà è triviale se è vera per **tutte** le macchine, oppure falsa per **tutte** le macchine (cioè non distingue nulla).

Perché il teorema è così utile. Basta controllare due caselle (triviale? semantico?) per dedurre **automaticamente** la terza (decidibile?), senza dover costruire una riduzione esplicita ogni volta — a patto che la proprietà sia semantica. Se non lo è, il teorema non si applica, e bisogna ragionare caso per caso.

Esempio: perché L_2 non è semantico.

Prendiamo $M_1 = \text{“accetta ogni input dopo 0 passi di computazione”}$, e $M_2 = \text{“fa 42 passi di calcolo a vuoto, poi accetta”}$. Entrambe calcolano **la stessa funzione** (la funzione costante 1: accettano sempre tutto) — quindi una proprietà semantica dovrebbe trattarle allo stesso modo.

Ma: $\lfloor M_1 \rfloor \notin L_2$ (perché M_1 accetta in 0 passi, non in 42), mentre $\lfloor M_2 \rfloor \in L_2$ (accetta esattamente in 42 passi). Due macchine equivalenti nel comportamento, ma diverse rispetto a L_2 : questo è esattamente cosa significa “non-semantico” — L_2 dipende da *come* è scritta la macchina (quanti passi impiega), non solo da *cosa* calcola.

La tabella completa

	Triviale?	Semantico?	Decidibile?
L_1	No	Sì	No (per Rice)
L_2	No	No	(*)
L_3	No	Sì	No (per Rice)
L_4	No	Sì	No (per Rice)
L_5	No	No	(**)
L_6	No	No	Sì
L_7	No	Sì	No (per Rice)
L_8	Sì	Sì	Sì

L_6 (più di 42 stati): **decidibile nonostante non sia semantico**. Questa è una proprietà puramente **sintattica** (basta contare gli stati elencati nella descrizione di M , senza eseguire nulla) — non essendo semantica, il Teorema di Rice non si applica affatto, e infatti qui il linguaggio risulta comunque decidibile (con un algoritmo banale di conteggio, come già visto in un esercizio precedente).

L_8 : **triviale** $\Rightarrow$ **decidibile a prescindere**. Ogni macchina M o accetta ε o lo rifiuta (nessuna terza opzione, assumendo che M termini sempre su ε — ma se non termina, resta comunque vero che $w \notin L_8$ solo se w non è una codifica valida): quindi L_8 è vero per *ogni* $w = \lfloor M \rfloor$ valida, rendendolo triviale, e triviale $\Rightarrow$ decidibile immediatamente (non serve nemmeno Rice).

(*) **Nota del tutor su L_2** .

“Non decidibile, credo... ma trovare una riduzione esplicita (ad esempio dall’Halting Problem) sarebbe un po’ doloroso.” Il tutor non fornisce una dimostrazione formale: è un’ipotesi plausibile (dato che L_2 non è semantico, il Teorema di Rice non può confermarlo automaticamente), lasciata aperta.

(**) **Nota del tutor su L_5** .

Anche qui, L_5 non è semantico, quindi il Teorema di Rice non si applica: la decidibilità (se confermata) andrebbe dimostrata con un argomento ad hoc, che il tutor stesso lascia come domanda aperta.

2 **19/03/2026**

Esercizio 1

Dimostrare che la funzione *minmax*, che data (la codifica di) una lista di numeri naturali $(a_1, \dots, a_m)$, restituisce sia il minimo che il massimo tra $a_1, \dots, a_m$, è in **FP**.

Esempio:

$$\text{minmax}(1, 7, 0, 2, 11, 3) = (0, 11)$$

Soluzione

Cos'è “tempo polinomiale”. Un algoritmo lavora in tempo polinomiale se il numero di passi è $O(n^k)$ per una qualche costante k (dove n è la dimensione dell'input). Include il tempo lineare ($k = 1$), ma anche $O(n^2)$, $O(n^3)$, ecc., tutti con k **fisso**, indipendente da n .

Perché è la soglia importante. Il vero confronto è con la crescita **esponenziale** ($O(2^n)$): con $n = 50$, $2^{50} \approx 10^{15}$ passi, impraticabile anche per un computer velocissimo. Con il polinomiale, per quanto grande sia k , aggiungere un elemento in più all'input cambia i passi solo di un fattore prevedibile e limitato, non “esplosivo” come l'esponenziale.

FP è la classe di tutte le funzioni calcolabili in tempo polinomiale (l'equivalente di **P**, ma per funzioni con output, non solo problemi sì/no). È lo standard con cui si definisce “efficiente” o “trattabile” in informatica teorica.

L'algoritmo.

Input: $(a_1, \dots, a_m)$, $m \in \mathbb{N}$, $a_i \in \mathbb{N}$, opportunamente codificati.

Output: (b, c) , dove b è il minimo tra $a_1, \dots, a_m$ e c è il massimo.

```
min <- a_1
max <- a_1
for i <- 2 to m do
  if a_i < min then min <- a_i
  if a_i > max then max <- a_i
return (min, max)
```

Poiché si cercano esaustivamente il minimo e il massimo scorrendo tutti gli $a_1, \dots, a_m$, l'algoritmo è banalmente **corretto**. Resta da dimostrare che lavora in tempo polinomiale.

Parte 1: numero di istruzioni eseguite.

- `min <- a_1` e `max <- a_1`: 2 istruzioni (fuori dal ciclo)
- il ciclo `for i <- 2 to m`: si ripete $O(m)$ volte
- dentro il ciclo: al massimo 4 istruzioni per iterazione (i due `if`, le due possibili assegnazioni)
- `return`: 1 istruzione finale

Il numero totale di istruzioni eseguite è quindi al massimo:

$$(2 + O(m) \cdot 4 + 1) \in O(m)$$

Parte 2: dimensione dei risultati intermedi.

Le uniche variabili usate sono `min`, `max`, e l'indice i . Si noti però che:

$$|min| \leq |a_2, \dots, a_m| \leq n \in O(n)$$

$$|max| \leq |a_2, \dots, a_m| \leq n \in O(n)$$

$$|i| \leq \ell_2(m) \in O(n)$$

(dove n è la dimensione totale dell'input). Tutti i risultati intermedi, di cui ce ne sono finitamente tanti, hanno dimensione polinomiale.

Parte 3: costo di ogni singola istruzione.

Ogni istruzione può essere eseguita a sua volta in tempo polinomiale, perché le uniche istruzioni non elementari sono i confronti tra numeri naturali.

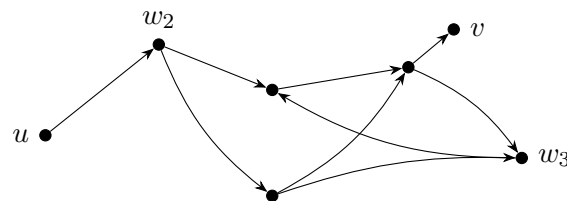
Conclusione.

Tutto questo implica che lo pseudocodice sopra lavora in tempo polinomiale effettivo, e in definitiva che $minmax \in \mathbf{FP}$.

3 Esercizio 2

Se $G = (V, E)$ è un grafo, il problema della **raggiungibilità** chiede di determinare, dati $u, v \in V$, se v è raggiungibile da u , cioè se esiste una sequenza $w_1, \dots, w_m$ di nodi con $u = w_1$, $v = w_m$, e $(w_i, w_{i+1}) \in E$ per ogni $i \in \{1, \dots, m-1\}$.

Questo problema è in **P**. Dimostrarlo!



Un esempio di grafo diretto: v è raggiungibile da u tramite il cammino $u \rightarrow w_2 \rightarrow a \rightarrow c \rightarrow v$ (dove a, c sono nodi intermedi senza nome specifico). Si noti che il cammino non è né unico né ovvio a colpo d'occhio: ci sono anche archi che portano verso w_3 , che però non fanno parte di *questo* particolare cammino da u a v — l'algoritmo di raggiungibilità deve esplorare sistematicamente tutte le diramazioni per scoprire se un cammino verso v esiste, indipendentemente da quanto sia “nascosto” tra gli altri archi.

Soluzione

Input: $(G = (V, E), u, v)$, come sopra.

Output: 0 o 1 a seconda che v sia raggiungibile da u in G .

L'idea è tenere due liste: **visited** (tutti i nodi raggiungibili da u scoperti finora) e **current** (i nodi “appena scoperti”, di cui esplorare ancora i vicini). Si espande progressivamente finché non c'è più nulla da esplorare, poi si controlla se v è finito in **visited**.

```
visited <- [u]
current <- [u]
while current != [] do:
  pick w from current
  remove w from current
  for each (w,t) in E
    if t not in visited
      add t to visited
```

```

        add t to current
if v in visited then
    return true
else
    return false

```

Il tutor inizialmente rimanda la dimostrazione di correttezza (“è hard, fuori dallo scopo del corso”), ma la fornisce comunque poco dopo nei suoi appunti — eccola qui sotto per completezza.

Perché il ciclo **while** termina, e in quanti passi.

A differenza di un ciclo **for** con estremi noti, qui non è ovvio a priori quante volte il **while** giri. Servono tre osservazioni:

- **current** è sempre un sottoinsieme di **visited**: ogni nodo che finisce in **current** ci finisce perché è stato aggiunto anche a **visited** nello stesso momento.
- **visited** è **inflazionabile**: cresce sempre, non si riduce mai.
- **ogni nodo entra in current al massimo una volta**. Quando aggiungiamo t a **current**, lo facciamo solo se $t \notin \text{visited}$ in quel momento. Se t fosse già stato in **current** in passato, sarebbe già stato aggiunto anche a **visited** allora (primo punto) — quindi ora lo troveremmo già in **visited**, e non verrebbe riaggiunto.

Come conseguenza, il ciclo **while** viene eseguito al massimo $|V|$ volte, che è un limite legato direttamente alla dimensione della rappresentazione del grafo.

Cosa significa $|V|$. La notazione $|V|$ (le due barrette verticali) si legge “*cardinalità di V*”: indica semplicemente **quanti elementi ci sono nell’insieme V** — qui, il numero di nodi del grafo. Non ha nulla a che vedere con il valore assoluto di un numero o con la norma di un vettore (concetti diversi): è solo “quanti elementi contiene l’insieme”. Ad esempio, se $V = \{u, v, w, z\}$, allora $|V| = 4$.

Dimostrazione che l’algoritmo è in P

Parte 1: numero di istruzioni eseguite.

Il ciclo **while** esterno gira al massimo $|V|$ volte (come appena dimostrato). Ad ogni iterazione, il ciclo interno **for each** (**w,t**) in **E** gira $O(n)$ volte (al massimo tanti archi quanti ce ne sono, limitati dalla dimensione dell’input n). Il numero totale di macro-istruzioni eseguite è quindi:

$$O(|V|) \times O(n) = O(n^2)$$

Parte 2: dimensione dei risultati intermedi.

Le uniche variabili usate sono **visited** e **current**, e contengono entrambe nodi di V , senza possibilità di duplicati (un nodo, una volta in **visited**, non viene mai rimosso né riaggiunto). La loro dimensione è quindi chiaramente polinomiale in n : al massimo $|V|$ nodi ciascuna, già limitato dalla dimensione dell’input stesso.

Parte 3: costo di ogni singola istruzione.

Ogni istruzione usata (aggiungere un elemento a una lista, rimuovere un elemento da una lista, controllare se un oggetto è parte di una lista) è un'operazione di manipolazione dati elementare, e tutti questi problemi sono in **FP**.

Conclusione.

Tutto questo implica che l'algoritmo lavora in tempo polinomiale ($O(n^2)$ macro-istruzioni, ciascuna eseguibile a sua volta in tempo polinomiale). Il problema della **raggiungibilità** è quindi in **P**.

Dimostrazione di correttezza

Vogliamo dimostrare che se v è raggiungibile da u , l'algoritmo lo scopre davvero (cioè v finisce in **visited**). Più precisamente: *se v è raggiungibile da u attraverso un cammino di lunghezza n , allora v finirà (prima o poi) in **visited***. Lo dimostriamo per **induzione su n** .

Caso base: $n = 0$.

Se $n = 0$, allora $u = v$ (il cammino ha lunghezza zero). E $v = u$ viene inserito in **visited** fin dall'inizio (**visited** $\leftarrow$ [**u**]). Il caso base è quindi banale.

Passo induttivo: $n > 0$.

Assumiamo che l'enunciato valga per $n - 1$ (ipotesi induttiva), e consideriamo un cammino di lunghezza n :

$$u \rightsquigarrow \dots \rightsquigarrow w \rightsquigarrow v$$

dove i primi $n - 1$ passi portano da u a un nodo w , e l'ultimo passo è l'arco (w, v) . Per ipotesi induttiva (applicata al cammino di lunghezza $n - 1$ fino a w): w **verrà inserito in visited**.

Restano da osservare due fatti per concludere che anche v finisce in **visited**:

- w **viene inserito in current nello stesso “momento” in cui viene inserito in visited**. Guardando il codice, ogni volta che un nodo viene aggiunto a **visited**, viene contemporaneamente aggiunto anche a **current** (due righe consecutive: **add t to visited** / **add t to current**).
- w **non può restare in current per sempre, perché l'algoritmo termina** (lo abbiamo già dimostrato: il ciclo **while** gira al massimo $|V|$ volte). Ogni nodo che entra in **current** deve prima o poi uscirne (essere “pescato” con **pick w from current** e rimosso).

Il momento decisivo.

Nel momento esatto in cui w viene rimosso da **current**, il ciclo interno **for each (w,t) in E** scorre tutti gli archi uscenti da w — incluso, in particolare, l'arco (w, v) (che sappiamo esistere, essendo l'ultimo passo del cammino). A quel punto, se v non è già in **visited**, viene aggiunto proprio lì.

Conclusione. w entra in **visited** (ipotesi induttiva) $\Rightarrow w$ entra anche in **current** $\Rightarrow w$ ne esce prima o poi (l'algoritmo termina) $\Rightarrow$ nel momento in cui esce, l'arco (w, v) viene considerato $\Rightarrow v$ viene inserito in **visited**. Questo completa il passo induttivo, e quindi l'intera dimostrazione.

Esercizio: Wedding Plan Problem

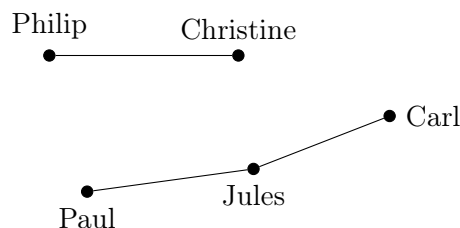
Dimostrare che il problema del “piano matrimonio”, come descritto all’inizio del corso, è in **FEXP**.

Soluzione

Il problema è definito come segue: dati una lista di invitati e una lista di vincoli di incompatibilità tra invitati, si vuole determinare un sottoinsieme di invitati che siano compatibili tra loro e che abbia lunghezza massimale.

Questo può essere visto come un problema su grafi: gli invitati sono i **nodi**, e le incompatibilità sono gli **archi**. Il problema diventa allora trovare il più grande sottoinsieme di nodi **senza nessun arco al proprio interno** — il problema del **Massimo Insieme Indipendente**.

Esempio.



$$G = (\{Philip, Christine, \dots\}, \{\{Philip, Christine\}, \{Paul, Jules\}, \{Jules, Carl\}\})$$

Definizioni necessarie.

Se $G = (V, E)$ è un grafo non orientato e $v \in V$, indichiamo con $N(v)$ il sottoinsieme dei nodi adiacenti a v :

$$N(v) = \{w \mid \{v, w\} \in E\}$$

Il problema del **Massimo Insieme Indipendente** su grafi non orientati è il problema di, dato un grafo G , determinare $W \subseteq V$ tale che:

- A. per ogni $v \in W$: $N(v) \cap W = \emptyset$ (**indipendenza**: nessun nodo scelto ha un vicino anche lui scelto)
- B. la cardinalità di W è massima tra i sottoinsiemi di V che soddisfano A (**massimalità**)

Lo pseudocodice.

Input: un grafo non orientato $G = (V, E)$.

Output: $W \subseteq V$ che soddisfa A e B.

```
Z <- {}
foreach W subsequence V do
  ind <- True
  foreach w in W do
    if N(w) intersect W != {} then ind <- false
  if ind then
    if |Z| < |W| then
      Z <- W
return Z
```

Analisi della complessità

Parte 1: numero di istruzioni eseguite.

- $Z \leftarrow \{\}$: 1 istruzione
- **foreach** W **subsequence** V : si ripete $O(2^{|V|})$ volte (ogni possibile sottoinsieme di V)
- dentro: $\text{ind} \leftarrow \text{True}$ (1), poi **foreach** w **in** W ($O(|V|)$ volte, ciascuna al più 2 istruzioni), poi al più 3 istruzioni per l'aggiornamento di Z
- **return** Z : 1 istruzione

Il totale è:

$$2 + O(2^{|V|}) \cdot (4 + O(|V|) \cdot 2) = O(2^{|V|}) \cdot O(|V|) = O(|V| \cdot 2^{|V|}) = O(2^{|V|^2})$$

Perché l'ultimo passaggio è lecito. $|V| \cdot 2^{|V|}$ non è *esattamente* $2^{|V|^2}$, ma lo possiamo limitare dall'alto: poiché $|V| \leq 2^{|V|}$ (per $|V|$ sufficientemente grande),

$$|V| \cdot 2^{|V|} \leq 2^{|V|} \cdot 2^{|V|} = 2^{2|V|} \leq 2^{|V|^2}$$

Per dimostrare che un algoritmo è in FEXP non serve il limite più stretto possibile: basta una qualsiasi funzione della forma $2^{\text{polinomio}}$ che maggiori il conteggio reale.

Parte 2: dimensione dei risultati intermedi.

- ind ha dimensione costante (un booleano)
- Z e W si possono implementare come vettori di bit di lunghezza $|V| \leq |G|$ (un bit per nodo)
- w è semplicemente un nodo

Tutte le variabili restano di dimensione **polinomiale** in $|G|$.

Parte 3: costo di ogni singola istruzione.

Tutte le istruzioni, tranne una, richiedono banalmente tempo polinomiale. L'unica da controllare con attenzione è $N(w) \cap W$:

- calcolare $N(w)$ richiede tempo polinomiale in $|G|$ (basta scorrere gli archi)
- l'intersezione $\cap$ è una semplice operazione bit a bit (AND) tra i due vettori di bit

Il punto chiave di questa dimostrazione. L'algoritmo è in **FEXP**, non perché una singola istruzione sia costosa (sono tutte polinomiali), ma perché il **numero di volte** che si ripete il ciclo esterno è esponenziale ($2^{|V|}$ sottoinsiemi da provare). È l'opposto di quanto succedeva con *minmax* e la raggiungibilità, dove sia il numero di iterazioni *che* il costo per iterazione erano polinomiali.

Conclusione.

Mettendo insieme le tre parti (numero di istruzioni $O(2^{|V|^2})$, dimensione dei risultati intermedi polinomiale, costo di ogni istruzione polinomiale): lo pseudocodice lavora in tempo $2^{\text{polinomio}}$, cioè in tempo esponenziale ma limitato da un esponente polinomiale. Quindi il problema del Wedding Plan (cioè il Massimo Insieme Indipendente) è in **FEXP**.

Teorema: $P \subsetneq EXP$

Cos'è EXP. La classe di tutti i problemi decisionali risolvibili da una macchina di Turing in **tempo esponenziale**:

$$EXP = \bigcup_k DTIME(2^{n^k})$$

cioè tempo del tipo $2^n, 2^{n^2}, 2^{n^{100}}$, ecc. (due elevato a una potenza polinomiale fissa di n). È l'analogo di P (che usa n^k) ma con 2^{n^k} al posto di n^k : contiene quindi tutti i problemi di P (avere più tempo non guasta), più tutti quelli per cui serve davvero tempo esponenziale.

Vogliamo dimostrare non solo che $P \subseteq EXP$ (facile), ma che l'inclusione è **stretta**: esiste almeno un problema che sta in EXP e non in P .

Dimostrazione (sketch)

Perché $P \subseteq EXP$ è ovviamente vero.

Ricordando le definizioni in termini di $DTIME$:

$$P = \bigcup_k DTIME(n^k) \qquad EXP = \bigcup_k DTIME(2^{n^k})$$

vale $DTIME(n^k) \subseteq DTIME(2^{n^k})$.

Se un problema si risolve in n^k passi, si risolve automaticamente anche in 2^{n^k} passi, dato che 2^{n^k} è un numero molto più grande di n^k (per n sufficientemente grande). Avere più tempo a disposizione non può mai far peggiorare le cose.

Perché l'inclusione è stretta: il Teorema della Gerarchia.

Il fatto che l'inclusione sia stretta si può dimostrare attraverso il **Teorema della Gerarchia**, una versione semplice del quale ci dice che:

$$DTIME(f(n)) \subsetneq DTIME(f(n)^2) \quad \text{per ogni "f ragionevole"}$$

In parole: se una macchina gira in tempo $f(n)$, darle il **quadrato** di quel tempo ($f(n)^2$, molto di più) permette di risolvere **strettamente** di più — non solo altrettanto, ma esistono davvero problemi in più risolvibili con quel tempo extra. Dare più tempo a una macchina non è mai sprecato: le permette sempre di fare qualcosa in più che con meno tempo non poteva fare.

Applicazione a P ed EXP .

Possiamo sfruttare il Teorema della Gerarchia come segue:

$$P \subseteq DTIME(2^n) \subsetneq DTIME((2^n)^2) = DTIME(2^{2n}) \subseteq EXP$$

La catena, spiegata pezzo per pezzo:

1. $P \subseteq DTIME(2^n)$: ogni problema polinomiale sta anche in $DTIME(2^n)$ (più tempo non guasta);
2. $DTIME(2^n) \subsetneq DTIME((2^n)^2)$: si applica il Teorema della Gerarchia con $f(n) = 2^n$ — qui l'inclusione è **stretta**;
3. $(2^n)^2 = 2^{2n}$: semplice conto algebrico;
4. $DTIME(2^{2n}) \subseteq EXP$: perché $2n$ è un caso particolare di n^k (con $k = 1$, a meno di una costante moltiplicativa), quindi rientra nella definizione di EXP .

Conclusione.

Mettendo insieme la catena: $P \subseteq DTIME(2^n) \subsetneq DTIME(2^{2n}) \subseteq EXP$. Il passaggio **stretto** ($\subsetneq$) si trova nel mezzo della catena, e questo basta per concludere che l'inclusione complessiva tra P ed EXP è anch'essa stretta:

$$P \subsetneq EXP$$

4 **16/04/2026**

Esercizio 1

Dimostrare che il prodotto tra matrici è in **FP**.

Soluzione

Quale funzione stiamo calcolando?

$$f : \left(p, q, r, (A_{ij})_{\substack{1 \leq i \leq p \\ 1 \leq j \leq q}}, (B_{jk})_{\substack{1 \leq j \leq q \\ 1 \leq k \leq r}} \right) \mapsto \left(\sum_{j=1}^q A_{ij} B_{jk} \right)_{\substack{1 \leq i \leq p \\ 1 \leq k \leq r}}$$

Se non hai mai visto il prodotto tra matrici scritto così, pensa A e B come tabelle di numeri: A ha p righe e q colonne, B ha q righe e r colonne (le colonne di A devono essere tante quante le righe di B), e il risultato $C = A \times B$ ha p righe e r colonne.

A_{ij} è semplicemente il numero nella riga i , colonna j , di A . La formula $C_{ik} = \sum_{j=1}^q A_{ij} B_{jk}$ è la regola standard: l'elemento in posizione (i, k) del risultato si ottiene moltiplicando la riga i di A per la colonna k di B , elemento per elemento, e sommando tutto.

L'algoritmo.

```
input (p,q,r,A,B)
initialise list C[]
For i from 1 to p:
    initialise list C[i][]
    For k from 1 to r:
        C[i][k] <- 0
        For j from 1 to q:
            C[i][k] <- C[i][k] + A[i][j] x B[j][k]
Return C
```

I tre cicli annidati (i , poi k , poi j) corrispondono esattamente ai tre indici della formula: il ciclo su i scorre le righe di C (e di A), il ciclo su k scorre le colonne di C (e di B), il ciclo su j calcola la sommatoria $\sum_j$ (si parte da $C[i][k] <- 0$, e ad ogni passo si aggiunge un pezzo della somma).

Esempio numerico.

Prendiamo $A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ e $B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}$. Calcoliamo C_{11} ($i = 1, k = 1$):

- $j = 1$: $C[1][1] <- 0 + A[1][1] \times B[1][1] = 0 + 1 \times 5 = 5$
- $j = 2$: $C[1][1] <- 5 + A[1][2] \times B[2][1] = 5 + 2 \times 7 = 19$

Quindi $C_{11} = 19$ — esattamente “riga 1 di A per colonna 1 di B ”: $1 \times 5 + 2 \times 7 = 19$. ✓

Dimostrazione che l'algoritmo è in FP

Parte 1: numero di istruzioni eseguite.

Contando l'annidamento dei cicli (i : $\times p$, k : $\times r$, j : $\times q$, con 3 istruzioni per iterazione più qualche istruzione costante fuori dai cicli), il numero di istruzioni cresce come $p \times q \times r$.

Il tutor scrive: l'algoritmo esegue $O(n^3)$ istruzioni, perché $p, q, r = O(n)$.

Perché $p, q, r \in O(n)$. Qui n è la dimensione della codifica dell'intero input $[p, q, r, A, B]$ — cioè quanto spazio (in caratteri/bit) serve per scrivere tutto, incluse le matrici. Ad esempio, per $p = q = r = 2$ e le matrici dell'esempio sopra, la codifica potrebbe assomigliare a:

$$\underbrace{2}_p \# \underbrace{2}_q \# \underbrace{2}_r \# \underbrace{1, 2, 3, 4}_A \# \underbrace{5, 6, 7, 8}_B$$

Il numero p (qui “2”) è solo un piccolo pezzo di questa stringa: non può occupare più spazio della stringa intera che lo contiene. Quindi $p, q, r \leq n$, cioè $p, q, r \in O(n)$. Di conseguenza $p \times q \times r \leq n^3$, da cui $O(n^3)$ istruzioni totali.

Parte 2: dimensione dei risultati intermedi.

L'unico dato interno rilevante è la matrice C che si sta costruendo. Il tutor mostra:

$$|C| \leq p \times r \times (q \times |A| \times |B|)$$

Perché $p \times r$ **davanti a tutto**. C ha $p \times r$ celle totali; per ognuna dobbiamo stimare quanti bit “pesa” il numero che ci sta dentro.

Perché $q \times |A| \times |B|$ **per singola cella**. Ogni cella è una somma di q prodotti (uno per ogni j), e ogni prodotto è “un elemento di A per un elemento di B ”: $|A| \times |B|$ stima quanto può pesare (in bit) un singolo prodotto, moltiplicato per q perché se ne sommano q .

Cosa significa $|A|$. Non è “il valore di A ”, ma *quanti bit servono per scrivere l'intera matrice A* (la sua dimensione in codifica).

La nota del tutor: $|A| \leq p \times q \times \log(\max_{i,j} A_{ij})$. La dimensione (in bit) dell'intera matrice A è al massimo $p \times q$ (il numero totale di elementi) moltiplicato per $\log(\max_{i,j} A_{ij})$ (quanti bit servono per scrivere il più grande elemento di A , dato che un numero in binario occupa $\log_2$ del suo valore).

Poiché $\max_{i,j} A_{ij}$ fa parte dell'input, la sua dimensione in bit è anch'essa limitata da n : quindi $\log(\max A_{ij}) \in O(n)$, e $|A| \leq p \times q \times \log(\max A_{ij}) \in O(n) \times O(n) \times O(n) = O(n^3)$. Lo stesso vale per $|B|$.

Mettendo insieme:

$$|C| \leq p \times r \times (q \times |A| \times |B|) \in O(n) \times O(n) \times (O(n) \times O(n^3) \times O(n^3)) = O(n^9)$$

Il punto cruciale non è il valore esatto dell'esponente (qui viene un po' alto, n^9): è che resta un **polinomio** in n , non esplode in modo esponenziale. Questo è tutto ciò che serve per la dimostrazione.

Parte 3: costo di ogni singola istruzione.

Ogni istruzione è calcolabile in tempo polinomiale, in particolare la somma e il prodotto nella riga `C[i][k] <- C[i][k] + A[i][j] * B[j][k]`: sommare e moltiplicare due numeri (anche grandi) sono operazioni standard, polinomiali nella dimensione dei numeri coinvolti.

Conclusione.

Mettendo insieme le tre parti: l'algoritmo esegue $O(n^3)$ istruzioni, i dati intermedi restano di dimensione polinomiale ($O(n^9)$ nel caso peggiore), e ogni istruzione singola è polinomiale. Quindi la moltiplicazione tra matrici è in **FP**.

Esercizio 2

Un grafo (V, E) è **bipartito** se $V = V_1 \uplus V_2$ (unione disgiunta: $V_1 \cap V_2 = \emptyset$) e per ogni $(v, w) \in E$: $v \in V_1$ e $w \in V_2$, oppure $v \in V_2$ e $w \in V_1$.

1. Il problema di determinare se un grafo è bipartito è in **P**?
2. È in **NP**?
3. È **NP-completo**?

Cosa vuol dire “bipartito”, intuitivamente. I nodi si possono dividere in due gruppi (V_1 e V_2) tali che ogni arco collega sempre un nodo di un gruppo con un nodo dell'altro — mai due nodi dello stesso gruppo. È come se si potesse colorare ogni nodo con uno di due colori in modo che nodi collegati abbiano sempre colori diversi.

Punto 1: BIPARTITE è in P?

L'idea è provare a **colorare** i nodi con 2 colori in modo che nodi collegati abbiano sempre colori diversi, usando una visita in ampiezza (BFS) su ogni componente connessa. Se si trova un conflitto (due nodi collegati con lo stesso colore), il grafo non è bipartito.

Input e rappresentazione del grafo.

Il grafo $G = (\{v_1, \dots, v_n\}, E)$ viene rappresentato con la sua matrice di adiacenza $[G]$: una matrice $n \times n$ tale che $[G]_{ij} = 1$ se $(v_i, v_j) \in E$, altrimenti 0.

L'algoritmo.

```
input: G presentato come matrice di adiacenza [G]
C <- [0,...,0]          (n volte)
Q <- []
For i from 1 to n:
  if C[i] = 0:
    C[i] <- 1
    Push i alla fine di Q
    While Q != []:
      Pop j dalla testa di Q
      For k from 1 to n:
        if [G]_jk = 1 or [G]_kj = 1:
          if C[j] = C[k] then return false
          if C[k] = 0 then
            C[k] <- 3 - C[j]
            Push k alla fine di Q
Return true
```

Il ciclo for esterno scorre *tutte le componenti connesse* del grafo (il suo corpo prova a colorare la componente connessa del nodo i , se non già fatto).

Il ciclo while interno esegue una **BFS** (Breadth-First Search, visita in ampiezza) a partire dal nodo i : esplora tutti i vicini di un nodo prima di passare ai vicini dei vicini, un “livello” alla volta (come un’onda che si propaga).

Il trucco 3 - $C[j]$ assegna sempre il colore opposto a quello di $C[j]$: se $C[j] = 1$, allora $3 - 1 = 2$; se $C[j] = 2$, allora $3 - 2 = 1$. Un modo compatto per dire “l’altro colore”.

Se $C[j] = C[k]$ per due nodi collegati, vuol dire che hanno lo **stesso** colore nonostante siano collegati: contraddizione con la bipartizione, quindi si restituisce **false** immediatamente.

Conteggio dettagliato delle istruzioni.

$C \leftarrow [0, \dots, 0]$	$(n \text{ volte})$	n
$Q \leftarrow []$		1
For i from 1 to n :		
if $C[i] = 0$:		1
$C[i] \leftarrow 1$		1
Push i alla fine di Q		$O(n)$
While $Q \neq []$:		
Pop j dalla testa di Q		1
For k from 1 to n :		$\times O(n)$
if $[G]_{jk} = 1$ or $[G]_{kj} = 1$:		1
if $C[j] = C[k]$ then return false		1
if $C[k] = 0$ then $C[k] \leftarrow 3 - C[j]$		1
push k alla fine di Q		$O(n)$
Return true		1

Il punto più sottile: perché non si moltiplica “per ogni i ”. Verrebbe naturale pensare: il ciclo **while** costa $O(n)$, e viene eseguito dentro il ciclo **for** i che gira $O(n)$ volte, quindi il totale sarebbe $O(n) \times O(n) = O(n^2)$ solo per quella parte. Ma questo **sovrastimerebbe** il lavoro reale.

Il punto chiave (annotato dal tutor in rosso) è: **ogni nodo viene inserito (“pushed”) in Q al massimo una volta in tutto l’algoritmo**, non una volta per ogni i del ciclo esterno — perché una volta colorato ($C[i] \neq 0$), un nodo non viene mai più ri-processato dal ciclo **for** i successivo (è proprio il controllo **if $C[i] = 0$** a saltarlo). Quindi il ciclo **while**, sommato su **tutte** le esecuzioni del ciclo **for** i messe insieme, esegue in totale al massimo $O(n)$ **pop** (uno per nodo), non $O(n)$ pop per ciascuno degli $O(n)$ valori di i .

Questo spiega perché nel conteggio finale il fattore $O(n)$ del ciclo **while** (per il numero totale di nodi processati) si moltiplica **una sola volta** per l’ $O(n)$ del ciclo **for** k interno — e non anche per l’ $O(n)$ del ciclo **for** i esterno (quello contribuisce solo un fattore $O(n)$ aggiuntivo per il suo overhead di controllo, non per il lavoro di BFS vero e proprio).

Mettendo insieme tutti i fattori, il tutor conclude che l’algoritmo esegue $O(n^4)$ istruzioni in totale.

- ogni istruzione è polinomiale
- tutti i dati interni sono polinomiali: C ha dimensione n , Q ha dimensione n

Attenzione alla dimensione reale dell'input! Qui la dimensione dell'input non è n ma n^2 , perché il grafo è dato come matrice di adiacenza $n \times n$ (bisogna scrivere tutti gli n^2 elementi della matrice, non solo n numeri). Quindi un algoritmo che esegue $O(n^4)$ istruzioni ha in realtà complessità **quadratica** rispetto alla dimensione reale dell'input: $O(n^4) = O((n^2)^2)$.

L'algoritmo lavora in tempo polinomiale: $\text{BIPARTITE} \in \mathbf{P}$.

Punto 2: BIPARTITE è in NP?

Sì, perché $P \subseteq NP$. Se un problema è già in P (come appena dimostrato), sta automaticamente anche in NP — dato che P è un sottoinsieme di NP : qualunque problema risolvibile velocemente in modo deterministico lo è anche in modo non deterministico.

Volerlo dimostrare esplicitamente? Si può scrivere BIPARTITE come linguaggio NP tramite un **certificato** y e un **verificatore** M :

$$\text{BIPARTITE} = \left\{ \lfloor G \rfloor \mid \exists y \in \{0, 1\}^{42 \times |G|^{42}}, M \text{ accetta } \lfloor G, y \rfloor \right\}$$

dove M è la macchina che esegue l'algoritmo sopra su $\lfloor G \rfloor$, **senza curarsi di y** (cioè y non viene nemmeno usato: dato che il problema è già risolvibile deterministicamente in tempo polinomiale, non serve davvero nessun certificato — la definizione formale di NP lo richiede comunque nella struttura, ma qui è semplicemente ignorato).

Punto 3: BIPARTITE è NP-completo?

Questa è una battuta che fa riferimento al **problema P vs NP** , uno dei sette Problemi del Millennio (con un premio reale di un milione di dollari per chi lo risolve). Se BIPARTITE fosse NP -completo, e sappiamo già che $\text{BIPARTITE} \in P$, allora si dimostrerebbe $P = NP$ — risolvendo così uno dei problemi aperti più famosi della matematica/informatica teorica. Naturalmente, questo non può essere vero con le conoscenze attuali (si crede fortemente che $P \neq NP$, anche se nessuno lo ha mai dimostrato): la battuta serve a farti capire che dimostrare BIPARTITE NP -completo è (quasi certamente) impossibile, dato quello che già sappiamo.

Esercizio 3

Questi problemi sono in NP ? (Sono NP -hard? Fai un'ipotesi...)

1. **HAMILTONIAN PATH:** dato un grafo, esiste un cammino che passa per ogni vertice esattamente una volta?

2. **EULERIAN PATH:** dato un grafo, esiste un cammino che passa per ogni **arco** esattamente una volta?
3. **TRAVELLING SALESMAN PROBLEM:** dato un grafo pesato G e un intero k , esiste un ciclo hamiltoniano in G di peso totale $\leq k$?
4. **INDEPENDENT SET:** dato un grafo (V, E) e un intero k , esiste un insieme $I \subseteq V$ di cardinalità k tale che $\forall u, v \in I, (u, v) \notin E$?
5. **HALT:** data una MT M e una stringa x , M termina sull'input x ?

Come si dimostra che un problema è in NP: la struttura certificato + verificatore. Un problema L è in NP se esiste un **certificato** y (di dimensione polinomiale rispetto all'input) e un **verificatore** M (che gira in tempo polinomiale) tali che: l'istanza appartiene a L se e solo se esiste un certificato y che M accetta insieme all'istanza. In pratica: non serve *trovare* la soluzione in tempo polinomiale, basta che, **una volta che qualcuno te la mostra**, tu riesca a **verificarla** velocemente.

1. HAMILTONIAN PATH

Input: un grafo G (dimensione polinomiale nella dimensione dell'input).

Certificato: un cammino hamiltoniano (dimensione polinomiale nella dimensione dell'input).

Si può verificare in tempo polinomiale che l'input è in HAM.PATH (basta controllare che il cammino dato passi davvero per ogni vertice esattamente una volta).

È **NP-completo**. Riduzione da HAM.CYCLE, che a sua volta è NP-completo per riduzione da 3-SAT.

2. EULERIAN PATH

Input: un grafo G .

Certificato: un cammino euleriano.

Si può verificare in tempo polinomiale che l'input è in EULER.PATH.

Sorpresa: è in P! Per il **Teorema di Eulero**: esiste un cammino euleriano se e solo se zero o due vertici hanno grado dispari, e il grafo è connesso. Questa condizione si può verificare in tempo polinomiale (basta contare i gradi dei vertici e controllare la connettività) — non serve nemmeno cercare esplicitamente il cammino!

3. TRAVELLING SALESMAN PROBLEM (TSP)

Input: un grafo G e un intero k (dimensione polinomiale nella dimensione dell'input).

Certificato: un ciclo hamiltoniano di peso totale $\geq k$.

Attenzione: possibile discrepanza nell'enunciato originale. La definizione iniziale del problema chiede peso totale $\leq k$, ma il certificato/la riduzione qui sotto usano “peso $\geq k$ ”. Non essendo certi di quale sia la versione corretta voluta dal professore, riportiamo qui esattamente quanto scritto nei suoi appunti (compresa l'incongruenza) — ti conviene chiedere conferma a lui su quale sia la versione esatta.

Si può verificare in tempo polinomiale che l'input è in TSP.

È NP-completo. La NP-hardness si dimostra per riduzione da HAM.CYCLE:

$$f : \text{input di HAM.CYCLE} \mapsto \text{input di TSP}$$

un grafo non pesato $(V, E) \mapsto$ (il grafo (V, E) con tutti gli archi di peso 0, l'intero 0)

- f è calcolabile in tempo polinomiale (si limita a etichettare ogni arco con peso 0);
- (V, E) ha un ciclo hamiltoniano se e solo se $f(V, E)$ ha un ciclo hamiltoniano di peso ≥ 0 (sempre vero, dato che tutti i pesi sono 0: un ciclo hamiltoniano qualsiasi ha peso esattamente $0 \geq 0$).

4. INDEPENDENT SET

Input: un grafo G , un intero k .

Certificato: un insieme indipendente $I \subseteq V$ di cardinalità k .

Si può verificare in tempo polinomiale che l'input è in IND.SET.

È NP-completo. La NP-hardness si dimostra per riduzione da 3-SAT.

Suggerimento del tutor (lasciato come esercizio): trasformare una formula del tipo $(L_{11} \vee L_{12} \vee L_{13}) \wedge \dots \wedge (L_{m1} \vee L_{m2} \vee L_{m3})$ in un grafo (V, E) dove $V = \{(i, j) \mid 1 \leq i \leq m, 1 \leq j \leq 3\}$ (un vertice per ogni letterale della formula), ed E esprime i vincoli logici imposti dalla formula.

5. HALT

HALT **non** è in NP, poiché $NP \subseteq EXPTIME \subseteq$ problemi decidibili, e HALT è indecidibile. Ma è comunque **NP-hard**! Questo è un fatto sorprendente: un problema può essere “più difficile di NP” (ind decidibile) e allo stesso tempo NP-hard.

Dimostrazione della NP-hardness.

Prendiamo un qualsiasi problema L in NP, cioè

$$L = \{x \in \{0, 1\}^* \mid \exists y \in \{0, 1\}^{p(|x|)}, M \text{ accetta } [x, y]\}$$

per un qualche polinomio p e una MT M che gira in tempo polinomiale.

Per ogni $x \in \{0, 1\}^*$, definiamo $f(x) := \llbracket M_x \rrbracket, x$, dove M_x è la macchina che fa così:

```

Leggi input  $x$ .
While true:
  For all  $y$  in  $\{0,1\}^p(|x|)$ :
    Simula  $M$  su  $[x,y]$ 
    Se accetta, accetta (ed esci)

```

Perché scrivere le istruzioni che simulano M costa tempo costante. M è fissata (è la macchina del problema L che abbiamo scelto all'inizio, non dipende da x): quindi scrivere il codice di M_x (che include, incorporate al suo interno, le istruzioni per simulare M) non dipende dalla dimensione di x — è sempre lo stesso codice “fisso”, a parte il valore di x stesso da incorporare.

Perché questo dimostra la riduzione.

- $f(x)$ è calcolabile in tempo polinomiale in $|x|$;
- $x \in L \iff \exists y \in \{0,1\}^{p(|x|)}, M \text{ accetta } [x,y] \iff M_x \text{ termina sull'input } x \iff f(x) \in HALT$.

Quindi $L \leq_p HALT$. Poiché L era un problema **arbitrario** in NP, questo dimostra che **ogni** problema in NP si riduce a HALT in tempo polinomiale — cioè HALT è NP-hard, anche se non è nemmeno decidibile.

Teorema

1. $\leq_p$ è un **preordine** (cioè è riflessiva e transitiva).
2. Se L è NP-hard e $L \in P$, allora $P = NP$.
3. Se L è NP-completo, allora $(L \in P) \iff P = NP$.

Dimostrazione

Cosa vuol dire “preordine”. Una relazione $\leq_p$ è un preordine se soddisfa due proprietà:

- **riflessiva:** ogni elemento è in relazione con se stesso, cioè $L \leq_p L$ per ogni L ;
- **transitiva:** se $A \leq_p B$ e $B \leq_p C$, allora $A \leq_p C$.

(Non si richiede invece l'**antisimmetria**, che servirebbe per un ordine parziale vero e proprio: puo benissimo capitare $A \leq_p B$ e $B \leq_p A$ senza che A e B siano lo stesso problema.)

Punto 1: $\leq_p$ è un preordine.

Riflessività: $L \leq_p L$. Basta prendere come funzione di riduzione l'identità $f(x) = x$: è banalmente calcolabile in tempo polinomiale, e $x \in L \iff f(x) = x \in L$ vale banalmente.

Transitività: se $A \leq_p B$ e $B \leq_p C$, allora $A \leq_p C$. Per ipotesi esistono due funzioni calcolabili in tempo polinomiale f e g tali che $x \in A \iff f(x) \in B$, e $y \in B \iff g(y) \in C$. Componendo le due funzioni, $h(x) := g(f(x))$ è ancora calcolabile in tempo polinomiale (comporre due algoritmi polinomiali dà ancora un algoritmo polinomiale), e:

$$x \in A \iff f(x) \in B \iff g(f(x)) \in C \iff h(x) \in C$$

quindi $A \leq_p C$.

Punto 2.

Supponiamo che L sia NP-hard e che L sia in P. Vogliamo provare, con questa ipotesi, che $NP \subseteq P$.

Prendiamo allora una qualsiasi lingua $H \in NP$, e proviamo a dimostrare che H è anche in P. Dalla definizione di NP-hardness, possiamo concludere che $H \leq_p L$.

Possiamo infine dire che H è in P, perché è al più tanto difficile quanto L , che sappiamo essere in P (se un problema si riduce a un altro che sta in P, anche il primo problema sta in P: risolvere L velocemente basta per risolvere H velocemente, tramite la riduzione).

Punto 3.

Supponiamo che L sia un problema NP-completo. Vogliamo dimostrare che $L \in P \iff P = NP$.

($\Rightarrow$) Supponiamo L sia in P. Sappiamo che L è anche NP-hard (essendo NP-completo, quindi NP-hard per definizione). Siamo allora in una situazione in cui il punto 2 può essere usato "off-the-shelf" (cioè applicato direttamente), concludendo che $P = NP$.

($\Leftarrow$) Supponiamo che $P = NP$. Poiché L è NP-completo, L è in $NP = P$: quindi L è anche in P stessa.

20/04/2026

Esempi di problemi NP-completi

Knapsack (zaino)

Dato un insieme di oggetti $\{1, \dots, n\}$, ciascuno con un peso w_i e un prezzo p_i , e data una capacità Z per lo zaino, determinare se esiste una scelta di oggetti $I \subseteq \{1, \dots, n\}$ tale che

$$\sum_{i \in I} w_i \leq Z \quad \sum_{i \in I} p_i \geq r$$

dove r è il valore obiettivo.

Certificato: una stringa binaria di lunghezza n che rappresenta I .

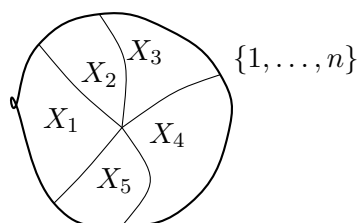
Verificatore: dato in input $\{w_i\}, \{p_i\}, Z, r, I$, verifica se le disuguaglianze $\sum_{i \in I} w_i \leq Z$ e $\sum_{i \in I} p_i \geq r$ valgono.

Esempio numerico. Supponiamo di avere 3 oggetti: peso/prezzo $(w_1, p_1) = (2, 3)$, $(w_2, p_2) = (3, 4)$, $(w_3, p_3) = (4, 5)$, capacità $Z = 5$, valore obiettivo $r = 7$.
Proviamo $I = \{1, 2\}$: $\sum w_i = 2 + 3 = 5 \leq 5 \checkmark$, $\sum p_i = 3 + 4 = 7 \geq 7 \checkmark$. Quindi $I = \{1, 2\}$ è un certificato valido: la risposta è “sì”.
Il certificato (la stringa binaria di lunghezza 3 che rappresenta I) sarebbe 110 (oggetto 1 e 2 inclusi, oggetto 3 no).

Bin-packing

Si hanno n oggetti $\{1, \dots, n\}$, ciascuno con una dimensione s_i , una dimensione di contenitore $B \in \mathbb{N}$, e un altro numero naturale $k \in \mathbb{N}$: si chiede se esiste una partizione $X_1, \dots, X_k$ di $\{1, \dots, n\}$ tale che

$$\sum_{i \in X_j} s_i \leq B \quad \forall j \in \{1, \dots, k\}$$



Partizione dell'insieme $\{1, \dots, n\}$ in $X_1, \dots, X_k$ (qui $k = 5$)

Certificato: una tupla di lunghezza n i cui elementi sono tutti in $\{1, \dots, k\}$. Il fatto che l' i -esimo elemento della tupla sia s significa che $i \in X_s$.

Verificatore: controlla che la disuguaglianza sopra sia soddisfatta per ogni j .

Cliques

Una **clique** in un grafo non orientato G è un sottoinsieme W di V tale che ogni coppia di nodi distinti $v, w \in W$ sia collegata da un arco nel grafo. Il problema chiede di determinare se un dato grafo non orientato G ha una clique di cardinalità almeno uguale a una soglia $k \in \mathbb{N}$.

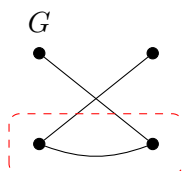
Certificato: (una codifica di) W , il sottoinsieme di V che stiamo cercando.

Verificatore: dati (G, k) e W , controlliamo che:

- $|W| \geq k$;
- ogni coppia di nodi distinti $v, w \in W$ sia collegata da un arco $\{v, w\} \in E$.

Vertex Cover

Dato un grafo non orientato G e un numero naturale $k \in \mathbb{N}$, ci interessa verificare se G ha una copertura di al più k nodi, dove una copertura di un grafo è un sottoinsieme dei suoi nodi tale che ogni arco abbia almeno uno dei suoi estremi nella copertura.



Esempio (con un grafo G a 4 nodi come sopra, con archi che si incrociano più il collegamento tra i due nodi cerchiati in rosso): $(G, 2)$ è in VERTEX COVER (i due nodi cerchiati in rosso coprono da soli tutti e 3 gli archi); $(G, 3)$ è in VERTEX COVER (banalmente, con un nodo in più si copre comunque tutto); $(G, 1)$ **non** è in VERTEX COVER (un solo nodo non può coprire i due archi “incrociati”, che non condividono nessun estremo).

Certificato: un sottoinsieme W di V .

Verificatore:

```

if |W| > k then
    return false
else
    foreach e in E do
        {v,w} <- e
        if v not in W and w not in W then
            return false
    return true

```

NP-hardness di Vertex Cover.

Lemma. Per ogni grafo $(V, E) = G$, un sottoinsieme $W \subseteq V$ è una vertex cover se e solo se $V \setminus W$ è un insieme indipendente:

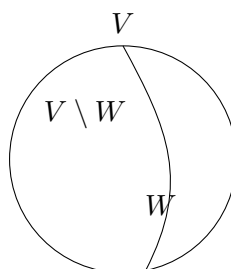
$$\begin{aligned}
 W \text{ è una vertex cover} &\iff \forall \{v, w\} \in E, v \in W \text{ oppure } w \in W \\
 &\iff \forall \{v, w\} \in E, \neg(v \in V \setminus W) \vee \neg(w \in V \setminus W) \\
 &\iff \forall \{v, w\} \in E, \neg(v \in V \setminus W \wedge w \in V \setminus W) \\
 &\iff V \setminus W \text{ è un insieme indipendente}
 \end{aligned}$$

Il passaggio logico chiave (De Morgan). “ $v \in W$ oppure $w \in W$ ” equivale a “non ($v \notin W$ e $w \notin W$)” — cioè $\neg A \vee \neg B \equiv \neg(A \wedge B)$, una delle leggi di De Morgan. Applicandola con $A = (v \in V \setminus W)$ e $B = (w \in V \setminus W)$, si ottiene esattamente la condizione di indipendenza per $V \setminus W$.

Sfruttare il lemma per la riduzione da IND.SET a VERTEX COVER.

Definiamo una funzione f tale che $x \in \text{INDSET} \iff f(x) \in \text{VERTEXCOVER}$:

$$f((G = (V, E)), k) = ((G = (V, E)), |V| - k)$$



Infatti, per il lemma:

$$\begin{aligned}(G, k) \in INDSET &\iff \exists W. INDSET(W, G) \wedge |W| \geq k \\ &\iff \exists W. VERTEXCOVER(V \setminus W, G) \wedge |W| \geq k \\ &\iff \exists W. VERTEXCOVER(V \setminus W, G) \wedge |V \setminus W| \leq |V| - k \\ &\iff (G, |V| - k) \in VERTEXCOVER\end{aligned}$$

Il passaggio dalla seconda alla terza riga usa semplicemente: $|W| \geq k \iff |V| - |W| \leq |V| - k \iff |V \setminus W| \leq |V| - k$ (dato che $|V \setminus W| = |V| - |W|$ per un sottoinsieme W di V).

5 27/04/2026

Esercizi

Classificare la complessità dei seguenti problemi, dimostrando la propria affermazione.

$$4SAT = \{F \in 4CNF \mid F \text{ è soddisfacibile}\}$$

$$3CLIQUEs = \{G \mid G \text{ contiene una clique di cardinalità } 3\}$$

$$VARINDSET = \{(G, k) \mid G \text{ è un grafo pesato (i nodi hanno una ricompensa),} \\ G \text{ ha un insieme indipendente di ricompensa totale almeno } k\}$$

$$GENHAMILTONIAN = \{(G, k) \mid G \text{ è un grafo i cui nodi hanno ciascuno un'altezza } h_v, \\ \text{e tale che } G \text{ ha un ciclo che passa per ogni nodo di altezza } k \text{ esattamente una volta}\}$$

Nota. Nell'immagine originale non compare la soluzione per GENHAMILTONIAN: probabilmente continua in una pagina successiva non ancora caricata.

a) 4SAT è NP-completo

La struttura della dimostrazione è sempre la stessa per i problemi NP-completi: (1) mostrare che il problema è in NP (certificato + verificatore), (2) mostrare la NP-hardness (di solito per riduzione da un problema già noto come NP-completo).

4SAT è in NP.

Perché SAT è in NP, e ogni istanza di 4SAT può essere verificata, insieme a un'assegnazione di valori di verità alle variabili sottostanti, per la soddisfacibilità, usando **esattamente lo stesso verificatore** già noto per SAT.

Perché lo stesso verificatore funziona. Una formula in 4CNF è comunque una formula proposizionale in forma normale congiuntiva (solo con clausole di 4 letterali invece di 3): il verificatore di SAT/3SAT — che semplicemente sostituisce i valori di verità dell'assegnazione e controlla se la formula risulta vera — funziona identico, a prescindere da quanti letterali ci sono in ciascuna clausola.

4SAT è NP-hard.

Basta notare che ogni 3CNF può essere trasformato in un 4CNF equivalente. Questo è facile:

$$(l_1 \vee l_2 \vee l_3) \longmapsto (l_1 \vee l_2 \vee l_3 \vee X), (l_1 \vee l_2 \vee l_3 \vee \neg X)$$

Si introduce una **nuova variabile** X (che non compariva nella formula originale), e si sostituisce ogni clausola di 3 letterali con **due** clausole di 4 letterali: una con X aggiunto, una con $\neg X$ aggiunto.

Perché preserva la soddisfacibilità. Se la clausola originale $(l_1 \vee l_2 \vee l_3)$ è vera (cioè almeno uno tra l_1, l_2, l_3 è vero), allora **entrambe** le nuove clausole sono vere, qualunque sia il valore di X (perché uno tra l_1, l_2, l_3 basta già a renderle vere). Se invece $(l_1 \vee l_2 \vee l_3)$ è falsa (tutti e tre falsi), allora le due nuove clausole diventano semplicemente X e $\neg X$: per essere entrambe vere serve X vero e $\neg X$ vero, il che è impossibile — quindi la nuova coppia di clausole è soddisfacibile se e solo se lo era la clausola originale.

Questa trasformazione preserva la soddisfacibilità, e può essere calcolata in tempo polinomiale. Possiamo concludere che $3SAT \leq_p 4SAT$.

b) 3CLIQUE

Sappiamo che il seguente problema è NP-completo:

$$CLIQUE = \{(G, k) \mid G \text{ ha una clique di cardinalità almeno } k\}$$

Ma **invece** di provare che 3CLIQUE (il caso speciale con $k = 3$ fissato) sia NP-completo, progettiamo direttamente un algoritmo in tempo polinomiale per esso!

L'algoritmo.

Input: G , un grafo non orientato.

Output: 0/1 a seconda che G abbia (o no) una clique di cardinalità 3.

```
for v in V do                                O(|G|)
  for w in V \ {v} do                         O(|G|)
    for z in V \ {v, w} do                   O(|G|)
      check whether {v, w, z} is a clique    2
      if so return 1
return false
```

Perché questo non contraddice l'NP-completezza di CLIQUES. CLIQUES (il problema generale, con k come parte dell'input, variabile) è NP-completo perché k può crescere insieme alla dimensione del grafo (nel caso peggiore, servirebbe controllare $\binom{|V|}{k}$ sottoinsiemi, esponenziale se k cresce con $|V|$). Ma qui $k = 3$ è **fissato**: bastano solo 3 cicli annidati (uno per ciascuno dei 3 nodi della potenziale clique), non uno per ogni possibile valore di k — quindi resta polinomiale.

- il numero totale di istruzioni è $O(|G|^3)$
- la dimensione dei risultati intermedi è polinomiale
- il controllo che facciamo nel corpo del ciclo più interno richiede tempo polinomiale

Quindi 3CLIQUES è in **P** (e quindi anche in NP, essendo $P \subseteq NP$).

c) VARINDSET è NP-completo

VARINDSET è in NP.

Si possono definire certificato e verificatore esattamente come in INDSET: il certificato è il sottoinsieme W di V che affermiamo essere indipendente e di ricompensa almeno k , mentre il verificatore controlla che W sia effettivamente indipendente e ne calcola la ricompensa totale.

VARINDSET è NP-hard.

Procediamo mostrando $INDSET \leq_p VARINDSET$. Dobbiamo trovare una funzione $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ tale che $f \in FP$ e $x \in INDSET \iff f(x) \in VARINDSET$.

$$(G, k) \xrightarrow{f} (G', k)$$

dove G' ha gli stessi nodi e archi di G , e le ricompense in G' sono tutte uguali a 1.

f è calcolabile in tempo polinomiale, perché produce in output un grafo con la stessa forma dell'originale, aggiungendo solo delle ricompense costanti (tutte 1) a ciascun nodo.

Inoltre:

$$\begin{aligned} (G, k) \in INDSET &\iff G \text{ ha un insieme indipendente di cardinalità } \geq k \\ &\iff G' \text{ ha un insieme indipendente di ricompensa totale } \geq k \\ &\iff (G', k) \in VARINDSET \iff f(G, k) \in VARINDSET \end{aligned}$$

Perché “cardinalità k ” equivale a “ricompensa totale k ” qui. Poiché ogni nodo in G' ha ricompensa esattamente 1, la ricompensa totale di un insieme di m nodi è semplicemente $m \times 1 = m$ — cioè coincide con la sua cardinalità. Ecco perché assegnare ricompensa 1 a tutti i nodi “traduce” esattamente la nozione di cardinalità in quella di ricompensa.

Teorema: INDSET è un problema NP-hard

Dimostrazione

Costruiamo una riduzione che testimonia il fatto che $3SAT \leq_p INDSET$, e questo sarà sufficiente. Dobbiamo trasformare una qualsiasi 3CNF F in un grafo $f(F)$ tale che F sia soddisfacibile se e solo se $f(F)$ è un grafo con un insieme indipendente “abbastanza grande”.

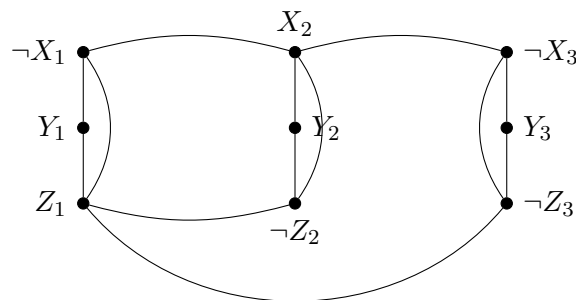
Costruzione del grafo.

Il grafo $f(C_1 \wedge C_2 \wedge \dots \wedge C_m)$ (dove ogni C_i è la disgiunzione di 3 letterali) ha $3m$ nodi, ciascuno corrispondente a un letterale in $C_1 \wedge \dots \wedge C_m$, e un certo numero di archi:

- tutti i possibili archi **interni** a ciascun C_i (cioè ogni coppia di letterali della stessa clausola è collegata — un “triangolo” per ogni clausola);
- archi tra letterali di clausole **distinte** che sono logicamente **incoerenti** (cioè sono nella forma l e $\neg l$).

Esempio.

$$f((\neg X \vee Y \vee Z) \wedge (X \vee Y \vee \neg Z) \wedge (\neg X \vee Y \vee \neg Z)) = (\dots, 3)$$



Perché la famiglia X forma un triangolo completo ma la famiglia Z no. Questo dettaglio specifico dipende dalla costruzione esatta scelta dal professore per questo esempio: non tutte le relazioni seguono necessariamente in modo simmetrico dalla regola generale enunciata (“ l e $\neg l$ ”) — ti conviene verificare con lui il criterio preciso, ma il disegno è fedelmente riportato così come descritto.

Quali archi (linee continue) compaiono, e perché. Un arco collega due letterali logicamente incoerenti (uno la negazione dell'altro) che stanno in clausole **diverse**:

- $\neg X$ (clausola 1) e X (clausola 2): incoerenti $\Rightarrow$ arco
- X (clausola 2) e $\neg X$ (clausola 3): incoerenti $\Rightarrow$ arco
- Z (clausola 1) e $\neg Z$ (clausola 2): incoerenti $\Rightarrow$ arco
- Z (clausola 1) e $\neg Z$ (clausola 3): incoerenti $\Rightarrow$ arco (l'arco lungo che passa sotto tutto)

Non c'è invece un arco tra $\neg X$ (clausola 1) e $\neg X$ (clausola 3), né tra $\neg Z$ (clausola 2) e $\neg Z$ (clausola 3): sono rispettivamente lo **stesso** letterale, non sono incoerenti tra loro.

Le linee **tratteggiate verticali** rappresentano invece gli archi “interni” a ciascuna clausola (il triangolo completo tra $\neg X, Y, Z$ nella prima colonna, e così via) — disegnati come un unico segmento verticale perché i tre nodi di ciascuna colonna sono allineati.

Perché f è calcolabile in tempo polinomiale.

Si limita a costruire un numero di nodi e archi proporzionale alla dimensione della formula — pura manipolazione sintattica.

Dobbiamo però convincerci che $F \in 3SAT \iff f(F) \in INDSET$.

($\Rightarrow$) Se $F \in 3SAT$, allora esiste un'assegnazione ρ alle variabili in $F = C_1 \wedge \dots \wedge C_m$ tale che $\llbracket F \rrbracket_\rho = 1$, cioè $\llbracket C_i \rrbracket_\rho = 1$ per ogni i .

Cosa significa $\llbracket F \rrbracket_\rho$. È una notazione compatta per “il valore di verità della formula F sotto l'assegnazione ρ ” — vale 1 se F risulta vera assegnando alle variabili i valori dati da ρ , 0 altrimenti.

Per ogni C_i c'è un letterale L_i al suo interno tale che $\llbracket L_i \rrbracket_\rho = 1$. Possiamo allora prendere come insieme indipendente in $f(F)$ l'insieme $W = \{L_1, \dots, L_m\}$.

Perché W è davvero indipendente. Non ci può essere nessun letterale della **stessa** C_i in W (per costruzione, ne scegliamo solo uno per clausola, quindi niente archi “interni” attivati). Inoltre non possiamo avere letterali logicamente incoerenti in W , perché $\llbracket L_i \rrbracket_\rho = 1$ per ogni $i \in \{1, \dots, m\}$: se L_i e L_j fossero incoerenti ($L_j = \neg L_i$), non potrebbero valere **entrambi** 1 sotto la stessa ρ .

($\Leftarrow$) Questo si può portare avanti in modo simile, e da un qualsiasi insieme indipendente W in $f(F)$ si può costruire un'assegnazione ρ che rende F vera.

6 30/04/2026 - Teoria

Teorema: 2SAT è in P

$$2SAT = \{F \mid F \text{ è una 2CNF che è soddisfacibile}\}$$

Dimostrazione

La cosa bella delle 2CNF è che ogni clausola al loro interno, diciamo $l_1 \vee l_2$, può essere vista come la congiunzione di **due implicazioni**:

$$\neg l_1 \Rightarrow l_2 \qquad \neg l_2 \Rightarrow l_1$$

Perché queste implicazioni equivalgono alla clausola originale. Ricorda che $A \Rightarrow B$ è logicamente equivalente a $\neg A \vee B$. Quindi:

$$\neg l_1 \Rightarrow l_2 \equiv \neg(\neg l_1) \vee l_2 \equiv l_1 \vee l_2$$

$$\neg l_2 \Rightarrow l_1 \equiv \neg(\neg l_2) \vee l_1 \equiv l_2 \vee l_1$$

Entrambe le implicazioni sono quindi *esattamente* la stessa clausola originale $l_1 \vee l_2$, solo scritta in un altro modo (una specie di “contronominale” vicendevole). Non stiamo aggiungendo nulla di nuovo: stiamo solo **re-interpretando** la stessa informazione logica come una relazione di causa-effetto tra letterali.

Costruzione del grafo delle implicazioni.

Possiamo allora organizzare tutti i letterali che compaiono possibilmente in F in un grafo tale che:

- i **nod**i sono i letterali $\neg X$ oppure X , dove X è una qualsiasi variabile proposizionale che compare in F ;
- gli **archi** sono diretti, e c'è un arco da l_1 a l_2 se $\neg l_1 \vee l_2$ è una clausola in F .

Quanti archi genera ogni clausola. Una clausola $(l_1 \vee l_2)$, essendo equivalente a **entrambe** le implicazioni viste sopra, genera **due** archi nel grafo: uno da $\neg l_1$ a l_2 (dalla prima implicazione), e uno da $\neg l_2$ a l_1 (dalla seconda).

Perché questo grafo è utile.

Si può dimostrare che le seguenti due condizioni sono equivalenti:

- F è insoddisfacibile;
- esiste una variabile X e una coppia di cammini nel grafo G : uno da X a $\neg X$, e uno da $\neg X$ a X (cioè X e $\neg X$ stanno nella stessa **componente fortemente connessa**).

Direzione ($\Leftarrow$): se ci sono cammini in entrambe le direzioni, F è insoddisfacibile.

Se sappiamo che in G ci sono cammini da X a $\neg X$ e da $\neg X$ a X , allora un qualsiasi assegnamento ρ tale che $\llbracket F \rrbracket_\rho = 1$ dovrebbe essere tale che: se $\rho(X) = 1$ allora $\rho(X) = 0$ (seguendo il cammino $X \rightarrow \neg X$, che rappresenta un'implicazione concatenata), e se $\rho(X) = 0$ allora $\rho(X) = 1$ (seguendo il cammino $\neg X \rightarrow X$). C'è quindi una contraddizione, e un tale ρ non può esistere.

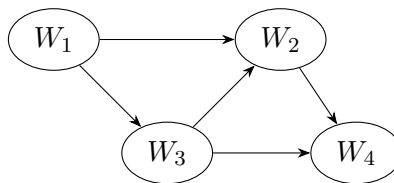
Perché un cammino nel grafo “forza” un’implicazione concatenata. Ogni arco nel grafo rappresenta un’implicazione che deve valere sotto un assegnamento soddisfacente (per come abbiamo costruito il grafo). Se ci sono archi consecutivi $l_1 \rightarrow l_2 \rightarrow l_3$, allora $l_1 \Rightarrow l_2$ e $l_2 \Rightarrow l_3$, quindi per transitività $l_1 \Rightarrow l_3$. Un cammino intero da X a $\neg X$ significa quindi che l’intera concatenazione di implicazioni forza $X \Rightarrow \neg X$: se X è vero, tutta la catena costringe anche $\neg X$ a essere vero, il che è assurdo se X è davvero vero. Da qui la contraddizione.

Direzione ($\Rightarrow$): il viceversa vale, ma è più delicato da dimostrare.

Questa implicazione vale, ma dimostrarla richiede di costruire un assegnamento soddisfacente ρ a partire da un qualsiasi grafo delle implicazioni **senza** “circoli viziosi” (cioè senza che nessuna variabile stia nella stessa componente fortemente connessa della sua negazione).

Cosa sono le componenti fortemente connesse (SCC). Un sottoinsieme $W \subseteq V$ è una componente fortemente connessa (SCC) se e solo se per ogni coppia di nodi $x, y \in W$ esiste un cammino da x a y (cioè, dentro W , si può sempre raggiungere qualunque nodo partendo da qualunque altro).

Questo si può fare organizzando i nodi del grafo in **SCC (componenti fortemente connesse)**.



È importante notare che, a livello “macro” (cioè considerando ogni SCC come un unico supernodo), questo grafo è **aciclico**. Possiamo allora costruire l’assegnamento ρ che cerchiamo ragionando “all’indietro” a livello delle SCC (cioè seguendo un ordinamento topologico del grafo aciclico delle SCC, decidendo i valori di verità partendo dalle componenti “più a valle” e procedendo a ritroso).

Perché il grafo delle SCC è sempre aciclico. Se ci fosse un ciclo tra due SCC diverse (es. $W_i \rightarrow W_j \rightarrow W_i$), allora tutti i nodi di W_i e W_j sarebbero mutuamente raggiungibili tra loro (attraverso il ciclo), e quindi W_i e W_j sarebbero in realtà la **stessa** componente fortemente connessa — contraddicendo il fatto che le avevamo definite come componenti distinte. Ecco perché il grafo “ridotto” alle sole SCC non può mai avere cicli.

Nota: DNF e DSAT

Le **DNF** sono, come sappiamo, forme normali disgiuntive. **DSAT**, il problema di verificare se una DNF sia soddisfacibile, è in **P**, perché una DNF $F = D_1 \vee D_2 \vee \dots \vee D_n$ è soddisfacibile se e solo se esiste una tra le D_i che non sia “contraddittoria”, cioè che non contenga sia un letterale che la sua negazione.

Perché basta controllare ogni D_i singolarmente. In una DNF, la formula intera è vera se **almeno una** delle disgiunzioni D_i è vera (essendo una disgiunzione di congiunzioni). Una singola D_i (congiunzione di letterali) può essere resa vera semplicemente assegnando 1 a ogni letterale positivo e 0 a ogni letterale negato al suo interno — **a meno che** D_i non contenga contemporaneamente sia l che $\neg l$ per qualche letterale l , nel qual caso D_i non può mai essere vera (è una contraddizione logica intrinseca). Basta quindi scorrere le D_i una alla volta (tempo polinomiale) e controllare se almeno una di esse non ha questo tipo di contraddizione interna.

7 04/05/2026

Teorema (PAC-learning di rettangoli)

Teorema (PAC-learning di rettangoli)

Per ogni distribuzione D , per ogni $0 < \varepsilon < 1/2$, e per ogni $0 < \delta < 1/2$, se $m \geq \frac{4}{\varepsilon} \ln \frac{4}{\delta}$, allora

$$\Pr_{D \sim D^m} \left[\text{error}_{D,T}(A_{BFT}(T(D))) < \varepsilon \right] > 1 - \delta$$

Cosa dice, in parole povere. Un algoritmo A_{BFT} “impara” un rettangolo T (il concetto target) a partire da m esempi di addestramento. Il teorema dice: se abbiamo **abbastanza** esempi (m grande a sufficienza rispetto a ε e δ), allora con **alta probabilità** ($> 1 - \delta$) l’errore dell’apprendimento è piccolo ($< \varepsilon$). Questo è il tipico schema di un risultato di **PAC-learning** (*Probably Approximately Correct*): probabilmente (con probabilità $\geq 1 - \delta$), l’algoritmo impara qualcosa di approssimativamente corretto (errore $< \varepsilon$).

Cosa significano i simboli.

- D è una distribuzione di probabilità sui punti dello spazio (da cui vengono estratti gli esempi);
- T è il rettangolo “vero” (il concetto target) che vogliamo imparare;
- $T(D)$ indica m esempi estratti secondo D , etichettati rispetto a T (cioè per ciascun punto si sa se sta dentro o fuori da T);
- A_{BFT} è l’algoritmo di apprendimento: dato il training set, restituisce il più piccolo rettangolo R che contiene tutti i punti positivi (dentro T) osservati — il “Best Fitting Tightest” rettangolo;
- $error_{D,T}(R)$ misura quanto R (il rettangolo appreso) sbaglia rispetto a T (il rettangolo vero), pesato secondo la distribuzione D .

Dimostrazione

Passo 1: srotoliamo la definizione di errore.

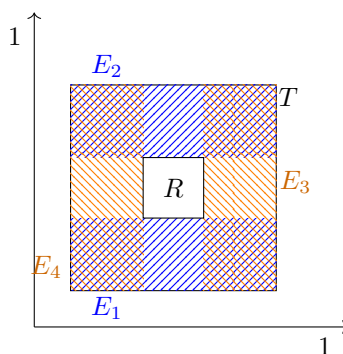
$$error_{D,T}(A_{BFT}(T(D))) = \Pr_{x \sim D} [x \in (R - T) \cup (T - R)]$$

dove R è il rettangolo prodotto in output da A_{BFT} sull’input $T(D)$.

Poiché $R \subseteq T$ (l’algoritmo costruisce R usando **solo** punti che sappiamo stare dentro T , quindi R non può mai “sconfinare” fuori da T), la parte $R - T$ è sempre vuota. Possiamo quindi semplificare l’equazione sopra in:

$$error_{D,T}(A_{BFT}(T(D))) = \Pr_{x \sim D} [x \in T - R]$$

Passo 2: organizziamo graficamente la regione $T - R$.

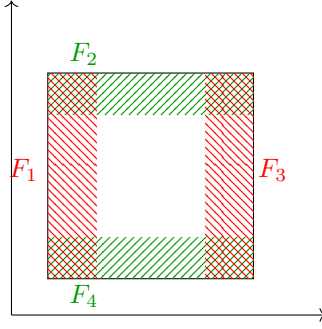


$$T - R = \bigcup_i E_i$$

L'evento probabilistico “ $x \in E_i$ ” vale precisamente quando **nessuno** dei dati di training era in E_i (perché se anche un solo punto di training fosse caduto in E_i , l'algoritmo A_{BFT} avrebbe esteso R fino a includerlo, eliminando quella porzione dalla “zona di errore”).

Passo 3: dimentichiamoci di R , e concentriamoci su T .

Definiamo quattro regioni $F_1, \dots, F_4$ come segue:



tali che $\Pr_{x \sim D}[x \in F_i] = \frac{\varepsilon}{4}$ per ogni i .

Perché queste 4 regioni, e perché proprio $\varepsilon/4$. Le F_i sono scelte **indipendentemente** da R (non dipendono da come va l'apprendimento): sono 4 strisce fisse ai bordi di T , ciascuna scelta in modo che la distribuzione D assegni loro esattamente probabilità $\varepsilon/4$ (una **per lato**, quindi $4 \times \varepsilon/4 = \varepsilon$ in totale se sommate). Sono un “righello di riferimento” per misurare quanto è vicino il bordo di R al bordo di T .

Passo 4: l'implicazione chiave.

“Alcuni punti rossi nel training set sono in ciascuna delle F_i ”

$$\begin{aligned} &\Rightarrow E_i \subseteq F_i \quad \forall i \\ &\Rightarrow \Pr_{x \sim D}[x \in E_i] \leq \Pr_{x \sim D}[x \in F_i] = \frac{\varepsilon}{4} \quad \forall i \\ &\Rightarrow \Pr_{x \sim D}[x \in E_i] \leq \frac{\varepsilon}{4} \\ &\Rightarrow \text{error}_{T,D}[R] \leq \sum_{i \in \{1, \dots, 4\}} \Pr[x \in E_i] \leq \varepsilon \end{aligned}$$

Perché $E_i \subseteq F_i$. Se c'è almeno un punto di training dentro F_i (che è più “verso l'interno” rispetto al bordo esatto di T), allora R si estende almeno fino a quel punto — quindi la zona di errore residua E_i (dove R non arriva) può al massimo essere la porzione di T ancora più vicina al bordo rispetto a quel punto, cioè E_i finisce per essere contenuta in F_i stessa.

Passo 5: la parte algebrica finale.

Vogliamo mostrare che, sotto l'ipotesi $m \geq \frac{4}{\varepsilon} \ln \frac{4}{\delta}$, la probabilità che il training set “manchi” completamente una delle 4 regioni F_i è al più δ .

$$\begin{aligned} m \geq \left(\ln \frac{4}{\delta} \right) \cdot \frac{4}{\varepsilon} &\Rightarrow \frac{\varepsilon \cdot m}{4} \geq \ln \frac{4}{\delta} \Rightarrow \frac{\varepsilon \cdot m}{4} + \ln \delta \geq \ln 4 \\ &\Rightarrow \ln 4 + \ln \left(e^{-\varepsilon/4} \right)^m \leq \ln \delta \\ &\Rightarrow \ln 4 \cdot \left(e^{-\varepsilon/4} \right)^m \leq \ln \delta \\ &\Rightarrow e^{-\varepsilon/4 \cdot m} \leq \delta \Rightarrow 4 \cdot \left(1 - \frac{\varepsilon}{4} \right)^m \leq \delta \\ &\Rightarrow \Pr \left[\exists i : \text{“nessuno dei punti nel training set è in } F_i \text{”} \right] \leq \delta \\ &\Rightarrow \Pr \left[\forall i : \text{“alcuni dei dati di training sono in } F_i \text{”} \right] \geq 1 - \delta \end{aligned}$$

Da dove viene $(1 - \frac{\varepsilon}{4})^m$. Ogni punto di training, indipendentemente, ha probabilità $\varepsilon/4$ di cadere in una data F_i (per costruzione), quindi probabilità $1 - \varepsilon/4$ di **non** caderci. Con m punti indipendenti, la probabilità che **nessuno** dei m punti cada in F_i è $(1 - \varepsilon/4)^m$. Moltiplicando per 4 (unione sulle 4 regioni possibili, con il union bound) si ottiene la probabilità che *almeno una* delle 4 regioni resti completamente “vuota”.

Il passaggio $e^{-\varepsilon/4}$. Si usa la disuguaglianza standard $1 - x \leq e^{-x}$ (vera per ogni x), applicata con $x = \varepsilon/4$: quindi $(1 - \frac{\varepsilon}{4})^m \leq (e^{-\varepsilon/4})^m = e^{-\varepsilon m/4}$.

Conclusione.

Mettendo insieme il Passo 4 (“qualche punto in ogni F_i ” $\Rightarrow$ errore $\leq \varepsilon$) e il Passo 5 (quella condizione vale con probabilità $\geq 1 - \delta$), otteniamo:

$$\Pr_{D \sim D^m} [\text{error}_{D,T}(A_{BFT}(T(D))) \leq \varepsilon] \geq 1 - \delta$$

che è (a meno di dettagli tecnici sulla disuguaglianza stretta/non stretta ai bordi) esattamente l'enunciato del teorema.

8 11/05/2026 - TEORIA

Teorema: le congiunzioni di letterali sono efficientemente PAC-learnabili

Dimostrazione

La classe delle istanze è quella degli assegnamenti a n variabili (indichiamo questi assegnamenti con x). Vogliamo mostrare che un algoritmo di apprendimento può, con abbastanza campioni, trovare una congiunzione di letterali h che approssima bene il concetto target c (anch'esso una congiunzione di letterali).

Il concetto di “letterale dannoso”.

Possiamo assegnare a ciascun letterale l la probabilità

$$p(l) = \Pr_{x \sim D} [c(x) = 1 \wedge l(x) = 0]$$

Cosa misura $p(l)$. c è il concetto target (la vera congiunzione di letterali che vogliamo imparare), mentre $l(x)$ è il valore di verità che otteniamo se consideriamo l come formula, applicata all'assegnamento x . $p(l)$ misura quindi: quanto spesso capita che il concetto target dica “vero” ($c(x) = 1$) ma il singolo letterale l dica “falso” ($l(x) = 0$) — cioè quanto l è “in disaccordo” col target sui positivi.

Un letterale l è **dannoso** (harmful) se $p(l) > \varepsilon/2n$.

Se l è tale che $p(l) > 0$, allora esiste almeno un x tale che $c(x) = 1$ e $l(x) = 0$: questo significa che l **non può** essere presente in c (perché se l facesse parte della congiunzione che definisce c , allora $c(x)$ dovrebbe essere 0 per quell' x , dato che $l(x) = 0$ — contraddizione con $c(x) = 1$).

Relazione tra letterali dannosi e l'errore.

Consideriamo prima di tutto la relazione tra letterali dannosi e l'errore

$$\text{error}(h) = \Pr_{x \sim D} [c(x) \neq h(x)]$$

Per un letterale dannoso l , sia A_l l'evento che, dopo $m > \frac{2n}{\varepsilon} \log \frac{2n}{\delta}$ campioni, il letterale l è ancora parte di h .

Quando si verifica A_l . Questo succede se, su m campioni consecutivi e indipendenti, un assegnamento x estratto da D tale che $c(x) = 1$ e $l(x) = 0$ **non** si presenta mai. Se un tale x si presentasse anche una sola volta, l'algoritmo di apprendimento si accorgerebbe che l è in disaccordo con il target su quel campione, e lo rimuoverebbe dall'ipotesi h .

Sia $E = \bigcup_{l \in B} A_l$, dove B è l'insieme di tutti i letterali dannosi. Allora:

$$\begin{aligned} \Pr[E] &\leq \sum_{l \in B} \Pr[A_l] \leq 2n \left(1 - \frac{\varepsilon}{2n}\right)^m \leq 2n (e^{-m})^{\varepsilon/2n} \\ &\leq 2n \left(e^{-\frac{2n}{\varepsilon} \log \frac{2n}{\delta}}\right)^{\varepsilon/2n} = 2n \cdot e^{-\log \frac{2n}{\delta}} = 2n \cdot \frac{\delta}{2n} = \delta \end{aligned}$$

I passaggi chiave di questo conto.

- $\Pr[E] \leq \sum_l \Pr[A_l]$: è il classico *union bound* (la probabilità di un'unione di eventi è al più la somma delle probabilità individuali);
- ci sono al più $2n$ letterali possibili (per ciascuna delle n variabili, il letterale positivo o negato), da cui il fattore $2n$;
- $(1 - \frac{\varepsilon}{2n})^m \leq (e^{-1})^{m\varepsilon/2n}$ usa la disuguaglianza standard $1 - x \leq e^{-x}$;
- sostituendo m con il suo valore minimo ipotizzato, l'esponente si semplifica esattamente fino a dare δ .

Cosa succede quando E non si verifica.

Vediamo ora cosa succede nel caso in cui h **non** includa nessun letterale dannoso. Sia B^c il complemento di B (l'insieme dei letterali che **non** sono dannosi). Allora:

$$\text{error}(h) = \Pr_{x \sim D} [c(x) = 1 \wedge h(x) = 0] \leq \sum_{l \in B^c} \Pr_{x \sim D} [c(x) = 1 \wedge l(x) = 0] \leq 2n \cdot \frac{\varepsilon}{2n} = \varepsilon$$

Perché vale questa disuguaglianza. Se $h(x) = 0$, vuol dire che **almeno uno** dei letterali che compongono h vale 0 su quell' x (essendo h una congiunzione). Quindi l'evento " $c(x) = 1 \wedge h(x) = 0$ " è contenuto nell'unione, su tutti i letterali l di h , dell'evento " $c(x) = 1 \wedge l(x) = 0$ ". Poiché stiamo assumendo che nessun letterale in h sia dannoso (cioè tutti hanno $p(l) \leq \varepsilon/2n$), sommando su al più $2n$ letterali possibili si ottiene il limite ε .

Conclusione.

Il fatto che $\Pr[E] \leq \delta$ e $\text{error}(h) \leq \varepsilon$ (ogniqualevolta h non contiene letterali dannosi) implica che:

$$\Pr_{x \sim D^m} [\text{err}(h) \leq \varepsilon] > 1 - \delta$$

che è la tesi.

Teorema: la classe dei 3-term DNF non è efficientemente PAC-learnabile, a meno che $RP = NP$

Teorema: la classe dei 3-term DNF non è efficientemente PAC-learnabile, a meno che $RP = NP$

Dimostrazione

Cos'è RP. RP è la classe dei linguaggi $L \subseteq \{0,1\}^*$ tali che esiste una macchina di Turing randomizzata in tempo polinomiale tale che:

- se $x \in L$, allora la MT restituisce 1 con probabilità almeno $1/2$;
- se $x \notin L$, allora la MT restituisce 0 con probabilità esattamente 1 (mai sbaglia sui “no”).

L'idea consiste nel disegnare una riduzione f (una funzione calcolabile in tempo polinomiale) tale che:

$$x \xrightarrow{f} (S_+, S_-)$$

dove S_+, S_- sono assegnamenti alle variabili proposizionali per un 3-term DNF target φ .

Nel progettare una tale f , ci assicureremo che:

- f è calcolabile in tempo polinomiale;
- $|S_+ \cup S_-|$ è limitata da un polinomio;
- vogliamo $x \in L$ se e solo se $f(x) = (S_+, S_-)$ è tale che esiste un 3-DNF φ per cui S_+ sono assegnamenti positivi per φ e S_- sono negativi per φ .

Se una tale f esiste per un problema NP-completo, allora quel problema è in RP, e quindi $RP = NP$.

Assumiamo che esista un algoritmo di apprendimento efficiente B per i 3-term DNF. Costruiamo, a partire da esso, un algoritmo di decisione randomizzato A per un problema NP-completo $\mathcal{L}$:

- l'algoritmo A chiama prima f sul suo input x , ottenendo (S_+, S_-) come sopra;
- l'algoritmo A interroga poi B usando (S_+, S_-) come dati etichettati, uniformemente distribuiti, e con $\varepsilon = \frac{1}{2|S_+ \cup S_-|}$, $\delta = 1/2$;
- B produce una formula φ . A controlla allora che φ sia **consistente** con (S_+, S_-) . Se φ è consistente con (S_+, S_-) , allora $x \in \mathcal{L}$, altrimenti $x \notin \mathcal{L}$.

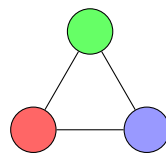
Perché l'algoritmo A restituisce 0 e 1 con le probabilità corrette (secondo la definizione di RP).

Caso $x \in \mathcal{L}$. Sappiamo che un 3-term DNF φ consistente con (S_+, S_-) esiste effettivamente. Per il fatto che B è un algoritmo PAC corretto, con probabilità almeno $1/2$, $error(h) \leq \frac{1}{2}|S_+ \cup S_-|$. Ma ci sono solo $|S_+ \cup S_-|$ istanze che estraiamo dalla distribuzione con cui lavoriamo, ciascuna con la stessa probabilità. In altre parole: h non può produrre nessun errore, perché siamo garantiti che $error(h) \leq \frac{1}{2}|S_+ \cup S_-|$, e questo è più piccolo della probabilità di ciascuno degli assegnamenti in $S_+ \cup S_-$.

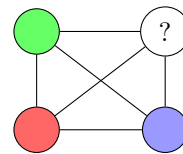
Caso $x \notin \mathcal{L}$. Non esiste nessun 3-term DNF che assegni correttamente valori di verità a S_+ e S_- . L'algoritmo di apprendimento, quindi, non può restituire un tale h , e noi (l'algoritmo A) possiamo facilmente accorgerci che è questo il caso interrogando h con gli assegnamenti in S_+ e S_- . Si noti che, in accordo con la definizione di RP, restituiamo sempre 0, cioè restituiamo 0 con probabilità 1.

Il problema NP-completo scelto: la 3-colorabilità dei grafi

Il problema NP-completo che stiamo cercando è quello dei grafi **3-colorabili**: grafi non orientati i cui nodi possono ricevere colori in $\{r, g, b\}$ (rosso, verde, blu) tali che i nodi adiacenti siano sempre colorati in modo diverso.



3-colorabile



non 3-colorabile

Perché il secondo grafo non è 3-colorabile. Il nodo con il punto interrogativo è collegato a **tutti e tre** gli altri nodi (verde, rosso, blu – che già usano tutti e 3 i colori disponibili r, g, b): qualunque colore si scelga per esso, sarà sempre uguale al colore di uno dei suoi vicini. Questo è il grafo completo K_4 (ogni nodo collegato a ogni altro): servirebbero almeno 4 colori per colorarlo correttamente.

La riduzione finale.

Per i grafi 3-colorabili (che formano un problema NP-completo), si può progettare una riduzione f come sopra, assicurandosi che G sia 3-colorabile se e solo se $f(G) = (S_+, S_-)$ è tale che esiste un 3-CNF φ consistente con esso.

9 18/05/2026

VC-dimension

$$VC(\mathcal{H}) = \max\{|S| \mid S \text{ è shattered da } \mathcal{H}\}$$

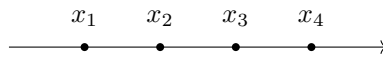
Cos'è la VC-dimension, in parole povere. $\mathcal{H}$ è una classe di ipotesi (un insieme di “concetti” possibili — es. tutti i possibili rettangoli, tutti i possibili triangoli, ecc.). Un insieme di punti S è **shattered** (frantumato) da $\mathcal{H}$ se, per **ogni** possibile modo di etichettare i punti di S come “dentro/fuori” (positivo/negativo), esiste **almeno un** concetto in $\mathcal{H}$ che realizza esattamente quell'etichettatura. La VC-dimension è la dimensione del **più grande** insieme di punti che $\mathcal{H}$ riesce a shatterare in tutti i modi possibili.

Esempio. Sia $\mathcal{H} =$ “tutti i possibili intervalli $[a, b]$ su $\mathbb{R}$ ”. Con 2 punti x_1, x_2 : ci sono $2^2 = 4$ etichettature possibili, e si può sempre trovare un intervallo che realizza ciascuna. Con 3 punti $x_1 < x_2 < x_3$: l'etichettatura “dentro, fuori, dentro” è impossibile (un intervallo è connesso, non può “saltare” x_2 lasciandolo fuori se include sia x_1 che x_3). Quindi $VC(\text{intervalli}) = 2$.

Esercizio 1: VC-dimension delle unioni di 2 intervalli in $\mathbb{R}$

Lower bound: $VC(\mathcal{H}) \geq 4$

Proviamo che esiste un insieme di 4 elementi che può essere shattered. Prendiamo $x_1 < x_2 < x_3 < x_4 \in \mathbb{R}$.



Etichetta	Punti catturati	Intervalli usati
0000	nessuno	(nessun intervallo, o due vuoti)
1000	x_1	$[x_1, x_1]$
0100	x_2	$[x_2, x_2]$
0010	x_3	$[x_3, x_3]$
0001	x_4	$[x_4, x_4]$
1100	x_1, x_2	$[x_1, x_2]$ (un solo intervallo)
0110	x_2, x_3	$[x_2, x_3]$
0011	x_3, x_4	$[x_3, x_4]$
1010	x_1, x_3	$[x_1, x_1] \cup [x_3, x_3]$
0101	x_2, x_4	$[x_2, x_2] \cup [x_4, x_4]$
1001	x_1, x_4	$[x_1, x_1] \cup [x_4, x_4]$
1110	x_1, x_2, x_3	$[x_1, x_3]$ (un solo intervallo)
0111	x_2, x_3, x_4	$[x_2, x_4]$
1101	x_1, x_2, x_4	$[x_1, x_2] \cup [x_4, x_4]$
1011	x_1, x_3, x_4	$[x_1, x_1] \cup [x_3, x_4]$
1111	x_1, x_2, x_3, x_4	$[x_1, x_4]$ (un solo intervallo)

Come si legge la tabella. Ogni riga rappresenta una delle $2^4 = 16$ possibili etichettature dei 4 punti:

- la colonna **Etichetta** è una stringa di 4 bit: il bit in posizione i vale 1 se x_i deve essere *catturato* (incluso nell'unione di intervalli), 0 se deve restare *escluso*. Ad esempio “1010” significa: x_1 dentro, x_2 fuori, x_3 dentro, x_4 fuori;

- la colonna **Punti catturati** traduce semplicemente quella stringa in un elenco esplicito dei punti che devono risultare “dentro”;
- la colonna **Intervalli usati** mostra **una** scelta concreta di (al più due) intervalli che realizza esattamente quell’etichettatura — è la prova costruttiva che quell’etichettatura è effettivamente raggiungibile.

Per dimostrare che l’insieme $\{x_1, x_2, x_3, x_4\}$ è shattered, dobbiamo esibire una costruzione valida per **ciascuna** delle 16 righe: se anche una sola etichettatura risultasse irrealizzabile, l’insieme non sarebbe shattered.

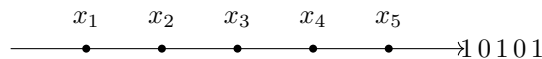
Con un’unione di 2 intervalli, si può “catturare” (includere) qualsiasi sottoinsieme dei 4 punti che formi al più **2 blocchi contigui** nella sequenza ordinata. Ad esempio:

- “1010” (cattura x_1, x_3): due blocchi separati, ciascuno di un punto solo — 2 intervalli piccoli;
- “1100” (cattura x_1, x_2): un blocco contiguo di 2 punti — basta un solo intervallo;
- “1001” (cattura x_1, x_4): due blocchi separati agli estremi — 2 intervalli.

Con soli 4 punti, il numero massimo di blocchi contigui separati possibile è 2: quindi **tutte** le $2^4 = 16$ etichettature sono realizzabili con al più 2 intervalli.

Upper bound: $VC(\mathcal{H}) < 5$

Prendiamo 5 punti $x_1 < \dots < x_5$, e consideriamo l’etichettatura “10101” (cattura x_1, x_3, x_5 , esclude x_2, x_4).



Per catturare x_1 senza x_2 , x_3 senza x_2 né x_4 , e x_5 senza x_4 , servirebbero **3 blocchi separati** (x_1, x_3, x_5 ciascuno isolato). Ma un’unione di 2 intervalli può formare al massimo 2 blocchi separati: contraddizione. Nessun’unione di due intervalli può catturare il sottoinsieme $\{x_1, x_3, x_5\}$, perché catturarlo richiederebbe almeno 3 intervalli.

Come conseguenza, $VC(\mathcal{H}) = 4$.

Esercizio 2: se $\mathcal{H} \subseteq \mathcal{H}'$ allora $VC(\mathcal{H}) \leq VC(\mathcal{H}')$

Possiamo seguire direttamente la definizione:

$$VC(\mathcal{H}) = \max_{S \subseteq X} \{|S| \mid S \text{ shattered da } \mathcal{H}\} \leq \max_{S \subseteq X} \{|S| \mid S \text{ shattered da } \mathcal{H}'\} = VC(\mathcal{H}')$$

Perché vale questa disuguaglianza. Se un insieme S è shattered da $\mathcal{H}$ (cioè $\mathcal{H}$ realizza tutte le $2^{|S|}$ etichettature possibili), allora S è shattered **anche** da $\mathcal{H}'$ — perché $\mathcal{H}'$ contiene tutte le ipotesi di $\mathcal{H}$ (essendo $\mathcal{H} \subseteq \mathcal{H}'$), quindi ha a disposizione almeno le stesse ipotesi per realizzare ogni etichettatura, e magari anche di più.

Esercizio 3: $VC(\mathcal{H}), VC(\mathcal{H}') \leq VC(\mathcal{H} \cup \mathcal{H}') \leq VC(\mathcal{H}) + VC(\mathcal{H}') + 1$

Nota. Il tutor lascia questo esercizio **incompleto** nei suoi appunti (segnato con una X): fornisce solo la parte sinistra della disuguaglianza. La parte destra, più delicata, viene completata qui, usando un risultato (il Lemma di Sauer-Shelah) che **non** è stato introdotto esplicitamente a lezione.

La parte sinistra (facile): $VC(\mathcal{H}), VC(\mathcal{H}') \leq VC(\mathcal{H} \cup \mathcal{H}')$

È una conseguenza immediata dell'Esercizio 2. Dato che $\mathcal{H} \subseteq \mathcal{H} \cup \mathcal{H}'$, l'Esercizio 2 ci dà direttamente $VC(\mathcal{H}) \leq VC(\mathcal{H} \cup \mathcal{H}')$. Stesso discorso per $\mathcal{H}'$.

La parte destra (difficile): $VC(\mathcal{H} \cup \mathcal{H}') \leq VC(\mathcal{H}) + VC(\mathcal{H}') + 1$

Lemma di Sauer-Shelah. Se $VC(\mathcal{H}) = d$, allora per qualunque insieme S con $|S| = n$, il numero di etichettature **distinte** che $\mathcal{H}$ riesce a realizzare su S è al massimo $\sum_{i=0}^d \binom{n}{i}$.

Intuizione del lemma. Se $\mathcal{H}$ ha VC-dimension bassa, non può “fare troppe cose diverse” su un insieme grande: il numero di comportamenti possibili cresce solo **polinomialmente** in n (non esponenzialmente come 2^n), a differenza di quanto ci si aspetterebbe se $\mathcal{H}$ fosse arbitrariamente espressiva.

Come si usa per dimostrare la parte destra.

Sia $d = VC(\mathcal{H})$ e $d' = VC(\mathcal{H}')$. Se S (con $|S| = n$) è shattered da $\mathcal{H} \cup \mathcal{H}'$, allora ogni etichettatura di S è realizzata o da $\mathcal{H}$ o da $\mathcal{H}'$ (o entrambe): quindi il numero di etichettature realizzabili da $\mathcal{H} \cup \mathcal{H}'$ è al più la **somma** di quelle realizzabili singolarmente:

$$2^n \leq \sum_{i=0}^d \binom{n}{i} + \sum_{i=0}^{d'} \binom{n}{i}$$

Perché questa disuguaglianza “fallisce” per n abbastanza grande (il punto più delicato). Il lato sinistro, 2^n , cresce **esponenzialmente** in n . Il lato destro, invece, è una somma di un numero **fisso** di termini binomiali (fino a d e d' , che sono costanti fissate, non crescono con n): per n grande, questa somma è un **polinomio** in n di grado d (rispettivamente d'), quindi cresce molto più lentamente di 2^n .

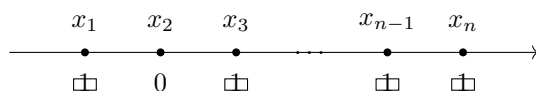
Si può dimostrare rigorosamente (è un fatto combinatorio standard, la cui dimostrazione completa richiede un po' di lavoro algebrico che esula da questo corso) che, non appena $n \geq d + d' + 2$, il lato destro diventa **più piccolo** del lato sinistro 2^n — rendendo la disuguaglianza impossibile da soddisfare. Quindi S **non può** essere shattered se $|S| \geq d + d' + 2$, il che significa $VC(\mathcal{H} \cup \mathcal{H}') \leq d + d' + 1$.

Esercizio 4: VC-dimension della classe delle unioni (finite) di intervalli

Qui la classe $\mathcal{H}$ è più permissiva rispetto all'Esercizio 1: non “al più 2 intervalli”, ma un **numero qualsiasi** (finito) di intervalli.

Ogni sottoinsieme (finito!) S di $\mathbb{R}$ può essere shattered da unioni finite di intervalli. Se $S = \{x_1 < x_2 < \dots < x_n\}$ e $S' \subseteq S$, S' può essere “catturato” da esattamente $|S'|$ intervalli, ciascuno circondante un $x_i \in S'$.

Esempio.



In questo esempio, $S' = \{x_1, x_3, x_{n-1}, x_n\}$ (i punti etichettati 1) viene catturato usando **esattamente 4 intervalli** piccolissimi, ciascuno che circonda un solo punto di S' senza toccare gli altri punti di S (in particolare, senza toccare x_2 , che deve restare escluso). Si noti che anche x_{n-1} e x_n , pur essendo etichette adiacenti, ricevono **due intervalli separati** (non uno solo che li unisca): questo per rispettare la regola generale “esattamente $|S'|$ intervalli”, anche se in questo caso specifico un solo intervallo che copra entrambi avrebbe comunque funzionato.

Perché questo funziona sempre, per ogni n .

Non c'è **nessun limite** al numero di intervalli che possiamo usare: quindi, qualunque sia $|S'|$ (da 0 fino a tutti gli n punti), possiamo sempre creare esattamente tanti intervallini isolati quanti servono, uno per ciascun punto da catturare, senza mai includere per sbaglio un punto che non doveva esserci (basta scegliere intervalli sufficientemente piccoli attorno a ciascun punto target).

Poiché questo funziona per **ogni** n (non c'è un valore oltre il quale si rompe, a differenza dell'Esercizio 1 dove il limite era 4), la VC-dimension di questa classe è **infinita**:

$$VC(\mathcal{H}) = \infty$$

Il confronto con l'Esercizio 1. Nell'Esercizio 1 il vincolo “al più 2 intervalli” era esattamente ciò che limitava la VC-dimension a un valore finito (4). Togliendo quel vincolo (permettendo un numero arbitrario di intervalli), la classe diventa “troppo potente”, capace di shatterare insiemi di qualsiasi dimensione, e la VC-dimension esplode a infinito.

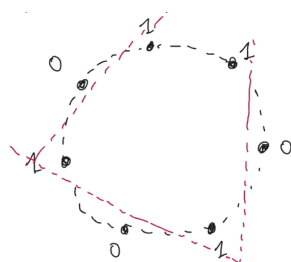
Esercizio 5: VC-dimension dei triangoli

Nota. Il tutor segna questo esercizio come **non completato** (X): fornisce solo l'idea generale del lower bound, dicendo esplicitamente che “l'upper bound è altamente non banale”. Qui riportiamo fedelmente quanto scritto dal tutor per il lower bound, e aggiungiamo (segnalandolo chiaramente come nostro completamento) qualche dettaglio in più per rendere l'idea più concreta. L'upper bound rigoroso resta, come detto dal tutor stesso, al di fuori della portata di questi appunti.

$VC(\mathcal{H}) = 7$, dove $\mathcal{H}$ è la classe di tutti i possibili triangoli nel piano $\mathbb{R}^2$.

Lower bound (dal tutor)

Si organizzano i sette punti in $\mathbb{R}^2$ attorno a un cerchio, a distanza radiale costante l'uno dall'altro (cioè equamente distanziati, come i vertici di un ettagono regolare).



L'idea intuitiva (completamento nostro). Con i 7 punti disposti in **posizione convessa** su un cerchio, per qualsiasi sottoinsieme S' che vogliamo catturare, i punti **esclusi** (quelli non in S') si raggruppano lungo il cerchio in al più **3 archi contigui** separati. Un triangolo ha esattamente **3 lati**: ciascun lato può essere posizionato in modo da “tagliare fuori” uno di questi archi di punti esclusi, lasciando dentro al triangolo esattamente i punti di S' .

Questo è lo stesso principio dell'Esercizio 1 (dove 2 intervalli — ciascuno un segmento con 2 “lati” — riuscivano a isolare al più 2 gruppi contigui su una retta): qui, con punti disposti su un cerchio invece che su una retta, un poligono di k lati può isolare fino a $2k + 1$ punti in questo modo. Per $k = 3$ (triangolo), si ottiene $2 \cdot 3 + 1 = 7$.

Upper bound

Il tutor annota esplicitamente: “l’upper bound è altamente non banale”. Non viene fornita una dimostrazione, né a lezione né nei suoi appunti. Dimostrare rigorosamente che **nessun** insieme di 8 punti può essere shattered da triangoli richiede argomenti di geometria combinatoria più avanzati (tipicamente legati al Teorema di Radon o risultati simili sulla combinatoria di insiemi convessi), che esulano dallo scopo di questi appunti.

Esercizio 6: funzioni di parità su stringhe

Nota. Il tutor segna questo esercizio con una X nei suoi appunti, ma in realtà lo svolge **per intero** (sia lower che upper bound) — probabilmente la X indica solo che non ha fatto in tempo a discuterlo a voce durante la lezione, non che sia incompleto.

Qui $X = \{0, 1\}^d$, mentre $\mathcal{H}$ è l’insieme delle funzioni di parità su X , cioè:

$$\mathcal{H} = \{h_a \mid h_a(x) = a \odot x\}$$

Il prodotto interno tra stringhe ($\odot$). È lo XOR di tutti i prodotti bit a bit. Esempio:

$$0010 \odot 0110 = (0 \cdot 0) \oplus (0 \cdot 1) \oplus (1 \cdot 1) \oplus (0 \cdot 0) = 0 \oplus 0 \oplus 1 \oplus 0 = 1$$

dove $\oplus$ è lo XOR (somma modulo 2):

b	c	$b \oplus c$
0	0	0
0	1	1
1	0	1
1	1	0

La VC-dimension di $\mathcal{H}$ è esattamente uguale a d .

Lower bound: $VC(\mathcal{H}) \geq d$

Dobbiamo trovare un sottoinsieme S di X con $|S| = d$ tale che $\mathcal{H}$ shatteri S . Possiamo scegliere $S = \{e_1, \dots, e_d\}$, dove e_i è la stringa con un 1 in posizione i e 0 altrove (cioè il “vettore unitario” i -esimo).

Dobbiamo convincerci che ogni sottoinsieme S' di S può essere catturato precisamente da qualche funzione di parità. Sia S' il sottoinsieme di S contenente $e_{i_1}, \dots, e_{i_k}$. Poniamo a uguale alla stringa in $\{0, 1\}^d$ tale che: il j -esimo bit di a è 1 se e solo se $j \in \{i_1, \dots, i_k\}$. Verifichiamo che $h_a(e_j) = 1$ se e solo se $j \in \{i_1, \dots, i_k\}$:

$$h_a(e_j) = a \odot e_j$$

e $a \odot e_j$ è lo XOR di bit che sono tutti uguali a 0 tranne sulla coordinata uguale a j , dove valgono 1.

Perché $a \odot e_j$ “estrae” esattamente il j -esimo bit di a . Poiché e_j ha un 1 solo in posizione j e 0 ovunque altro, il prodotto bit a bit $a \cdot e_j$ è 0 ovunque tranne in posizione j , dove vale a_j (il j -esimo bit di a). Lo XOR di una sequenza con un solo bit potenzialmente non-nullo restituisce semplicemente quel bit: quindi $a \odot e_j = a_j$. Per come abbiamo costruito a , $a_j = 1$ esattamente quando $j \in \{i_1, \dots, i_k\}$ — cioè esattamente quando $e_j \in S'$. Questo mostra che h_a realizza esattamente l’etichettatura che corrisponde a S' .

Upper bound: non si può shatterare nulla di “più grande” di d

Se $|S| = d + 1$, siamo in presenza di un insieme di stringhe che, viste come vettori, sono elementi di uno spazio vettoriale di dimensione d .

Uno di questi $d + 1$ vettori è necessariamente **linearmente dipendente** dagli altri (un fatto standard di algebra lineare: in uno spazio di dimensione d , non possono esistere più di d vettori linearmente indipendenti). E poiché le funzioni di parità sono **lineari**, i loro valori su questo $(d + 1)$ -esimo vettore sono **determinati** dai loro valori sugli altri d vettori.

Perché la linearità “blocca” la libertà di scelta. Se il vettore v_{d+1} è combinazione lineare (su $GF(2)$, cioè XOR) di alcuni tra $v_1, \dots, v_d$, allora, essendo h_a lineare (cioè $h_a(u \oplus v) = h_a(u) \oplus h_a(v)$), il valore $h_a(v_{d+1})$ è automaticamente determinato una volta fissati i valori $h_a(v_1), \dots, h_a(v_d)$ — non possiamo scegliere $h_a(v_{d+1})$ liberamente e indipendentemente dagli altri. Questo significa che, qualunque etichettatura vogliamo imporre sui primi d vettori, l’etichetta sul $(d + 1)$ -esimo vettore è già fissata di conseguenza: non tutte le 2^{d+1} etichettature sono raggiungibili.

Come conseguenza, c’è un sottoinsieme di S che non può essere catturato, e S non può essere shattered. Quindi $VC(\mathcal{H}) \leq d$, e combinato con il lower bound: $VC(\mathcal{H}) = d$.

Esercizio 1

Indichiamo con $|w|_0$ il numero di 0 nella parola $w \in \{0, 1\}^*$.

Parte 1: TM che decide il linguaggio

$$\alpha := \{w \in \{0, 1\}^* \mid |w|_0 \bmod 4 = 0\}$$

Qual è la sua complessità?

α contiene tutte le stringhe che hanno un numero di 0 **multiplo di 4** (0, 4, 8, 12, ...). L'idea è la stessa dell'esercizio sulla parità: invece di tracciare “pari/dispari” (2 possibilità), qui tracciamo il **resto della divisione per 4** (4 possibilità: 0, 1, 2, 3).

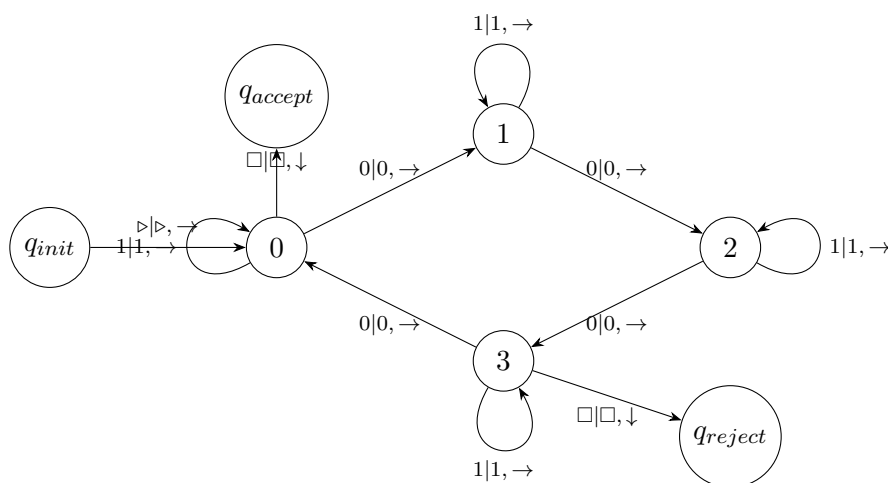
I 4 stati “contatore”.

Usiamo 4 stati, chiamati 0, 1, 2, 3 — ciascuno rappresenta “finora ho visto un numero di 0 con resto k modulo 4”.

- se leggi un 1: il conteggio degli 0 non cambia (non hai appena visto uno 0), quindi **resti nello stesso stato**;
- se leggi uno 0: il conteggio aumenta di 1, quindi passi allo stato “successivo” (da k a $k + 1 \bmod 4$).

Lo stato si muove **solo** quando succede l'evento che stiamo contando (vedere uno 0): un 1 è come se “non esistesse” per il contatore.

L'automa.



Nel disegno sono mostrate esplicitamente solo le transizioni su $\square$ da 0 (verso accept) e da 3 (verso reject) per non appesantire troppo la figura: anche da 1 e 2, leggendo $\square$, si va in q_{reject} (con la stessa logica: solo resto 0 accetta).

Definizione formale.

$$Q = \{q_{init}, 0, 1, 2, 3, q_{accept}, q_{reject}\} \quad \Gamma = \{0, 1, \square, \triangleright\}$$

$$\delta : \begin{cases} (q_{init}, \triangleright) \mapsto (0, \triangleright, \rightarrow) \\ (k, 0) \mapsto (k+1 \bmod 4, 0, \rightarrow) & \text{per } 0 \leq k \leq 3 \\ (k, 1) \mapsto (k, 1, \rightarrow) \\ (0, \square) \mapsto (q_{accept}, 0, \downarrow) \\ (k, \square) \mapsto (q_{reject}, 1, \downarrow) & \text{per } 0 < k \leq 3 \end{cases}$$

Simulazione passo-passo: $w = 10101010$ (4 zeri).

Nastro iniziale: $\triangleright 10101010$

Passo	Stato	Simbolo	Azione
1	q_{init}	$\triangleright$	vai in 0, muovi $\rightarrow$
2	0	1	non è 0: resta in 0, muovi $\rightarrow$
3	0	0	1° zero: vai in 1, muovi $\rightarrow$
4	1	1	resta in 1, muovi $\rightarrow$
5	1	0	2° zero: vai in 2, muovi $\rightarrow$
6	2	1	resta in 2, muovi $\rightarrow$
7	2	0	3° zero: vai in 3, muovi $\rightarrow$
8	3	1	resta in 3, muovi $\rightarrow$
9	3	0	4° zero: $3+1 \bmod 4=0$, torni in 0, muovi $\rightarrow$
10	0	$\square$	fine, stato 0 $\rightarrow q_{accept}$

La macchina accetta $w = 10101010$ — corretto, perché w ha esattamente 4 zeri, e $4 \bmod 4 = 0$.

Complessità in tempo.

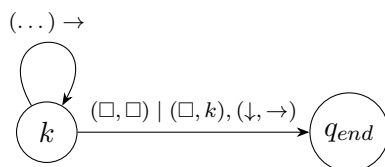
La macchina scorre la stringa una sola volta, senza mai tornare indietro. Se $|w| = n$: $n + 1 = O(n)$ passi (tempo lineare).

Quindi il problema/linguaggio corrispondente α è (al più) in **P**.

Parte 2: TM che calcola la funzione

Scrivere una TM che calcola la funzione $\{0, 1\}^* \rightarrow \{0, 1, 2, 3\}$, $w \mapsto |w|_0 \bmod 4$.

Idea del tutor: è la stessa macchina di prima, ma invece di accettare/rifiutare, si scrive il nome dello stato corrente sul nastro di output quando si raggiunge la fine dell'input.



Perché funziona così facilmente. Il numero dello stato è già la risposta cercata: non serve nessun calcolo aggiuntivo, perché si stava già “contando modulo 4” per decidere se accettare o no. Ora basta rendere visibile quel conteggio (scrivendolo sul nastro di output) invece di usarlo solo per una decisione binaria.

Complessità.

Stessa identica macchina, stesso numero di passi: $O(n)$.

Questa funzione è (al più) in FP (non P , perché qui non stiamo decidendo un linguaggio, ma calcolando una funzione con output).

Esercizio 2

Consideriamo il seguente problema:

Input: n città $C_1, \dots, C_n$, il tempo di percorrenza $t(C_i, C_j)$ tra città C_i e C_j ($\forall i, j$), la durata della giornata T .

Output: è possibile per un'azienda che possiede due camion organizzare due percorsi (ciclici) tali che ogni città sia visitata una volta al giorno?

Punto 1: questo problema è in NP?

L'input può essere presentato come una coppia $((V, t), T)$, dove (V, t) è un grafo etichettato ($V = \{C_1, \dots, C_n\}$, $t : V \times V \rightarrow \mathbb{N}$) e T è un intero.

Il problema si può presentare come il linguaggio

$$\mathcal{L} = \{((V, t), T) \mid \exists \text{ cammini } p, q \text{ in } (V, t) \text{ t.c. } p, q \text{ sono cicli di peso } \leq T \text{ che coprono tutti i } V\}$$

Dati (V, t) , T , p , q (cioè con il certificato — i due cicli proposti), questo può essere verificato in tempo polinomiale.

Quindi il problema è in **NP**.

Punto 2: è NP-completo?

Sì. Possiamo fare una riduzione da HAMILTONIAN-CYCLE.

Richiamo su HAM.CYCLE. Dato un grafo (V, E) , esiste un ciclo che passa per ogni nodo esattamente una volta (tornando al punto di partenza)? È NP-completo (già usato in precedenza come sorgente di riduzioni, ad es. per TSP).

La costruzione.

Un'istanza di HAM.CYCLE (V, E) , con $V = \{v_1, \dots, v_n\}$, viene mappata in un'istanza di $\mathcal{L}$ costruita così:

- **una copia completata di V :** stesso insieme di nodi $v_1, \dots, v_n$, ma ora con *tutti* gli archi possibili tra ogni coppia — gli archi che erano in E hanno peso 2, quelli che non c'erano hanno peso 3;
- **due nodi aggiuntivi w_1, w_2 ,** collegati a tutti i nodi $v_1, \dots, v_n$ con archi di peso $2n + 1$;
- si pone $T = 2n$.

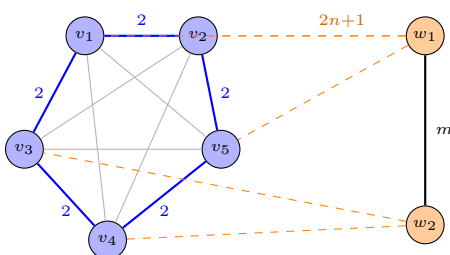


Diagramma semplificato. Per non appesantire la figura, sono mostrati solo alcuni degli archi “pesanti” $2n + 1$ tra w_1, w_2 e i v_i (nella costruzione reale, **ogni** v_i è collegato sia a w_1 che a w_2). Gli archi grigi sottili rappresentano tutti gli altri archi della copia completata di V (di peso 3, non evidenziati singolarmente).

Perché questi pesi specifici forzano la divisione tra i due camion.

Il punto cruciale: un *singolo* arco che tocca w_1 o w_2 verso un qualsiasi v_i pesa $2n + 1$. Ma il budget totale è $T = 2n$:

$$2n + 1 > 2n = T$$

Quindi basta un solo arco di questo tipo per sfiorare già da solo l'intero budget T , indipendentemente da qualsiasi altro arco nel ciclo. **Nessun ciclo può toccare sia w_1 (o w_2) sia un qualsiasi v_i .**

Conseguenza. Se un ciclo deve includere w_1 o w_2 , non può includere nessun v_i . L'unico modo per “coprire” w_1 e w_2 è un ciclo che stia solo tra loro due (collegamento diretto, di peso ragionevole m , percorso avanti e indietro). Questo è il compito del **primo camion**.

Il **secondo camion** deve invece visitare tutti gli n nodi $v_1, \dots, v_n$: il suo ciclo usa esattamente n archi. Con budget $T = 2n$:

- se tutti gli n archi pesano 2 (tutti presenti in E): costo totale = $2n = T$ ✓, esatto per un pelo;
- se anche un solo arco pesa 3 (non presente in E): costo totale $\geq 2(n-1) + 3 = 2n + 1 > T$ 55, troppo.

Il secondo camion riesce a completare il giro **se e solo se** può usare esclusivamente archi di peso 2 — cioè se e solo se esiste un **ciclo hamiltoniano** nel grafo originale (V, E) .

Punto 3: è in P? in FP? in EXPTIME? in PSPACE?

- **In P?** → “We don’t know”: essendo NP-completo, saperlo equivarrebbe a sapere se $P = NP$.
- **In FP?** → “Has no sense! FP is a class of functions”: il nostro problema è un problema di **decisione** (sì/no), non una funzione con output più complesso — la domanda non si applica alla categoria di oggetto giusta.
- **In EXPTIME?** → Sì, perché $NP \subseteq EXPTIME$.
- **In PSPACE?** → Sì, perché $NP \subseteq PSPACE$ (un algoritmo non deterministico in tempo polinomiale può sempre essere simulato usando solo una quantità polinomiale di spazio, anche se richiedesse tempo esponenziale).

Esercizio 3

Qual è la classe di complessità di questi problemi?

1. “ M decide 2-SAT?”

Input: il codice $\lfloor M \rfloor$ di una macchina di Turing.

Output: M decide 2-SAT? (si assume fissata una codifica delle formule 2-SAT come stringhe in $\{0, 1\}^*$)

$$\mathcal{L} = \{\lfloor M \rfloor \mid M \text{ decide 2-SAT}\}$$

Si applica il **Teorema di Rice** (già visto e dimostrato in dettaglio nell’esercizio sugli 8 linguaggi $L_1, \dots, L_8$): ogni proprietà non-triviale e semantica è indecidibile.

- **È semantica?** Sì: se M, N accettano lo stesso linguaggio, $\lfloor M \rfloor \in \mathcal{L}$ se e solo se $\lfloor N \rfloor \in \mathcal{L}$ (dipende solo dal comportamento, non da come è scritto il codice).
- **È non-triviale?** Sì: ci sono $\lfloor M \rfloor \in \mathcal{L}$, e $M' \notin \mathcal{L}$.

Per il Teorema di Rice: $\mathcal{L}$ è **indecidibile**.

2. Esiste un insieme indipendente di taglia 2?

Input: un grafo (V, E) .

Output: esiste un insieme indipendente di cardinalità 2 nel grafo?

L'algoritmo.

```
input {v_1, ..., v_n}, E
For i=1 to n
  For j=1 to n
    if i!=j and (i,j) not in E then
      return 1
Return 0
```

Si prova ogni coppia di nodi distinti; se se ne trova una **senza** arco tra loro, quella coppia è già un insieme indipendente di taglia 2 — si restituisce subito “sì”.

Perché questo decide 2-IND.SET ed è polinomiale.

1. esegue $O(n^2)$ istruzioni (corrisponde a una MT in tempo polinomiale);
2. ogni istruzione gira in tempo polinomiale nell'input;
3. la dimensione dei dati intermedi (qui: nulla, non si accumula nessuna struttura) è polinomiale.

Quindi **2-INDSET** $\in$ **P**.

Esercizio 4

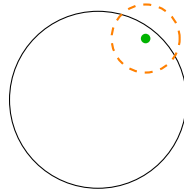
Dato lo spazio delle istanze $X = \{\text{punti su un cerchio}\} \subseteq \mathbb{R}^2$, calcolare la VC-dimension della classe di ipotesi di tutti i **dischi**. Cosa si può concludere sulla learnability dei dischi?

Curiosità. La sigla “VC” viene da **Vapnik** e **Chervonenkis**, i due matematici che hanno introdotto questo concetto.

Il collegamento con gli intervalli. Un disco, intersecato con i punti su un cerchio, si comporta in modo simile a un intervallo su una retta (cattura generalmente un arco contiguo). Ma su un cerchio un arco può “avvolgersi” attorno al bordo (non ha un inizio/fine fissi come su una retta): questo grado di libertà in più fa salire la VC-dimension da 2 (intervalli su $\mathbb{R}$) a 3 (dischi su un cerchio).

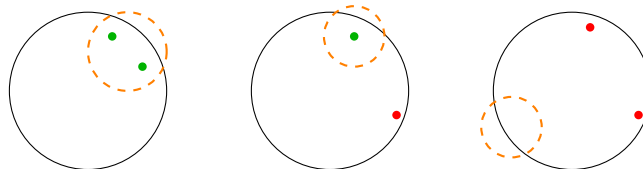
Lower bound: $VC \geq 3$

Con 1 campione.

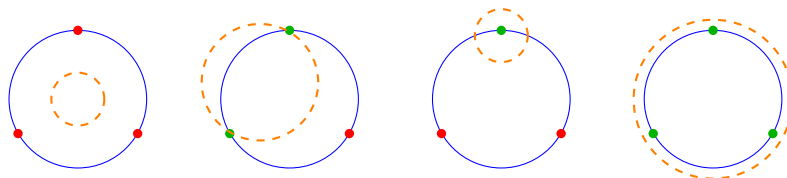


Banale: un disco che contiene il punto, uno che non lo contiene.

Con 2 campioni.



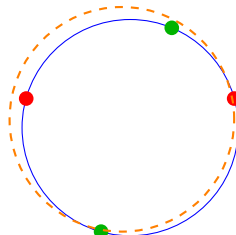
Con 3 campioni: 4 configurazioni rappresentative.



Con 3 punti ci sono $2^3 = 8$ etichettature possibili, ma per **simmetria** (ruotando/riflettendo la configurazione) non serve disegnarle tutte esplicitamente: alcuni casi rappresentativi, combinati con le simmetrie del cerchio, coprono implicitamente gli altri.

... quindi $VC\text{-dim} \geq 3$.

Upper bound: con 4 campioni c'è sempre un'etichettatura irrealizzabile



Perché nessun disco può realizzare questa etichettatura. I 4 punti sono disposti in ordine alternato attorno al cerchio: verde, rosso, verde, rosso. Un disco, intersecato con il cerchio, produce sempre **un solo arco contiguo** — non può “saltare” da un punto all’altro lasciando fuori qualcosa in mezzo. Per catturare solo i 2 punti verdi servirebbero **due archi separati** (uno per ciascun verde, saltando i rossi), ma un disco ne produce uno solo.

... quindi $VC\text{-dim} = 3$.

Conseguenze

Conseguenza: la classe dei dischi è **PAC-learnable** (sul cerchio) + il teorema del corso ci dice quanti dati servono.

Questo si ricollega direttamente al primo teorema PAC-learning visto (quello dei rettangoli, con $m \geq \frac{4}{\epsilon} \ln \frac{4}{\delta}$): sapere che $VC(\text{dischi}) = 3$ è esattamente l’ingrediente che serve per applicare un teorema generale di PAC-learnability basato sulla VC-dimension, e ottenere un bound analogo sul numero di campioni necessari per imparare bene la classe dei dischi.

Esercizio 5

Dato lo spazio delle istanze $X = \mathbb{R} \times \{0\}$ (cioè l’asse orizzontale), calcolare la VC-dimension della classe di ipotesi di tutti i (semipiani superiori delimitati da) funzioni polinomiali

$$x \mapsto \sum_{i=0}^N a_i x^i, \quad \text{per } a_0, \dots, a_N \in \mathbb{R}$$

Cosa vuol dire “semipiano superiore delimitato da un polinomio”. L’ipotesi “positiva” per un punto x sull’asse orizzontale è: *il polinomio, calcolato in x , vale ≥ 0* (il punto sta “sopra o sull’asse” rispetto alla curva del polinomio). Ogni scelta di coefficienti $a_0, \dots, a_N$ definisce una diversa ipotesi (un diverso polinomio, quindi un diverso confine tra “sopra” e “sotto”).

L’ingrediente chiave: il Teorema di interpolazione

Per ogni $N \in \mathbb{N}$, punti $x_0, \dots, x_N \in \mathbb{R}$, e etichette $y_0, \dots, y_N \in \{-1, 1\}$, esistono coefficienti $a_0, \dots, a_N$ tali che

$$\forall 0 \leq j \leq N, \quad \sum_{i=0}^N a_i x_j^i = y_j$$

(questo è il **Teorema di interpolazione** polinomiale)

Cosa dice davvero questo teorema, in parole povere. Dati $N + 1$ punti **distinti** $x_0, \dots, x_N$ sull'asse, e $N + 1$ valori target *qualsiasi* $y_0, \dots, y_N$ (ciascuno $+1$ o -1 , a piacere), esiste **sempre** un polinomio di grado al più N che, calcolato in ciascun x_j , restituisce *esattamente* il valore y_j richiesto. Non importa quali siano i punti o quali valori vogliamo raggiungere: un polinomio adatto esiste sempre, basta scegliere il grado N abbastanza alto (grande quanto il numero di punti meno uno).
 È lo stesso principio che forse conosci come “per $N + 1$ punti passa un unico polinomio di grado N ” (es. per 2 punti passa una retta, per 3 punti passa una parabola, ecc.) — qui però non ci interessa che il polinomio passi per punti (x_j, y_j) nel senso classico: vogliamo che il valore del polinomio in x_j sia *esattamente* y_j , cioè $+1$ oppure -1 .

Dal teorema di interpolazione alla shattering

Questo significa in particolare che $(x_j, 0)$ sta nel semipiano superiore delimitato dal polinomio **esattamente** quando $y_j = -1$...

Attenzione, controlliamo il segno con calma: se il polinomio calcolato in x_j restituisce $\sum_i a_i x_j^i = y_j$, dove $y_j \in \{-1, +1\}$:

- se $y_j = +1$: il valore del polinomio in x_j è $+1 \geq 0 \Rightarrow$ il punto $(x_j, 0)$ sta **sopra o sull'asse**, cioè *dentro* il semipiano superiore;
- se $y_j = -1$: il valore del polinomio in x_j è $-1 < 0 \Rightarrow$ il punto **non** sta nel semipiano superiore, cioè *escluso*.

Quindi: **etichetta** $+1$ corrisponde a “punto catturato” (dentro l'ipotesi); **etichetta** -1 corrisponde a “punto escluso”.

Perché questo dimostra la shattering, per qualsiasi N . Ricorda la definizione di shattering: un insieme S di punti è shattered da $\mathcal{H}$ se, per **ogni** possibile etichettatura di S , esiste un'ipotesi in $\mathcal{H}$ che la realizza. Il teorema di interpolazione garantisce esattamente questo: qualunque insieme di punti $x_0, \dots, x_N$ e qualunque etichettatura target $y_0, \dots, y_N$ tu scelga, esiste **sempre** un polinomio (cioè un'ipotesi nella nostra classe $\mathcal{H}$) che la realizza esattamente. Questa è precisamente la definizione di shattering, applicata a un insieme di $N + 1$ punti **qualsiasi**.

Conclusione

Poiché abbiamo shattered qualsiasi insieme $x_0, \dots, x_N$, per **qualsiasi** N (non c'è un valore di N oltre il quale il teorema smette di funzionare — vale sempre, basta scegliere un polinomio di grado abbastanza alto):

$$VC(\mathcal{H}) = \infty$$

Confronto con l'Esercizio 4 (unioni finite di intervalli). Questo è esattamente lo stesso schema logico di quell'esercizio: lì, non ponendo limiti al numero di intervalli, potevamo sempre costruire abbastanza intervallini per catturare qualsiasi sottoinsieme. Qui, non ponendo limiti al grado N del polinomio, possiamo sempre costruire un polinomio di grado abbastanza alto per realizzare qualsiasi etichettatura. In entrambi i casi, la “libertà illimitata” nel numero di parametri disponibili (intervalli, o grado del polinomio) porta a VC-dimension infinita.

Perché è sorprendente. Normalmente ci si aspetterebbe che “più parametri liberi = più potere espressivo” (nei polinomi, per shatterare $N+1$ punti servono $N+1$ coefficienti — il numero di parametri cresce insieme alla dimensione dell'insieme da shatterare). Ma con le funzioni seno, un **singolo** numero reale ω (la frequenza) basta già a rendere la classe infinitamente espressiva: questo è possibile perché ω , essendo un numero reale, può codificare al suo interno informazione arbitrariamente complessa (nella sua espansione in cifre decimali/binarie) — un solo parametro “continuo” può nascondere infinita informazione discreta.

Recap: Teoremi

1. **Indecidibilità dell'Halting Problem.** La funzione $halt(\alpha, x)$ (dice se $M_\alpha(x)$ termina) non è calcolabile.

Idea chiave: per assurdo, si costruisce M_{uc} usando un ipotetico M_{halt} , e si applica M_{uc} a se stessa: in entrambi i casi (termina/non termina) si arriva a una contraddizione logica diretta.

2. **Teorema di Rice.** Ogni proprietà non-triviale e semantica di un linguaggio riconosciuto da una macchina di Turing è indecidibile.

Idea chiave: si dimostra per riduzione dall'Halting Problem, sfruttando il fatto che una proprietà semantica non può distinguere macchine equivalenti nel comportamento.

3. **Teorema di Eulero.** Esiste un cammino euleriano in un grafo se e solo se zero o due vertici hanno grado dispari, e il grafo è connesso.

Idea chiave: questa condizione è verificabile in tempo polinomiale (basta contare i gradi e controllare la connettività), quindi $EULER.PATH \in P$.

4. **Teorema** $P \subsetneq EXP$. L'inclusione $P \subseteq EXP$ è stretta.

Idea chiave: $P \subseteq DTIME(2^n) \subsetneq DTIME(2^{2n}) \subseteq EXP$, dove il passaggio stretto centrale segue dal Teorema della Gerarchia.

5. **Lemma:** W è vertex cover $\iff V \setminus W$ è insieme indipendente.

Idea chiave: è una riscrittura logica diretta via De Morgan: “ $v \in W$ o $w \in W$ ” $\equiv$ “non ($v \in V \setminus W$ e $w \in V \setminus W$)”.

6. **Teorema generale su NP-completezza (3 parti).**

- (a) $\leq_p$ è un preordine (riflessiva e transitiva).
- (b) Se L è NP-hard e $L \in P$, allora $P = NP$.
- (c) Se L è NP-completo, allora $L \in P \iff P = NP$.

Idea chiave (b): ogni $H \in NP$ si riduce a L (per NP-hardness); se $L \in P$, allora anche $H \in P$ (comporre riduzione+algoritmo polinomiale resta polinomiale); quindi $NP \subseteq P$.

7. **Lemma di Sauer-Shelah.** Se $VC(\mathcal{H}) = d$, il numero di etichettature distinte realizzabili su un insieme di n punti è al più $\sum_{i=0}^d \binom{n}{i}$.

Idea chiave: bassa VC-dimension implica che il numero di comportamenti possibili cresce solo polinomialmente in n , non esponenzialmente.

8. **Teorema:** $\mathcal{H} \subseteq \mathcal{H}' \Rightarrow VC(\mathcal{H}) \leq VC(\mathcal{H}')$.

Idea chiave: se $\mathcal{H}$ shattera S , anche $\mathcal{H}'$ lo fa, avendo a disposizione tutte le ipotesi di $\mathcal{H}$ e potenzialmente altre.

9. **Teorema:** $VC(\mathcal{H}), VC(\mathcal{H}') \leq VC(\mathcal{H} \cup \mathcal{H}') \leq VC(\mathcal{H}) + VC(\mathcal{H}') + 1$.

Idea chiave: la parte sinistra segue dal teorema precedente; la parte destra usa Sauer-Shelah, confrontando 2^n (crescita esponenziale) con la somma di due termini polinomiali.

10. **Teorema PAC-learning dei rettangoli.** Se $m \geq \frac{4}{\varepsilon} \ln \frac{4}{\delta}$, allora

$$\Pr [\text{error}_{D,T}(A_{BFT}(T(D))) < \varepsilon] > 1 - \delta.$$

Idea chiave: si divide la “cornice” d’errore in 4 regioni di probabilità $\varepsilon/4$ ciascuna; se il training set tocca tutte e 4, l’errore è $\leq \varepsilon$; si limita con m grande la probabilità di mancarne anche una.

11. **Teorema: le congiunzioni di letterali sono efficientemente PAC-learnabili.**

Idea chiave: si limita la probabilità che restino “letterali dannosi” dopo m campioni; senza letterali dannosi, l’errore resta $\leq \varepsilon$ per costruzione.

12. **Teorema: i 3-term DNF non sono efficientemente PAC-learnabili, a meno che $RP = NP$.**

Idea chiave: riduzione da 3-colorabilità dei grafi (NP-completo); un ipotetico algoritmo di apprendimento efficiente per 3-term DNF permetterebbe di decidere 3-colorabilità in tempo polinomiale randomizzato.

13. **Teorema di interpolazione polinomiale.** Per ogni $N + 1$ punti distinti e $N + 1$ valori target, esiste un polinomio di grado al più N che li realizza esattamente.

Idea chiave: usato per dimostrare che la classe dei polinomi ha VC-dimension infinita: qualunque etichettatura, su qualunque insieme di punti, è sempre realizzabile.

14. **Teorema: 2SAT $\in$ P.**

Idea chiave: si costruisce il grafo delle implicazioni; F è insoddisfacibile $\iff$ esiste una variabile X nella stessa componente fortemente connessa di $\neg X$ — verificabile in tempo polinomiale.